Linear-Communication ACSS with Guaranteed Termination and Lower Amortized Bound

Chengyi Qin[1,2,3], Mingqiang Wang[1,2⋆], and Haiyang Xue[3]

[1] School of Mathematics, Shandong University
QCY521111@163.com, wangmingqiang@sdu.edu.cn
[2] State Key Laboratory of Cryptography and Digital Economy Security, Jinan 250100, China
[3] Singapore Management University
haiyangxue@smu.edu.sg

Abstract. An Asynchronous Complete Secret Sharing (ACSS) protocol enables a dealer to distribute N Shamir shares such that all parties eventually receive their shares over an asynchronous network. It serves as a fundamental building block for asynchronous Secure Multiparty Computation (MPC) and Byzantine Agreement. In this work, we focus on the statistically secure ACSS with optimal resilience ($t < n/3$). Recently, Ji, Li, and Song [CRYPTO'24] proposed the first ACSS protocol with amortized linear communication. However, their scheme lacks guaranteed termination, and they identified the construction of a linear-communication ACSS with guaranteed termination as an open problem. Furthermore, their protocol requires a large amortized bound of $N = \Omega(n^{11}\kappa)$, where n is the number of parties and κ is the size of the secret.
In this work, we resolve the open problem and significantly reduce the amortized bound by presenting a linear-communication ACSS protocol with guaranteed termination and a lower bound of $N = \Omega(n^4 + n\kappa)$. Our ACSS protocol can be directly applied to asynchronous MPC protocols, ensuring both guaranteed termination and improved communication per multiplication gate, as well as to asynchronous Byzantine Agreement.

Keywords: ACSS · Guaranteed Termination · Linear Communication · Asynchronous Communication Model.

1 Introduction

Verifiable Secret Sharing (VSS) schemes [24,42] enable a dealer—a party holding a secret—to distribute the secret among multiple parties in such a way that each party can verify the validity and correctness of its share, where a small subset of the parties, including the dealer, may be malicious. The secret-sharing process is verifiable because each party can verify the validity and correctness of its share. An important property of VSS protocols is *completeness*, which

⋆ Corresponding author

guarantees that every honest party receives its share of the secret. A VSS protocol that satisfies this property is known as a *Complete Secret Sharing* (CSS) scheme. Research on CSS remains active due to its wide-ranging applications in secure multiparty computation (MPC)[34,2], Byzantine Agreement algorithms (BA) [33,41], Blockchain [31], Threshold signatures [8], etc.

Most classical CSS protocols (e.g., [35,14,26,3]) assume a synchronous network model, where parties share a global clock and message delays have known upper bounds—so timeouts are treated as indicators of sender misbehavior. However, real-world networks are inherently asynchronous, as modeled in the asynchronous communication model [12,19], where there is no global clock and messages may be delayed arbitrarily, with only eventual delivery guaranteed. This discrepancy motivates the study of asynchronous CSS, which aims to achieve complete secret sharing in asynchronous networks.

Asynchronous Complete Secret Sharing. Roughly, an asynchronous complete secret sharing (ACSS) protocol allows a dealer to distribute a set of degree-t Shamir sharings among n parties over an asynchronous network, while satisfying the following properties: i) If the dealer is honest, all honest parties eventually receive correct shares. ii) If the dealer is corrupted, then either no honest party accepts its shares, or all honest parties eventually receive correct shares. iii) If any honest party successfully terminates the sharing protocol, then, regardless of the dealer's behavior, all other honest parties will also eventually terminate.

Compared to CSS protocols in the synchronous setting, ACSS typically requires a stricter corruption threshold for information-theoretic security. In the synchronous setting, perfect security is achievable with up to $t < n/3$ corrupt parties [11], and statistical security can be attained with $t < n/2$ [43]. In contrast, perfectly secure ACSS is possible only if $t < n/4$ [10,19], and statistically secure ACSS protocols exist if and only if $t < n/3$ [13,23]. We say that (A)CSS achieves optimal resilience if it remains secure against the largest possible number of corrupt parties.

In this paper, we focus on the communication complexity of statistically secure ACSS, aiming for linear communication and optimal resilience (i.e., $n = 3t + 1$). In the synchronous setting with optimal resilience (i.e., $n = 2t + 1$), it has been known for many years that CSS can be achieved with linear communication complexity $\mathcal{O}(n\kappa)$ bits per share [14,26], where κ denotes the size of a field element. In contrast, a linear-communication ACSS protocol with optimal resilience (i.e., $n = 3t + 1$) was discovered only recently by Ji, Li, and Song [36].

Ji, Li, and Song [36] proposed the first ACSS protocol with communication complexity $\mathcal{O}(Nn\kappa + n^{12}\kappa^2)$ bits for N amortized Shamir sharings. Their construction leverages novel ideas such as packed secret sharing using $(t, 2t)$-degree bivariate polynomials $F(x, y)$, asynchronous information-checking protocol, and authentication tags. However, the protocol has two notable drawbacks: it lacks guaranteed termination and requires a large amortization parameter N.

Guaranteed Termination. As noted in point iii) above, guaranteed termination is an essential property of ACSS. The termination property directly impacts the protocols built on top of ACSS, such as liveness in Byzantine consen-

sus [17,41,1], universally composable (UC) security frameworks [21], and secure multiparty computation (MPC). Without guaranteed termination, higher-level protocols may become stuck during initialization or output phases, rendering the entire system "safe but unavailable". This introduces a liveness problem in Byzantine agreement [17,41,1] and UC security proofs [21].

Although, as observed in [27] and [36, Appendix F], MPC protocols built on non-terminating ACSS can still achieve termination by invoking a consensus protocol on the final output, this approach introduces overhead and complicates the design of the higher-level protocol. Thus, Ji, Li, and Song [36], in Appendix E, proposed an ACSS protocol that guarantees termination at the cost of $\mathcal{O}(Nn^2\kappa + n^{13}\kappa^2)$ bits communication, resulting in an amortized communication cost of at least $\mathcal{O}(n^2\kappa)$ bits. The protocol of Cai, Qin, and Wang [18] also achieves guaranteed termination and optimal resilience. Its total communication complexity is $\mathcal{O}\big(Nn^2\kappa+n^4\kappa^2\big)$ bits, yielding an amortized communication cost of $\mathcal{O}(n^2\kappa)$ bits. In contrast, deterministic constant-round asynchronous consensus is impossible (e.g., with crash failures), and practical consensus protocols may incur additional coordination/latency. Accordingly, our goal is to avoid relying on consensus solely to guarantee ACSS completeness, and to achieve termination directly within linear-communication ACSS in the information-theoretic asynchronous setting.

Amortization Parameter N. From the result of [36], it is easy to see that achieving truly linear amortized communication requires $N = \Omega(n^{11}\kappa)$, due to the $n^{12}\kappa^2$ term in the complexity. Ideally, the smaller the required N, the better. The reduction would not only relax the conditions required for achieving linear communication but also decrease the fixed communication overhead in the amortized setting.

Motivation and Open Question. Thus, in this paper, we aim to address the following open question posed in [36], while keeping the amortization parameter N as small as possible:

> *Is it possible to design an ACSS protocol that achieves (amortized) linear communication complexity, guarantees termination, and attains optimal resilience $n = 3t + 1$?*

1.1 Our Contribution

In this work, we answer the posed question affirmatively by presenting a novel ACSS protocol that achieves linear communication complexity while guaranteeing termination.

Our main contribution is the design of an ACSS protocol with a total communication complexity of $\mathcal{O}(Nn\kappa + n^5\kappa + n^2\kappa^2)$ bits for distributing N degree-t Shamir sharings, where κ denotes the size of a field element. As a result, the proposed protocol achieves an **amortized communication complexity** of $\mathcal{O}(n\kappa)$ bits per sharing when $N = \Omega(n^4 + n\kappa)$, while also ensuring protocol **termination**, thereby resolving the open question raised in [36]. Formally, our result is stated in the following theorem.

Theorem 1. *For a finite field $\mathbb{F}$ of size $2^{\Theta(\kappa)}$, where κ is the security parameter, we propose a fully malicious, statistically secure ACSS protocol resilient against up to $t < n/3$ corrupted parties. This protocol facilitates the sharing of N degree-t Shamir sharings over $\mathbb{F}$ with a communication complexity of $\mathcal{O}(Nn\kappa + n^5\kappa + n^2\kappa^2)$ bits, while incurring a statistical error of $\mathcal{O}((Nn+n^3\kappa^2)/2^\kappa)$. The protocol achieves a round complexity of $\mathcal{O}(1)$ rounds, leveraging point-to-point channels, along with $\mathcal{O}(1)$ rounds of invocations to the ACast protocol in [16].*

Table 1 presents a comparison between our ACSS protocol and those in [27], [36] and [18]. The protocol in [27] (resp. [18]) guarantees termination but only achieves an amortized communication complexity of $\mathcal{O}(n^3\kappa)$ (resp. $\mathcal{O}(n^2\kappa)$). The protocols in [36] achieve linear communication complexity but either do not guarantee termination or require $\mathcal{O}(n^2\kappa)$-bit amortized communication to do so. In contrast, our protocol achieves both linear communication and guaranteed termination, with a lower amortization threshold N.

Table 1: Comparison of ACSS Protocols with (amortized) linear communication.

Reference	Comm.	Amortized Comm.	$N \geq$	Termination
[27]	$Nn^3\,\kappa + n^4\,\kappa^2 + n^5$	$\mathcal{O}(n^3\kappa)$	$\Omega(n\,\kappa + n^2/\kappa)$	✓
[36]	$Nn\,\kappa + n^{12}\,\kappa^2$	$\boldsymbol{\mathcal{O}(n\kappa)}$	$\Omega(n^{11}\,\kappa)$	×
[36, App. E]	$Nn^2\,\kappa + n^{13}\,\kappa^2$	$\mathcal{O}(n^2\kappa)$	$\Omega(n^{11}\,\kappa)$	✓
[18]	$Nn^2\kappa + n^4\kappa^2$	$\mathcal{O}(n^2\kappa)$	$\Omega(n^2\,\kappa)$	✓
This work	$\boldsymbol{Nn\kappa + n^5\kappa + n^2\kappa^2}$	$\boldsymbol{\mathcal{O}(n\kappa)}$	$\boldsymbol{\Omega(n^4 + n\kappa)}$	✓

Table notes: "Comm." denotes the total communication (in bits) required to share N secrets. "Amortized Comm." refers to the communication cost per secret, averaged over N secrets (in bits). "$N \geq$" indicates the minimum value of N required to achieve the stated amortized communication. "Termination" specifies whether the protocol guarantees that all honest parties eventually terminate.

We realize our proposed protocol with the following techniques. Refer to Section 2 for a detailed discussion on these techniques.

- We propose an Asynchronous Information-Checking Protocol (AICP), inspired by the classical information-checking protocol from [43], which we refer to as Asynchronous Message-Checking (AMC). Similar to the AICP protocol in [38], AMC operates in an asynchronous setting but additionally supports linear homomorphism and multiple receivers. Furthermore, it achieves an amortized communication complexity of $\mathcal{O}(1)$ bits per bit of information, with a total communication cost significantly lower than that of the protocol in [36].
- We leverage our AMC and further ensure completeness and termination by exploiting the favorable properties of the symmetric polynomial $V(x, y) =$

$F(x,y)+F(y,x)$, i.e., satisfying $V(x,y)=V(y,x)$, where $F(x,y)$ is a bivariate polynomial of degree $(t,2t)$, as used in [36].

- We further reduce the amortized communication to linear using *algebraic fingerprinting*, which checks equality of polynomial-encoded data via a random linear combination evaluated at a random point. Specifically, we collapse the m symmetric polynomials $V_\ell(x,y)$ into a single aggregate $V(x,y)=\sum_{\ell=1}^{m} d^\ell V_\ell(x,y)$, under a public coin d, and verify this aggregate. By the Schwartz–Zippel lemma, soundness is preserved and the lower bound on N remains unchanged.

Our ACSS protocol can be readily used in asynchronous MPC and asynchronous Byzantine agreement protocols.

Application in asynchronous MPC (AMPC). Building on the $\mathcal{O}(n\kappa)$-bit-per-gate compiler of [34], we can obtain the information-theoretic AMPC with both linear communication and optimal resilience. We maintain the same circuit-size and depth-dependent communication costs $\mathcal{O}(|C|\,n\kappa + D\,n^2\kappa)$ bits, while cutting the secret-sharing overhead from $\mathcal{O}(n^{14}\kappa^2)$ bits in [34] to $\mathcal{O}(n^4\kappa^2)$ bits, all with linear communication and guaranteed termination.

Lemma 1. *([34]) Let $n=3t+1$. For any circuit C with size $|C|$ and depth D, there exists a fully malicious asynchronous MPC protocol capable of securely evaluating the circuit in the presence of up to t corrupted parties. The protocol guarantees output delivery in the $\mathcal{F}_{\mathsf{ACSS}}$-hybrid model. Its communication complexity is given by $\mathcal{O}(|C|\cdot n\kappa + D\cdot n^2\kappa + n^8\kappa)$ bits, alongside $\mathcal{O}(n^2)$ invocations of $\mathcal{F}_{\mathsf{ACSS}}$ to facilitate the sharing of $\mathcal{O}(|C|)$ degree-t Shamir sharings.*

Corollary 1. *Given $n=3t+1$, for any circuit C of size $|C|$ and depth D, we construct a fully malicious information-theoretic asynchronous MPC protocol that tolerates up to t corrupted parties and ensures guaranteed output delivery. The total communication complexity is $\mathcal{O}(|C|\cdot n\kappa + D\cdot n^2\kappa + n^8\kappa + n^4\kappa^2)$ bits.*

Application in asynchronous Byzantine agreement (ABA). ACSS also serves as the linchpin for optimally resilient ABA, where n parties must agree despite $t<n/3$ malicious colluders, satisfying both agreement and liveness (termination). Classical ABA designs [32,33] reduce common-coin generation to ACSS; hence nearly all optimal-fault-tolerant ABA protocols (e.g. [4,40]) follow this reduction. Crucially, ACSS's termination guarantees propagate to ABA, ensuring that once a sufficient quorum of honest parties completes the share phase, all will terminate with a common decision.

1.2 Related Works

For existing research on ACSS with efficient communication complexity, the protocols are generally categorized into three distinct security settings: Statistical Security, Perfect Security, and Computational Security.

Statistical Security. In the setting of statistical security against up to $t<n/3$ corrupted parties, a series of works [15,23,39,38,13,27,36,18] have progressively

optimized the communication complexity of ACSS protocols. Among these, the most advanced result [36] presents the first protocol to achieve an amortized communication complexity of $\mathcal{O}(n\kappa)$ bits per sharing. However, as discussed above, this protocol does not guarantee termination.

Perfect Security. In the setting of perfect security, it has been established that $t < n/4$ is a fundamental constraint [12]. Over the years, numerous studies [46,10,25,41,26] have refined the communication complexity of ACSS under this constraint. The state-of-the-art protocol [26] achieves linear communication complexity, requiring only $\mathcal{O}(n\kappa)$ bits per sharing.

Computational Security. For computational security against $t < n/3$ corrupted parties, ACSS protocols with linear communication complexity have been proposed in [5], leveraging discrete logarithms and pairings, and in [47], which utilizes fast Reed-Solomon interactive oracle proofs (RS-IOP). A sequence of works [5,45,7,47] has progressively reduced the communication overhead of ACSS protocols in this setting. Notably, the work [45] employs lightweight cryptographic techniques, including collision-resistant hash functions and pseudorandom functions, achieving an amortized communication complexity of $\mathcal{O}(n^2\kappa)$ bits per sharing.

2 Technical Overview

This section provides a concise overview of the core techniques employed in this work. We assume each participating party has access to a point-to-point asynchronous and secure communication network.

2.1 Review of Ji, Li and Song's work [36]

Ji, Li and Song begin by packing $t+1$ Shamir secret sharings into a bivariate polynomial $F(x, y)$ of degree $(t, 2t)$. For $i \in [n]$ and distinct $\alpha_i \in \mathbb{F}$, they define

- the column polynomials as $g_i(y) = F(\alpha_i, y)$, each of degree $2t$,
- and the row polynomials as $f_i(x) = F(x, \alpha_i)$, each of degree t.

A key property of $F(x, y)$ is that any set of $t+1$ honest parties with access to their respective column polynomials can reconstruct the entire bivariate polynomial. In contrast, even if t corrupted parties collude and reveal both their row and column polynomials, they still require $t+1$ additional evaluations on a column polynomial of degree $2t$ to reconstruct F. Their key insight is that $t+1 = \mathcal{O}(n)$ Shamir sharings can be efficiently packed into a single instance of $F(x, y)$, potentially reducing the amortized communication cost by a factor of $\mathcal{O}(n)$. The process roughly runs as follows:

1. **Sharing**: A dealer D sends to each P_i the column polynomial $g_i(y) = F(\alpha_i, y)$. On collecting signatures from $2t+1$ parties, D broadcasts the set $\mathcal{M}$, identifying these $2t+1$ parties, among which at least $t+1$ are honest.

2. **Row polynomial reconstruction**: Each party $P_i \in \mathcal{M}$ sends $g_i(\alpha_j)$ to P_j for all $j \in [n]$. If P_j can distinguish the $t+1$ correct shares (potentially sent by honest parties) from incorrect ones, it can reconstruct polynomial $f_j(x)$.
3. **Column polynomial reconstruction**: Each party P_j for $j \in [n]$ sends $f_j(\alpha_o)$ to P_o. If P_o can distinguish the $2t+1$ correct shares (potentially sent by honest parties) from the incorrect ones, it can reconstruct the column polynomial $g_o(x)$, and finally recover the shared secrets.

The main challenge lies in distinguishing correct shares from incorrect ones. They address this using the Asynchronous Packed Information-Checking Protocol (APICP) together with authentication tags.

APICP. APICP can be viewed as a signature-like protocol involving a signer (Sig), an intermediary (I), and a receiver (R). It is an extension of the earlier Asynchronous Information-Checking Protocol (AICP) [38]. To achieve linear communication complexity, Ji, Li, and Song [36] augment AICP with two key properties: linear homomorphism and support for multi-revealing.

However, their APICP instantiation embeds secrets into polynomials of degree $L + t(T+1)^2\kappa$, where $L = \Omega(n^7\kappa)$ and $T = \Omega(n^3\kappa)$. This results in a communication cost of $\mathcal{O}(n^{11}\kappa^2)$ bits, contributing a term of $\mathcal{O}(n^{12}\kappa^2)$ to the overall communication complexity (since n parties are involved). Consequently, to achieve an amortized communication of $\mathcal{O}(n\kappa)$ bits per secret, the protocol requires $N = \Omega(n^{11}\kappa)$ sharings.

Authentication Tags. The concept of authentication tags was first introduced in [14]. Ji, Li, and Song [36] extended it to asynchronous settings and demonstrated that, with pair-wise authentication tags, any set of $2t+1$ parties can assist the remaining t parties in recovering their shares, thereby enabling ACSS.

However, this technique relies on a random challenge r generated by honest parties. In the final phase of their protocol, honest parties must remain online to provide row-polynomial evaluations $f_j(x)$ along with the corresponding authentication tags. If any of these parties goes offline, lagging participants may be unable to collect the necessary $t+1$ valid points required to reconstruct their shares, thereby violating the guaranteed termination property.

2.2 Our Approach

Our approach also begins by embedding N Shamir secret sharings into $m \approx m' = \lceil N/(t+1) \rceil$ bivariate polynomials $F_\ell(x, y)$ of degree $(t, 2t)$, for $\ell \in [m]$, where each encodes $t+1$ secret shares[4]. Let $g_{\ell,i}(y) = F_\ell(\alpha_i, y)$ and $f_{\ell,i}(x) = F_\ell(x, \alpha_i)$ be the column and row polynomials, respectively. Following the approach of [36], we begin with Sharing, followed by row/column polynomial reconstruction.

– A dealer D sends to each P_i the column polynomial $g_{\ell,i}(y) = F_\ell(\alpha_i, y)$ and the row polynomial $f_{\ell,i}(x) = F_\ell(x, \alpha_i)$, and receives a signature from P_i via

[4] Actually, $m = m' + 2n^3$ to support the UC security proof. However, to simplify the presentation, we assume $m \approx m'$ here since it does not affect the analysis.

some APICP. After receiving signatures from $2t+1$ parties, D broadcasts the set $\mathcal{M}$, identifying these $2t+1$ parties.

As said before, during row/column polynomial reconstruction, each party P_j faces the challenge of distinguishing correct shares from incorrect ones, or verifying the correctness of the received shares (in the Completion phase of both [36] and our protocol). Pairwise authentication tags are used in [36], which we aim to avoid.

We begin with the correctness check in row polynomial reconstruction. Before reconstruction, each party $P_i \in \mathcal{M}$ is required to aggregate its column polynomial evaluations for P_j, namely $\{g_{\ell,i}(\alpha_j)\}_{\ell\in[m]}$, into a commit polynomial $\Delta_{i\to j}(r) = \sum_{\ell=1}^{m} g_{\ell,i}(\alpha_j)\cdot r^{\ell}$ This commit polynomial with a zero constant term then allows P_j to later verify the correctness of the values $g_{\ell,i}(\alpha_j)$ during row polynomial reconstruction.

AcceptPoly via Asynchronous Message-Checking (AMC). While the commit polynomial removes the need for authentication tags, it does not fully resolve the challenge, as the correctness of the commit polynomial itself must still be ensured. To address this, we introduce a new protocol, AcceptPoly, building upon an APICP scheme. However, we remark that directly using the APICP protocol from [36] still suffers from the drawback of requiring $N = \Omega(n^{11}\kappa)$.

Instead, we propose a novel signature/verification mechanism tailored to AcceptPoly, called *Asynchronous Message-Checking* (AMC). AMC replaces APICP's pointwise checks with *batched* and *masked* tests, while operating under a significantly lighter parameter regime, namely $L = n$ and $T = 2n$. AMC consists of two phases. The initialization phase (denoted Init) binds secret vectors $\boldsymbol{s}_\ell = (s_{\ell,1}, \ldots, s_{\ell,n})$ for $\ell \in [m]$. The revelation phase (denoted Rev), upon receiving from the designated receiver P_j (with $j \in [n]$) the vector $\boldsymbol{c} = (c, c^2, \ldots, c^m)$ for some $c \in \mathbb{F}$, outputs $\sum_{\ell=1}^{m} c^{\ell} s_{\ell,j}$ to P_j. Details on the construction of AMC can be found in Section 4.

The AcceptPoly, which leverages AMC, operates roughly as follows:

- Bind the values $g_{\ell,i}(\alpha_j)$ from each $P_i \in \mathcal{M}$ to $s_{\ell,j} = g_{\ell,i}(\alpha_j)$ for $\ell \in [m]$, $j \in [n]$ by invoking the Init phase of AMC with the help of an intermediary, i.e., the dealer.
- To account for parties that may remain offline indefinitely, all currently online parties proactively assist each offline party by jointly sampling a random challenge c, and query AMC to obtain the authenticated evaluation at c, i.e., $\boldsymbol{g}_{\mathsf{Sender}} = (g_{\mathsf{Sender}}(\alpha_1), \ldots, g_{\mathsf{Sender}}(\alpha_n))$, where $g_{\mathsf{Sender}}(\alpha_j) = \sum_{\ell=1}^{m} c^{\ell} s_{\ell,j}$;
- Each acceptor P_j checks: (i) the received value $g_{\mathsf{Sender}}(\alpha_j)$ equals $\Delta_{i\to j}(c)$; and (ii) $g_{\mathsf{Sender}}(\alpha_o) = \Delta_{i\to o}(c)$ for at least $2t+1$ indices $o \in [n]$, where each $\Delta_{i\to o}(c)$ is provided by the other parties P_o. Party P_j accepts $\Delta_{i\to j}(r)$ as the commit polynomial if both checks pass.

According to the Schwartz–Zippel lemma, if the coefficient set of $\Delta_{i\to j}(r)$ differs from the committed values $s_{\ell,j}$, then the difference polynomial is nonzero and has degree at most m. Hence, it will be detected at the random challenge c with probability at least $1 - m/|\mathbb{F}|$.

This approach yields a communication complexity of $\mathcal{O}(mn\kappa + n^4\kappa + n\kappa^2)$ bits. Since the protocol must be executed by all n parties, the total communication cost in the final protocol becomes $\mathcal{O}(mn^2\kappa + n^2\kappa^2)$ bits, where $m = \mathcal{O}(n^3)$. Consequently, the batching requirement drops from $N = \Omega(n^{11}\kappa)$ to a polynomial scale; in our concrete instantiation, $N = \Omega(n^4 + n\kappa)$ suffices to meet the $\mathcal{O}(n\kappa)$ amortized target.

At this point, the row polynomial reconstruction is complete: every party can eventually obtain its row polynomials, with at least $2t + 1$ of them held by honest parties. Additionally, up to $2t + 1$ parties have received their column polynomials, among which up to t may be corrupted. It remains necessary to ensure that the remaining honest parties (at most t of them), denoted P_j's, correctly reconstruct their column polynomials $g_{\ell,j}(y) = F_\ell(\alpha_j, y)$. The question now is: *how can this be achieved?*

Symmetric Bivariate Polynomial. We address this by leveraging the fact that at least $t + 1$ honest parties already possess both their row and column polynomials, along with a key property of the symmetric bivariate polynomial $V_\ell(x, y) = F_\ell(x, y) + F_\ell(y, x)$, which has degree $(2t, 2t)$ and satisfies the symmetry $V_\ell(\alpha_i, \alpha_j) = V_\ell(\alpha_j, \alpha_i)$. This symmetry allows $V_\ell(x, y)$ to be uniquely reconstructed from any $t+1$ honest evaluations. We aim to utilize the identity[5]:

$$V_\ell(\alpha_i, y) = F_\ell(\alpha_i, y) + F_\ell(y, \alpha_i) = g_{\ell,i}(y) + f_{\ell,i}(y).$$

The key observation is that the $t+1$ honest parties, each holding both their row and column polynomials, can collectively reconstruct $V_\ell(x, y)$. They can then assist the remaining honest parties—those who possess only their row polynomials $f_{\ell,j}(x)$—in computing their missing column polynomials $g_{\ell,j}(y)$ using the relation $g_{\ell,j}(y) = V_\ell(\alpha_j, y) - f_{\ell,j}(y)$.

This approach offers the benefit of guaranteed termination and completeness. Let O denote the set of parties that possess both row and column polynomials, i.e., the same as $\mathcal{M}$. We designate $P_o \in O$, $P_{o'}$ (with index o') as a party holding row polynomials, and P_w as an arbitrary party. A formal description follows, organized into several phases:

1. **Broadcast Phase:** Each party P_o that holds both its row and column polynomials locally computes, for each $\ell \in [m]$, $V_\ell(\alpha_o, y) = g_{\ell,o}(y) + f_{\ell,o}(y)$, where $f_{\ell,o}(y)$ denotes the univariate polynomial obtained by substituting $x = \alpha_o$ in the bivariate slice $f_{\ell,o}(x)$. Then P_o broadcasts the set $\{V_\ell(\alpha_o, y)\}_{\ell\in[m]}$ to all parties and also sends $\{f_{\ell,o}(\alpha_w)\}_{\ell\in[m]}$ directly to each P_w.
2. **Receiving Phase:** Every party $P_{o'}$ that holds its row polynomial similarly sends $\{f_{\ell,o'}(\alpha_w)\}_{\ell\in[m]}$ to P_w. Upon collecting these values from all such P_o and $P_{o'}$, party P_w stores them in the set $\mathcal{W}_w$.
3. **Verification Phase:** For each $\ell \in [m]$ and for each broadcast from P_o, party P_w checks $V_\ell(\alpha_o, \alpha_w) = g_{\ell,o}(\alpha_w) + f_{\ell,o}(\alpha_w)$, where $g_{\ell,o}(\alpha_w)$ is computed from the row polynomial $f_{\ell,w}(x)$ at α_o held by P_w. If all ℓ-checks succeed, P_w

[5] We remark that the second term is $f_{\ell,i}(y)$, which shares the same coefficients as $f_{\ell,i}(x)$.

broadcasts an "OK_{V_o}" message[6]. Once P_w has received OK_{V_o} from at least $2t+1$ distinct parties, it accepts the set of polynomials $\{V_\ell(\alpha_o, y)\}_{\ell\in[m]}$ broadcast by P_o. Once P_w has accepted polynomials from at least $t+1$ distinct parties P_o, it interpolates these to reconstruct the m symmetric bivariate polynomials $\{V_\ell(x, y)\}_{\ell\in[m]}$ each of degree-$(2t, 2t)$.

4. **Column Polynomial Computation:** Using the symmetric bivariate polynomials, P_w computes its column slices by $g_{\ell,w}(y) = V_\ell(\alpha_w, y) - f_{\ell,w}(y), \ell \in [m]$. Then P_w substitutes the values in $\mathcal{W}_w$ into these column polynomials; if at least $2t+1$ of the stored shares agree with $\{g_{\ell,w}(y)\}_{\ell\in[m]}$, P_w broadcasts "Done_j" message[7].
5. **Termination Condition:** Upon receiving "Done_i" from at least $2t+1$ distinct parties P_i, P_w outputs its reconstructed shares $\{g_{\ell,w}(\alpha_k)\}_{\ell\in[m],\, k\in[-t,0]}$ and terminates the protocol.

The above steps ensure termination and completeness of the ACSS protocol. **However**, since each P_o broadcasts $\{V_\ell(\alpha_o, y)\}_{\ell\in[m]}$ to all parties, it incurs $\mathcal{O}(mn^3\kappa)$ bits communication, which far exceeds the desired cost of $\mathcal{O}(mn^2\kappa)$ bits (i.e., amortized cost of $\mathcal{O}(n\kappa)$ bits). *How can we overcome this?*

Aggregation via Algebraic Fingerprinting. We address this challenge by introducing an alternative approach, *Algebraic Fingerprinting*, which requires broadcasting only two aggregated polynomials instead of all m polynomials. Specifically, instead of broadcasting $V_\ell(\alpha_i, y) = g_{\ell,i}(y) + f_{\ell,i}(y)$ for each $\ell \in [m]$, we only need to broadcast the aggregated polynomial(s):

$$f_o(y) = \sum_{\ell=1}^{m} d^\ell f_{\ell,o}(y), \text{ of degree } t,$$
$$g_o(y) = \sum_{\ell=1}^{m} d^\ell g_{\ell,o}(y), \text{ of degree } 2t,$$

where d is a publicly agreed random challenge (a public coin). Once $t+1$ aggregated accepting pairs $(g_o(y), f_o(y))$ are identified, party P_w performs the following steps:

- It first computes and reconstructs the degree-$(2t, 2t)$ bivariate polynomial $V(x, y) = \sum_{\ell=1}^{m} d^\ell V_\ell(x, y)$, by interpolating from the $t+1$ evaluations $g_o(y) + f_o(y)$ (i.e., $V(\alpha_o, y)$), leveraging the symmetric structure of the underlying polynomials.
- With the reconstructed $V(x, y)$ in hand, party P_w finally derives its (aggregated) column polynomial according to $g_w(y) = V(\alpha_w, y) - f_w(y)$.

We still face two problems: 1) When should the public coin challenge d be chosen, and at what point should the aggregated polynomials be broadcast? 2) How can the column polynomials $\{g_{\ell,w}(y)\}_{\ell\in[m]}$ be reconstructed from the aggregated column polynomial $g_w(y)$?

Regarding the first problem, the challenge d should be chosen after P_w has sent $\{f_{\ell,w}(\alpha_j)\}_{\ell\in[m]}$ to P_j and after the set $\mathcal{W}_w$, contains $\{f_{\ell,i}(\alpha_w)\}_{\ell\in[m]}$'s from

[6] OK_{V_o} indicating that, for each $\ell \in [m]$, $V_\ell(\alpha_o, \alpha_w) = f_{\ell,w}(\alpha_o) + f_{\ell,o}(\alpha_w)$ has been successfully verified by P_w.

[7] Done_j indicating that P_w has accepted the column polynomials $\{g_{\ell,w}(y)\}_{\ell\in[m]}$.

at least $2t+1$ other parties (like P_i), has been selected for the subsequent check. This corresponds to the Pointwise Exchange step described below.

For the second problem, the aggregated polynomial $g_w(y)$ can be used to determine the correct evaluations of $g_{\ell,w}(y)$ at $2t+1$ points for each $\ell \in [m]$, and subsequently reconstruct the column polynomials $g_{\ell,w}(y)$. First, one can select multiple pairs $(\alpha_i, g_{\ell,w}(\alpha_i))$ as candidate evaluations from $\mathcal{W}_w$, since $g_{\ell,w}(\alpha_i) = f_{\ell,i}(\alpha_w)$. The aggregated polynomial $g_w(y)$ can then be used to identify the correct evaluations by verifying whether $g_w(\alpha_i) = \sum_{\ell=1}^{m} d^\ell g_{\ell,w}(\alpha_i)$. This soundness of the checking follows from the Schwartz–Zippel lemma again.

1. **Pointwise Exchange.** Each P_w unicasts to P_j the row evaluations $\{f_{\ell,w}(\alpha_j)\}$ for $\ell \in [m]$, and in return collects from row-only providers $P_{o'}$ and row+column providers P_o the sets $\{f_{\ell,o'}(\alpha_w)\}_{\ell\in[m]}$ and $\{f_{\ell,o}(\alpha_w)\}_{\ell\in[m]}$, respectively; once for every ℓ at least $2t+1$ such points are gathered, P_w announces Yes'_w.
2. **Public-Coin Batching.** Upon receiving a public coin $d \xleftarrow{\$} \mathbb{F}$ from $\mathcal{F}_{\mathsf{coin}}$, each party P_w forms the batched rows $f_w(x) = \sum_{\ell=1}^{m} d^\ell f_{\ell,w}(x)$ (degree t) and, if available (when $P_w \in O$), the batched columns $g_w(y) = \sum_{\ell=1}^{m} d^\ell g_{\ell,w}(y)$ (degree $2t$), and broadcasts the resulting polynomials;
   P_w then performs the lightweight consistency tests $f_o(\alpha_w) \stackrel{?}{=} \sum_\ell d^\ell f_{\ell,o}(\alpha_w)$,[8] $g_o(\alpha_w) \stackrel{?}{=} \sum_\ell d^\ell f_{\ell,w}(\alpha_o)$,[9] $f_{o'}(\alpha_o) \stackrel{?}{=} g_o(\alpha_{o'})$,[10] and degree checks on the polynomials it receives. If all pass, it defines $V(\alpha_o, y) = g_o(y) + f_o(y)$ and broadcasts OK_{V_o}, to indicate $g_o(y)$ and $f_o(x)$ from P_o are verified.
3. **Symmetric Reconstruction.** Once OK_{V_o} has been seen from at least $2t+1$ distinct senders, P_w accepts $\{V(\alpha_o, y)\}$;
   after accepting $t+1$ different such polynomials $V(\alpha_o, y)$'s, it interpolates a symmetric degree-$(2t, 2t)$ bivariate $V(x, y)$ and finally recovers its column via $g_w(y) = V(\alpha_w, y) - f_w(y)$.
   Independently, it also reconstructs $g_{\ell,w}(y)$ from the $2t+1$ points $(\alpha_i,\ f_{\ell,i}(\alpha_w))$, which satisfy that $g_w(\alpha_i)$ (obtained by evaluating $g_w(y)$ at α_i) equals to $\sum_{\ell=1}^{m} d^\ell f_{\ell,i}(\alpha_w)$, before broadcasting Done_w.
4. **Termination Condition:** Upon receiving "Done_i" from at least $2t+1$ distinct parties P_i, P_w outputs its reconstructed shares $\{\, g_{\ell,w}(\alpha_k)\}_{\ell\in[m],\, k\in[-t,0]}$ and terminates the protocol.

In this way, the broadcast of polynomials contributes only $\mathcal{O}(n^3\kappa + n^3 \log n)$ bits, while the exchange of $\{f_{\ell,w}(\alpha_j)\}_{\ell\in[m]}$ dominates the communication, amounting to $\mathcal{O}(mn^2\kappa)$ bits. Consequently, the overall communication complexity is $\mathcal{O}(mn^2\kappa)$ bits, which meets the amortized target of $\mathcal{O}(n\kappa)$ bits.

[8] where the right-hand side is assembled from the shares in $\mathcal{W}_w$.
[9] where the right-hand side is computed locally from P_w's row polynomials
[10] where $f_{o'}(\alpha_o)$ is obtained by evaluating the broadcast $f_{o'}(x)$ at $x = \alpha_o$ and $g_o(\alpha_{o'})$ is obtained by evaluating the broadcast $g_o(y)$ at $y = \alpha_{o'}$.

2.3 High-Level Overview of Our ACSS Protocol

Our ACSS protocol contains four phases: **Sharing Phase**, **Verification Phase**, **AcceptPoly Phase**, and **Completion Phase**. Refer to Appendix E.3 for a detailed analysis of communication.

1. **Sharing Phase.** In the Sharing Phase, the dealer distributes the secret shares to all parties. Specifically, the dealer distributes the degree-$2t$ column polynomials and the degree-t row polynomials to all parties, and collects replies of AMC signatures. Once valid signatures from at least $2t+1$ parties have been received, the dealer forms the set $\mathcal{M}$ of signers and broadcasts it. Ignoring the communication cost of $\mathcal{F}_{\mathsf{AMC}}$, the communication in this phase is $\mathcal{O}(mn^2\kappa + n^3 \log n)$ bits.
2. **Verification Phase.** In the Verification Phase, each party checks that its column/row polynomials are consistent with the bivariate polynomials reconstructed from any $t+1$ honest parties in $\mathcal{M}$. Ignoring the communication cost of $\mathcal{F}_{\mathsf{AMC}}$, the communication in this phase is $\mathcal{O}(n^3\kappa)$ bits.
3. **AcceptPoly Phase.** In the AcceptPoly Phase, each party verifies the correctness of received aggregate polynomial. A detailed description appears in Section 4.2. Ignoring the communication cost of $\mathcal{F}_{\mathsf{AcceptPoly}}$, we need $\mathcal{O}(n^4\kappa)$ bits communication in this phase.
4. **Completion Phase.** In the Completion Phase, leveraging the AcceptPoly protocol and the symmetric polynomials, each party first reconstructs its row polynomial and then computes its column polynomial to recover the final shares. Further details are given in Section 5.1. The total communication in this phase is $\mathcal{O}(mn^2\kappa + n^4\kappa + n^4 \log n)$ bits.

3 Preliminary

3.1 Notations and Schwartz–Zippel Lemma

- For any positive integer $N \in \mathbb{Z}^+$, let $[N]$ represent the set $\{1, 2, \ldots, N\}$. For integers $a, b \in \mathbb{Z}$ where $a < b$, we define $[a, b]$ as the set $\{a, a+1, \ldots, b\}$. The security parameter is denoted by κ, and $\mathbb{F}$ represents a finite field where $|\mathbb{F}| = 2^{\Theta(\kappa)}$. We assume that $\alpha_1, \ldots, \alpha_n$ are distinct and non-zero elements of $\mathbb{F}$. Let $\mathcal{P} = \{P_1, ..., P_n\}$ denote the set of n parties participating in the protocol. Let $[s]_t$ denote a t-sharing of the secret s, and let $[s]_{t-i}$ denotes the share of the secret s assigned to P_i in a t-sharing.
- We assume that n is a polynomial function of κ, expressed as $n = \mathsf{poly}(\kappa)$. Additionally, $\mathsf{negl}(\kappa)$ denotes a negligible function in κ, meaning that for sufficiently large κ, $\mathsf{negl}(\kappa)$ is smaller than any fraction $\mathsf{poly}(\kappa)/2^{\kappa}$. All polynomials discussed are defined over field $\mathbb{F}$. A polynomial of degree-t is expressed as $f(x) = a_0 + a_1 x + \cdots + a_t x^t$, where each coefficient $a_i \in \mathbb{F}$.
- A *bivariate polynomial* of degree-$(t, 2t)$ is written in the form $F(x, y) = \sum_{i=0}^{t} \sum_{j=0}^{2t} r_{ij} x^i y^j$, where the coefficients $r_{ij} \in \mathbb{F}$. For each party index i, we then define the *ith row polynomial* and *ith column polynomial* by $f_i(x) = F(x, \alpha_i)$, $g_i(y) = F(\alpha_i, y)$, respectively.

- Given $F(x, y)$ as defined above, we define a degree-$(2t, 2t)$ *symmetric bivariate polynomial* of form $V(x, y) = F(x, y) + F(y, x)$. Obviously, we have $V(x, y) = V(y, x)$ for any $x, y \in \mathbb{F}$. For $V(\alpha_i, y) = F(\alpha_i, y) + F(y, \alpha_i)$, we have any $t + 1$ distinct column polynomials $V(\alpha_i, y)$ suffice to uniquely reconstruct $V(x, y)$.

Lemma 2 (Schwartz–Zippel Lemma [44,48]). *Let $f \in \mathbb{F}[x_1, \ldots, x_m]$ be a nonzero polynomial of total degree $d \geq 0$, and let $S \subseteq \mathbb{F}$ be a finite set. If $x_1, \ldots, x_m$ are sampled independently and uniformly from S, then*

$$\Pr\big[f(x_1, \ldots, x_m) = 0\big] \;\leq\; \frac{d}{|S|}.$$

Equivalently, the number of zeros of f in S^m is at most $d\,|S|^{m-1}$. In particular, taking $S = \mathbb{F}$ yields $\Pr[f(x) = 0] \leq d/|\mathbb{F}|$. The bound is tight (e.g., when f depends on a single variable and has d distinct roots).

3.2 Definitions

Asynchronous secret sharing (ASS) is a fundamental tool in secure distributed computing. This section defines key schemes such as *Asynchronous Weak Secret Sharing* (AWSS), *Asynchronous Verifiable Secret Sharing* (AVSS), and *Asynchronous Complete Secret Sharing* (ACSS). Intuitively, AWSS guarantees termination, secrecy, and a *weak commitment*; AVSS strengthens this to a *strong commitment* (all honest parties reconstruct the same value); ACSS further guarantees *completeness*, i.e., that once sharing succeeds for any honest party, consistent shares of a unique (possibly dealer-dependent) value eventually exist for all honest parties. Since our protocols rely precisely on this completeness guarantee, we state and use only the ACSS definition below. For formal definitions of AWSS and AVSS and their relationship to ACSS, see Appendix A.2.

Definition 1 (Asynchronous Complete Secret Sharing (ACSS)[13]). *Let $(\mathsf{Sh}, \mathsf{Rec})$ denote a pair of protocols where a dealer $D \in \mathcal{P}$ shares a secret s using protocol Sh. The pair (Sh[11], Rec[12]) is defined as a t-resilient statistical ACSS scheme if the following conditions are satisfied:*

- ***Termination:*** *With probability at least $1 - \epsilon$, the following conditions hold:*
  1) *If D is honest, every honest party will eventually terminate the execution of protocol Sh.*
  2) *If any honest party successfully terminates protocol Sh, then regardless of D's behavior, all other honest parties will eventually terminate Sh.*
  3) *Once all honest parties have completed protocol Sh and initiated protocol Rec, every honest party will eventually complete Rec.*
- ***Correctness:*** *If D is corrupted and at least one honest party has successfully completed the protocol Sh, then there exists a unique value $s' \in \mathbb{F}$, such that every honest party, upon completing Rec, outputs exactly s'.*

[11] Sh refers to the sharing phase protocol of the ACSS scheme.
[12] Rec refers to the reconstruction phase protocol of the ACSS scheme.

- ***Secrecy:*** *If D is honest and no honest party has initiated protocol* Rec*, then the adversary $\mathcal{A}_t$ has no knowledge of the secret s.*
- ***Completeness:*** *If at least one honest party successfully completes* Sh*, then there exists a random degree-t polynomial $f(x)$ over $\mathbb{F}$, such that $f(0) = s'$ and each honest party $P_i \in \mathcal{P}$ eventually holds their share $s_i = f(i)$ of the secret s'. Furthermore, if the dealer D is honest, then $s' = s$.*

Definition 2 (t-sharing [9,11]). *A value $s \in \mathbb{F}$ is said to be a t-sharing among the parties in $\mathcal{P}$ if there exists a random polynomial $f(x)$ of degree t over $\mathbb{F}$, where $f(0) = s$, such that each (honest) party $P_i \in \mathcal{P}$ holds their share $s_i = f(i)$ of the secret s.*

The aforementioned definitions of AWSS, AVSS, and ACSS can be extended to handle a secret s comprising multiple elements (denoted by N, where $N > 1$) from $\mathbb{F}$.

3.3 The Security Model

In this section, our study follows the security framework established in [27,28].

Security Model. UC framework introduced by Canetti [21] serves as the foundation for defining the security of our protocols, employing the well-established *real and ideal world paradigm* [20]. In essence, a protocol Π is deemed secure if its execution in the real world can be replicated in an idealized environment with equivalent outcomes. The complete formal details will be provided in Appendix A.1.

3.4 Sub-protocols

In this section, we describe the sub-protocols required for the construction of our ACSS protocol. An ACSS protocol must meet the following criteria: if the dealer is honest, the protocol guarantees that all honest parties receive a complete t-sharing. If the dealer is corrupted, the protocol ensures that once an honest party accepts its share as output, every other honest party will eventually obtain its corresponding share of a t-sharing. The ideal functionality of ACSS is shown in Fig. 1.

The current state-of-the-art protocol that securely implements $\mathcal{F}_{\mathsf{ACSS}}$ is presented in [36], achieving an amortized communication complexity of $\mathcal{O}(n\kappa)$-bit per sharing. As the construction of our sub-protocols involves invoking $\mathcal{F}_{\mathsf{ACSS}}$, we restate their result in the following lemma:

Functionality $\mathcal{F}_{\mathsf{ACSS}}$

Public Input: $(\alpha_0, \ldots, \alpha_n), N$.
Upon receiving $(\mathsf{dealer}, \mathsf{ACSS}, \{q_1(\cdot), \ldots, q_N(\cdot)\})$ from $D \in \mathcal{P}$, the trusted party does the following:

1. If all the polynomials $q_1(\cdot), \ldots, q_N(\cdot)$ are degree-t polynomials, the trusted party sends a request-based delayed output $\{q_1(\alpha_i), \ldots, q_N(\alpha_i)\}$ to each party $P_i \in \mathcal{P}$.
2. If any of the $\{q_1(\cdot), \ldots, q_N(\cdot)\}$ is not a degree-t polynomial, the trusted party does nothing.

Fig. 1: Ideal functionality for asynchronous complete secret sharing.

Lemma 3. *([36]) For any $N \in \mathbb{Z}^+$, there exists a protocol that t-securely implements $\mathcal{F}_{\mathsf{ACSS}}$ with statistical security. The communication complexity of this protocol is $\mathcal{O}(N \cdot n\kappa + n^{12}\kappa^2)$ bits.*

A-Cast. Bracha introduced an efficient asynchronous byzantine reliable broadcast (ABRB) protocol, known as A-Cast [16]. This protocol allows a designated sender to reliably broadcast a message to all parties in an asynchronous network. The protocol ensures eventual message delivery to all parties, with the message delivery schedule being controlled by the adversary. The formal definition of the A-Cast functionality, as presented in [16], is provided in Fig. 2. The complete formal definition will be provided in Appendix A.2. More recently, Alhaddad et al. [6] constructed an *error-free* ABRB protocol, which has near-optimal communication complexity $\mathcal{O}(n\ell + n^2 \log n)$ for an ℓ-bit message.

Functionality $\mathcal{F}_{\mathsf{ACast}}$

Upon receiving $(\mathsf{sender}, \mathsf{ACast}, m)$ from $P_S \in \mathcal{P}$, the trusted party sends a request-based delayed output (P_S, ACast, m) to each $P_i \in \mathcal{P}$.

Fig. 2: Ideal functionality for broadcasting a message.

Private Reconstruction. Given a set of complete t-sharings, all parties can facilitate the private reconstruction of secrets to a designated party using the Online Error Correction (OEC) process, as introduced in [19]. Additionally, the OEC process enables a single party to reconstruct all shared values, a feature that proves instrumental in constructing subsequent protocols. The formal functionality of private reconstruction is presented in Fig. 3. For completeness, the construction and security proof of Π_{PrivRec}, which implements $\mathcal{F}_{\mathsf{PrivRec}}$, are detailed in Appendix D.1. The communication complexity of this protocol is $\mathcal{O}(Nn\kappa)$-bit.

Definition 3 (Online Error Correction (OEC)). *Let s be a secret that is t-sharing among a set of parties $\mathcal{P}$ through a degree-t polynomial $f(x)$, such that $f(0) = s$. Consider a specific party $P_\alpha \in \mathcal{P}$ who aims to reconstruct s. To achieve this, each party P_i sends its share s_i to P_α. The shares may arrive at P_α in arbitrary order, and up to t of them may be incorrect or missing. Despite these challenges, P_α can reconstruct the secret $s = f(0)$ in an online manner by applying the OEC process to the received s_i.*

The OEC method leverages the error correction properties of Reed-Solomon codes, allowing P_α to identify the received shares to define a unique degree-t

interpolation polynomial $f(x)$. This ensures accurate and reliable reconstruction of the secret s, even in the presence of erroneous or incomplete shares.

Functionality $\mathcal{F}_{\mathsf{PrivRec}}$

Public Input: The receiver R, number N.
For complete t-sharing $[s_1]_t, \ldots, [s_N]_t$:

1. The trusted party receives the set of corrupted parties $\mathcal{C} \subset \mathcal{P}$.
2. Upon receiving a request $(\mathsf{Request}, \mathsf{PrivRec}, R, N)$ from an honest party, for all $i \in [N]$, the trusted party receives the shares of $[s_i]_t$ from all honest parties and sends the whole sharing $[s_i]_t$ as a request-based delayed output to R.
3. The trusted party sends the corrupted parties' shares to the ideal adversary.

Fig. 3: Ideal functionality for private reconstruction.

Generating Random Coins. The description of $\mathcal{F}_{\mathsf{coin}}$ appears below. Such a functionality can be realized by first preparing random degree-t Shamir sharings and then reconstruct the secrets to all parties when needed. Following [27], $\mathcal{F}_{\mathsf{coin}}$ can be realized with communication complexity $\mathcal{O}(n^3\kappa)$-bit per random value.

Functionality $\mathcal{F}_{\mathsf{coin}}$

Public Input: The parties $P_1, ..., P_n \in \mathcal{P}$.

1. Upon receiving $2t+1$ parties' requests $(\mathsf{Request}, \mathsf{RandCoin}, \mathcal{P})$, the trusted party samples a random value r.
2. The trusted party sends r to all parties as request-based delayed outputs.

Fig. 4: Ideal functionality for random coins.

4 The Asynchronous Message Checking (AMC)

In this section, we present the construction of AMC, which is inspired by the previous works on AICP [38] and APICP [36]. Unlike AICP, AMC represents a novel signature scheme tailored for asynchronous environments. It exhibits two key properties: linear homomorphism and multi-receiver support. Moreover, AMC achieves an amortized communication complexity of $\mathcal{O}(1)$-bit per bit of message, significantly reducing the overall communication cost compared to [36].

Overview of AICP and APICP. AICP is a signature scheme involving a signer (Sig), an intermediary (I), and a receiver (R). The signer generates a signature on a message and transmits it to the receiver via the intermediary. The receiver then verifies the signature to ensure that the information originates from Sig. The message is encoded as a vector $\boldsymbol{s} \in \mathbb{F}^L$. APICP extends AICP to support multiple secret vectors and multiple rounds of revelation. Both schemes follow three main phases:

1. **Generating a Signature on the Vector.** In AICP, the signer Sig randomly samples κ points $\alpha_1^{(i)}, \ldots, \alpha_\kappa^{(i)}$ from the field $\mathbb{F}$ as base points and com-

putes the corresponding verification points on a polynomial $f(x)$. Sig sends $f(x)$ to I and distributes the verification points to the parties. In APICP, this process is extended to m secret vectors, with the signer sampling $n(T+1)^2\kappa$ points for each vector and performing the analogous operations.

2. **Verifying the Validity of the Signature.** Upon receiving $f(x)$, the intermediary I verifies whether the provided verification points are consistent with $f(x)$. Specifically, if at least $2t+1$ parties' verification points lie on $f(x)$, I accepts the polynomial. In APICP, each $f(x)$ is verified in a similar manner; however, the verification points are partitioned into $T+1$ disjoint sets, each of size $(T+1)\kappa$.
3. **Revelation and Signature Checking.** Once I accepts $f(x)$, it forwards the polynomial to R, and each party transmits its remaining verification points to R. The receiver verifies whether at least one verification point from each party's set lies on $f(x)$. If this condition is satisfied for at least $t+1$ parties, R reconstructs the secret vector $\boldsymbol{s}$ from $f(x)$ and verifies its authenticity. In APICP, this revelation process can be repeated up to T times, enabling multiple secret disclosures.

It is important to note that in AICP, each party's verification points are divided into two subsets: one allocated for verification by I and the other for verification by R, making the scheme suitable for a single receiver. In contrast, APICP partitions the verification points into $T+1$ disjoint subsets, where one subset is used by I and the remaining T subsets are reserved for different receivers, thus supporting multiple revelations to multiple receivers.

The Functionality of AMC. The ideal functionality of AMC is defined in Fig. 5. The protocol proceeds in two phases: an initialization phase, followed by a revelation phase. Once the initialization phase is invoked by the signer Sig, the revelation phase may be invoked up to T times. Prior to each invocation of the revelation phase, it is assumed that all parties have agreed on a request of the form $(\mathsf{Request}, \mathsf{AMC}, R, \boldsymbol{c})$. Our ACSS protocol is designed to ensure that this agreement is achieved before any revelation occurs.

Functionality $\mathcal{F}_{\mathsf{AMC}}$

For fixed signer Sig and intermediary I:

Initialization Phase: Init$(T, (\boldsymbol{s_1}, \ldots, \boldsymbol{s_m}))$

1. The trusted party receives the identities of corrupted parties $\mathcal{C} \subset \mathcal{P}$.
2. Upon receiving[a] $(\mathsf{Init}, \mathsf{AMC}, T, (\boldsymbol{s_1}, \ldots, \boldsymbol{s_m}))$ from Sig, the trusted party sends a request-based delayed output $(Sig, \mathsf{AMC}, (\boldsymbol{s_1}, \ldots, \boldsymbol{s_m}))$ to I and sets $\mathsf{count} = T$.

Revelation Phase: Rev$(R, \boldsymbol{c}, (\boldsymbol{s_1}, \ldots, \boldsymbol{s_m}))$

1. Each time the trusted party receives a request $(\mathsf{Request}, \mathsf{AMC}, R, \boldsymbol{c})$ from an honest party, where $\boldsymbol{c} = (c_1, \ldots, c_m)$. If $\mathsf{count} > 0$, the trusted party does the following things and replaces count by $\mathsf{count} - 1$:
   1) If $I \in \mathcal{C}$, the trusted party waits to receive an instruction from $\mathcal{S}$:
      - If $\mathcal{S}$ sends Ignore, the trusted party does nothing.

- If $\mathcal{S}$ sends Proceed, if $Sig \in \mathcal{C}$, the trusted party waits to receive $\boldsymbol{s}'$ from $\mathcal{S}$ and sends a request-based delayed output $\boldsymbol{s}'$ to the receiver R. Otherwise, the trusted party sends a request-based delayed output $\boldsymbol{s} = \sum_{k=1}^{m} c_k \boldsymbol{s_k}$ to the receiver R.

2) If $I \notin \mathcal{C}$, the trusted party sends a request-based delayed output $\boldsymbol{s} = \sum_{k=1}^{m} c_k \boldsymbol{s_k}$ to the receiver R.

2. If R is honest, R outputs the results received from the trusted party. Corrupted parties may output anything they want.

[a] *Note*: Here, for each $k \in [m]$ we write $\boldsymbol{s}_k = (s_{k,1}, s_{k,2}, \ldots, s_{k,n})$.

Fig. 5: Ideal functionality for AMC.

Overview of AMC Construction. We outline the high-level design of our AMC construction that realizes the ideal functionality described in Fig. 5. The protocol proceeds in two phases: initialization and revelation.

1. **Init (high-level).** The signer Sig receives $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$, samples a hidden vector $\boldsymbol{\alpha} \in \mathbb{F}^n$, and publishes Shamir (t, n)-sharings of $\boldsymbol{\alpha}$ and of the tags $\tau_\ell = \langle \boldsymbol{\alpha}, \boldsymbol{s}_\ell \rangle = \sum_{i=1}^{n} \alpha_i s_{\ell,i}$. Each $\boldsymbol{s}_\ell$ is embedded as a low-degree polynomial $L_\ell(x)$ and masked to $H_\ell(x) = L_\ell(x) + Q(x) r_\ell(x)$ with $Q(\alpha_u^i) = 0$, so evaluations at per–party points $\{\alpha_u^i\}_{u \in [\kappa]}$ reveal $L_\ell(\alpha_u^i)$. Using public coins ρ_i, Sig gives each P_i validation values $y_u^i = \sum_{\ell=1}^{m} H_\ell(\alpha_u^i) \rho_i^{\ell-1}$ together with shares $[\boldsymbol{\alpha}]_{t \to i}$, $\{[\tau_\ell]_{t \to i}\}$, and auxiliary triples, while $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$ go to I. The intermediary broadcasts weights $(a^{(i)}, b^{(i)})$; each P_i returns $\Sigma_i = \sum_{u=1}^{\kappa} (a^{(i)} u + b^{(i)})\, y_u^i$ and its points. Using public data, I checks $\widehat{\Sigma}_i = \Sigma_i$ for at least $2t{+}1$ distinct i; if so, it accepts the initialization from Sig.
2. **Rev (high-level).** Given coefficients $\boldsymbol{c} = (c_1, \ldots, c_m)$ and while the counter allows, I forms $\boldsymbol{s} = \sum_{\ell=1}^{m} c_\ell \boldsymbol{s}_\ell$, sends $\boldsymbol{s}$ to R, and Shamir–shares $\boldsymbol{s}$ to all parties. Each P_i pairs its fresh share with a pre-sampled triple $(\boldsymbol{a}_r, \boldsymbol{b}_r, c_r)$ by broadcasting masked differences $\delta\boldsymbol{\alpha} = [\boldsymbol{\alpha}]_{t \to i} - [\boldsymbol{a}_r]_{t \to i}$ and $\delta\boldsymbol{s} = [\boldsymbol{s}]_{t \to i} - [\boldsymbol{b}_r]_{t \to i}$; from any $2t{+}1$ broadcasts, $\mathcal{F}_{\mathsf{PrivRec}}$ reconstructs $\Delta\boldsymbol{\alpha} = \boldsymbol{\alpha} - \boldsymbol{a}_r$ and $\Delta\boldsymbol{s} = \boldsymbol{s} - \boldsymbol{b}_r$. Using these deltas and local shares, each P_i sends tokens $v_i' = [c_r]_{t \to i} + \langle \Delta\boldsymbol{\alpha}, [\boldsymbol{b}_r]_{t \to i} \rangle + \langle \Delta\boldsymbol{s}, [\boldsymbol{a}_r]_{t \to i} \rangle$ and $\tau_i' = \sum_{\ell=1}^{m} c_\ell [\tau_\ell]_{t \to i}$ to R. The receiver reconstructs $v' = c_r + \langle \Delta\boldsymbol{\alpha}, \boldsymbol{b}_r \rangle + \langle \Delta\boldsymbol{s}, \boldsymbol{a}_r \rangle$ and $\tau = \sum_{\ell=1}^{m} c_\ell \tau_\ell$, then verifies the linking check $v' + \langle \Delta\boldsymbol{\alpha}, \Delta\boldsymbol{s} \rangle = \tau$; if it holds (and the deltas agree from at least $2t{+}1$ parties), R accepts and outputs $\boldsymbol{s}$. This phase can reveal at most T times in total.

4.1 Instantiation of AMC

Our protocol Π_{AMC} consists of two sub-protocols: Π_{Init} and Π_{Rev}, corresponding to the initialization phase and the revelation phase, respectively.

The procedure for Π_{Init} is illustrated in Fig. 6. In this phase, the signer (Sig) receives the secret vectors from the environment, samples a hidden secret seed, and creates threshold shares and authentication tags for these vectors. Each vector is encoded into a masked low-degree polynomial and evaluated at per–party

validation points derived from public coins. Sig sends to every party its validation bundle (validation points, evaluations, and the relevant shares) and sends the plaintext secret vectors to the intermediary I.
Upon receipt, I broadcasts random weights to all parties. Each party forms a weighted aggregate of its validation evaluations and returns this aggregate together with the corresponding points to I. Using only public data and the reported points, I recomputes the expected aggregates and checks consistency. If at least $2t+1$ parties pass the check, I accepts the initialization instance as a valid one from Sig; otherwise, it rejects. The communication complexity of Π_{Init} is $\mathcal{O}(mn\kappa + n^3\kappa + n\kappa^2)$ bits.

Protocol Π_{Init}

Protocol Init$(Sig, I, T, m, L, \kappa)$

Parameters. All parties agree on distinct public field elements $\alpha_1, \ldots, \alpha_n \in \mathbb{F}$ and on public random coins $\rho_1, \ldots, \rho_n \in \mathbb{F}$, with ρ_i corresponding to party P_i. The identities of Sig and I, the revelation time $T = 2n$, the secret-vector length $L = n$, and the security parameter κ.

1. Sig receives its input $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$ from the environment.
2. Sig samples $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_n) \xleftarrow{\$} \mathbb{F}^n$. For each $i \in [n]$, Sig creates a Shamir (t, n)-sharing $[\alpha_i]_t$.
3. For each $\ell \in [m]$, let $\boldsymbol{s}_\ell = (s_{\ell,1}, \ldots, s_{\ell,n}) \in \mathbb{F}^n$. Sig computes $\tau_\ell = \langle \boldsymbol{\alpha}, \boldsymbol{s}_\ell \rangle = \sum_{i=1}^{n} \alpha_i \, s_{\ell,i}$, and produces a Shamir (t, n)-sharing $[\tau_\ell]_t$.
4. Sig locally samples T independent triples $\{(\boldsymbol{a}_r, \boldsymbol{b}_r, c_r)\}_{r \in [T]}$, to be used in subsequent checks (e.g., as randomness/beaver-like material as required by the protocol, $c_r = \langle \boldsymbol{a}_r, \boldsymbol{b}_r \rangle$).
5. For each $i \in [n], u \in [\kappa]$, Sig samples κ points $\alpha_1^i, \ldots, \alpha_\kappa^i \xleftarrow{\$} \mathbb{F}$, independently across i and u.
6. For every $\ell \in [m]$ with vector $\boldsymbol{s}_\ell = (s_{\ell,1}, \ldots, s_{\ell,n})$, let $L_\ell(x) := \sum_{j=0}^{n-1} s_{\ell,j+1} \, x^j$, so that the first n coefficients of $L_\ell(x)$ equal $\boldsymbol{s}_\ell$. Define $Q(x) = \prod_{i=1}^{n} \prod_{u=1}^{\kappa} \big(x - \alpha_u^i\big)$, $r_\ell(x) \xleftarrow{\$} \{\text{polynomials over } \mathbb{F} \text{ of degree } \leq n\}$, and set $H_\ell(x) = L_\ell(x) + Q(x) \cdot r_\ell(x)$. Then $\deg H_\ell \leq n + n\kappa$. Moreover, since $Q(\alpha_u^i) = 0$ for all $i \in [n], u \in [\kappa]$, we have $H_\ell(\alpha_u^i) = L_\ell(\alpha_u^i)$ for all $i \in [n], u \in [\kappa]$[a].
7. For each $i \in [n]$ and $u \in [\kappa]$, Sig computes $y_u^i = \sum_{\ell=1}^{m} H_\ell(\alpha_u^i) \, \rho_i^{\ell-1} = \sum_{\ell=1}^{m} L_\ell(\alpha_u^i) \, \rho_i^{\ell-1}$, where $\rho_i \in \mathbb{F}$ is the public coin associated with P_i.
8. To each party P_i, Sig sends $\big\{(\alpha_u^i, y_u^i)\big\}_{u \in [\kappa]}$, $[\boldsymbol{\alpha}]_{t \to i}$, $\{[\tau_\ell]_{t \to i}\}_{\ell \in [m]}$, $\big\{([\boldsymbol{a}_r]_{t \to i}, [\boldsymbol{b}_r]_{t \to i}, [c_r]_{t \to i})\big\}_{r \in [T]}$. In parallel, Sig sends $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$ to I.
9. Upon receiving Sig's message, for each $i \in [n]$, I samples two independent random weights $a^{(i)}, b^{(i)} \xleftarrow{\$} \mathbb{F}$ and broadcasts the pair $(a^{(i)}, b^{(i)})$.
10. Each P_i, after receiving $(a^{(i)}, b^{(i)})$ and Sig's data, defines for every $u \in [\kappa]$, $w_u^{(i)} = a^{(i)} \cdot u + b^{(i)}$, and computes the linear aggregate $\Sigma_i = \sum_{u=1}^{\kappa} w_u^{(i)} \, y_u^i$. Party P_i sends $\big(\Sigma_i, \{\alpha_u^i\}_{u \in [\kappa]}\big)$ to I.

11. For each received pair from P_i, I recomputes $\widehat{\Sigma}_i = \sum_{u=1}^{\kappa} w_u^{(i)} \left(\sum_{\ell=1}^{m} L_\ell(\alpha_u^i) \rho_i^{\ell-1} \right)$, and checks whether $\widehat{\Sigma}_i = \Sigma_i$. If this equality holds for at least $2t+1$ distinct indices $i \in [n]$, then I accepts $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$; otherwise, it rejects.

[a] Because $Q(\alpha_u^i) = 0$, the mask vanishes at these points.

Fig. 6: The protocol of the Π_{Init}.

As illustrated in Fig. 7, the Π_{Rev} protocol assumes that all parties agree on the coefficients used for the linear combination prior to protocol execution; this agreement is ensured by our ACSS protocol. Given coefficients $\boldsymbol{c}$ (while the counter allows), I derives the target linear combination of the accepted secrets, sends it to R, and Shamir–shares it to all parties. Each party masks its fresh share with a pre-sampled triple, broadcasts masked differences for $2t$+1-reconstruction, and—via the private-reconstruction functionality—obtains public deltas. Using these deltas, each party sends R two short consistency tokens (from the triples and from the authenticated tags). R checks threshold consistency and a single linking equality; if it passes, R accepts and outputs the vector. Notably, the Π_{Rev} protocol may be executed up to T times following each initialization of AMC, with each execution incurring a communication complexity of $\mathcal{O}(n^3 T \kappa)$ bits.

Protocol Π_{Rev}

Protocol $(I, R, \boldsymbol{c} = (c_1, \ldots, c_m), \mathsf{count}, T, L, \kappa)$

Parameter: The identity of I, R, vector $\boldsymbol{c}$, counter count, revelation times $T = 2n$, packed secrets number m, each secret vector length $L = n$, and security parameter κ.

1. If I accepts $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$, it computes $\boldsymbol{s} = \sum_{\ell=1}^{m} c_\ell \boldsymbol{s}_\ell$ and sends $\boldsymbol{s}$ to R.
2. I secret-shares $\boldsymbol{s}$ component-wise using Shamir (t, n), yielding $[\boldsymbol{s}]_t$, and sends the corresponding share $[\boldsymbol{s}]_{t \to i}$ to party P_i for all $i \in [n]$.
3. Upon receiving I's message, each party P_i (for a fixed index $r \in [T]$) proceeds as follows:
   1) Compute the share differences $\delta\boldsymbol{\alpha} = [\boldsymbol{\alpha}]_{t \to i} - [\boldsymbol{a}_r]_{t \to i}$ and $\delta\boldsymbol{s} = [\boldsymbol{s}]_{t \to i} - [\boldsymbol{b}_r]_{t \to i}$, and send $\delta\boldsymbol{\alpha}, \delta\boldsymbol{s}$ to all parties P_j with $j \in [n]$.
   2) After receiving at least $2t$+1 pairs $\delta\boldsymbol{\alpha}, \delta\boldsymbol{s}$ from distinct parties, invoke $\mathcal{F}_{\mathsf{PrivRec}}$ to reconstruct $\Delta\boldsymbol{\alpha} = \boldsymbol{\alpha} - \boldsymbol{a}_r$ and $\Delta\boldsymbol{s} = \boldsymbol{s} - \boldsymbol{b}_r$, and forward $\Delta\boldsymbol{\alpha}, \Delta\boldsymbol{s}$ to R.
   3) Compute $v_i' = [c_r]_{t \to i} + \langle \Delta\boldsymbol{\alpha}, [\boldsymbol{b}_r]_{t \to i} \rangle + \langle \Delta\boldsymbol{s}, [\boldsymbol{a}_r]_{t \to i} \rangle$ (here the inner products are taken component-wise via linearity over Shamir shares), and send v_i' to R.
   4) Compute $\tau_i' = \sum_{\ell=1}^{m} c_\ell [\tau_\ell]_{t \to i}$ and send τ_i' to R.
4. After receiving messages from I and the parties, R performs:
   1) Check that at least $2t$+1 parties report consistent $(\Delta\boldsymbol{\alpha}, \Delta\boldsymbol{s})$; if so, proceed.

2) Using $\mathcal{F}_{\mathsf{PrivRec}}$ on $\{v'_i\}_{i\in[n]}$ and $\{\tau'_i\}_{i\in[n]}$, reconstruct $v' = c_r + \langle \Delta\boldsymbol{\alpha}, \boldsymbol{b}_r\rangle + \langle \Delta\boldsymbol{s}, \boldsymbol{a}_r\rangle$ and $\tau = \sum_{\ell=1}^{m} c_\ell \tau_\ell$.
3) Compute $v = v' + \langle \Delta\boldsymbol{\alpha}, \Delta\boldsymbol{s}\rangle$. If $v = \tau$, then R accepts $\boldsymbol{s}$ and outputs $\boldsymbol{s}$.

Fig. 7: The protocol of the Π_{Rev}.

The protocol Π_{AMC} is presented in Fig. 8. Its communication complexity is $\mathcal{O}(mn\kappa + n^3T\kappa + n\kappa^2)$ bits, and a detailed complexity analysis is provided in Appendix B.2.

Protocol Π_{AMC}

Parameter: The identity of dealer Sig and intermediary I, packed secrets number m, each secret vector length L and security parameter κ.

Initialization Phase: Init(T)

1. All parties participate in Π_{Init}.
2. All parties initialize a counter $\mathsf{count} = T$.

Revelation Phase: Rev$(R, \boldsymbol{c})$

1. All parties participate in Π_{Rev} with inputs R, $\boldsymbol{c}$ and replace count by $\mathsf{count} - 1$.

Fig. 8: The protocol of the Π_{AMC}.

Theorem 2. *The protocol Π_{AMC} realizes $\mathcal{F}_{\mathsf{AMC}}$ in the $\mathcal{F}_{\mathsf{PrivRec}}$-hybrid model with statistically secure and $\mathcal{O}(mn\kappa + n^3T\kappa + n\kappa^2)$ bits communication.*

We prove Theorem 2 in Appendix B.1.

4.2 AMC-based protocol - AcceptPoly Protocol

We present the functionality of our AcceptPoly protocol in Fig. 9. The AcceptPoly protocol is divided into two phases: the initialization phase and the acceptance phase.

Once the initialization phase is invoked by the sender *Sender*, the acceptance phase may be invoked up to T' times. Prior to each invocation of the acceptance phase, it is assumed that all parties have agreed on a request ($\mathsf{Request}$, $\mathsf{AcceptPoly}$, *Acceptor*). This agreement is guaranteed by our ACSS protocol.

Functionality $\mathcal{F}_{\mathsf{AcceptPoly}}$

For fixed sender *Sender* and acceptor *Acceptor*:

Initialization Phase: Init$(T', (\boldsymbol{s_1}, \ldots, \boldsymbol{s_m}))$

1. The trusted party receives the identities of corrupted parties $\mathcal{C} \subset \mathcal{P}$.
2. Upon receiving $(\mathsf{Init}, \mathsf{AcceptPoly}, T', (\boldsymbol{s_1}, \ldots, \boldsymbol{s_m}))$ from *Sender*, the trusted party sets $\mathsf{count} = T'$.

Acceptance Phase: Acc$(Acceptor, (\boldsymbol{s_1}, \ldots, \boldsymbol{s_m}))$

1. Each time the trusted party receives a request (Request, AcceptPoly, *Acceptor*) from an honest party. If count > 0, the trusted party does the following things and replaces count by count $- 1$:
   1) If $Sender \in \mathcal{C}$, the trusted party waits to receive an instruction from $\mathcal{S}$:
      i. If $\mathcal{S}$ sends Ignore, the trusted party does nothing.
      ii. If $\mathcal{S}$ sends Proceed, the trusted party waits for $s'_{\mathsf{Acceptor}}(r) = \sum_{\ell=1}^{m} r^{\ell}\, s'_{\ell,\mathsf{Acceptor}}$, a degree-$m$ univariate polynomial in the indeterminate r; and sends a request-based delayed output $s'_{\mathsf{Acceptor}}(r) = \sum_{\ell=1}^{m} r^{\ell} s'_{\ell,\mathsf{Acceptor}}$ to the *Acceptor*.
   2) If $Sender \notin \mathcal{C}$, the trusted party sends a request-based delayed output $s_{\mathsf{Acceptor}}(r) = \sum_{\ell=1}^{m} r^{\ell} s_{\ell,\mathsf{Acceptor}}$ to the *Acceptor*.
2. If *Acceptor* is honest, *Acceptor* outputs the results received from the trusted party. Corrupted parties may output anything they want.

Fig. 9: Ideal functionality for AcceptPoly.

Instantiation of AcceptPoly Protocol. The $\Pi_{\mathsf{AcceptPoly}}$ protocol consists of two sub-protocols: Π_{Init} and Π_{Acc}, corresponding to the initialization phase and the acceptance phase, respectively.

The procedure for Π_{Init} is illustrated in Fig. 10. In this sub-protocol, the *sender* receives m secret vectors, for each party P_i, collects the i-th coordinates across all vectors into a length-m column. It encodes this column as a single univariate polynomial in r of degree at most m (the coefficients are the collected coordinates) and delivers that polynomial to P_i. This sets up per-party packed inputs for the subsequent phases. The communication complexity of Π_{Init} is $\mathcal{O}(mn\kappa)$ bits.

Protocol Π_{Init}

Protocol Init$(Sender, T', m, L, \kappa)$

Parameter: All parties agree on distinct public field elements $\alpha_1, \ldots, \alpha_n$ in $\mathbb{F}$. The identity of *sender*, revelation times $T' = n$, length of each secret vector $L = n$, and security parameter κ.

1. *Sender* receives his input $(\boldsymbol{s_1}, \ldots, \boldsymbol{s_m})$ from the environment.
2. For each $i \in [n]$, let $\boldsymbol{s}_{\mathsf{Sender}\to i} := (s_{1,i}, \ldots, s_{m,i})$, where $s_{\ell,i}$ is the i-th component of $\boldsymbol{s}_{\ell}$. *Sender* forms the univariate polynomial in the indeterminate r, $\Delta_{\mathsf{Sender}\to i}(r) := \sum_{\ell=1}^{m} r^{\ell} s_{\ell,i}$, which has degree at most m over $\mathbb{F}$.
3. For each $i \in [n]$, *Sender* sends $\Delta_{\mathsf{Sender}\to i}(r)$ to P_i.

Fig. 10: The protocol of the Π_{Init}.

The procedure for Π_{Acc} is illustrated in Fig. 11. Prior to the execution of the protocol, it is assumed that all parties have agreed on the coefficients for the linear combination, which is guaranteed by our ACSS protocol. The acceptor P_j receives its packed polynomial from the *sender*, performs a basic sanity check (e.g., the constant term is zero), and evaluates it at the challenge point c derived from $\boldsymbol{c}$. All other parties locally evaluate their own packed polynomials at the

same c and send these evaluations to P_j; in parallel, the authenticated evaluation handle for the sender is obtained from $\mathcal{F}_{\mathsf{AMC}}(P_{\mathsf{Sender}}, I)$.
Using the authenticated handle, P_j cross-checks its own evaluation and those received from the others. If a threshold of at least $2t+1$ evaluations are consistent with the authenticated values, P_j accepts and outputs its received polynomial; otherwise, it rejects. The Π_{Acc} protocol can be executed up to T' times after each AcceptPoly initialization, with a communication complexity of $\mathcal{O}(n\kappa)$ bits per execution.

Protocol Π_{Acc}

Protocol Acceptance$(Sender, Acceptor, \boldsymbol{c}, \mathsf{count}, T', L, \kappa)$

Parameter: The identity of I, $\mathcal{R}$, vectors $\boldsymbol{c} = (c^1, ..., c^m)$, counter count, revelation times $T' = n$, packed secrets number m, each secret vector length $L = n$, and security parameter κ. Let $\mathcal{F}_{\mathsf{AMC}}(P_{\mathsf{Sender}}, I)$ denote $\mathcal{F}_{\mathsf{AMC}}$ with dealer P_{Sender} and intermediary I.

1. The *Acceptor* P_j receives $\Delta_{\mathsf{Sender}\to j}(r)$ from *Sender*, and this polynomial is degree m, P_j verifies whether $\Delta_{\mathsf{Sender}\to j}(0) = 0$; if so, it computes $\Delta_{\mathsf{Sender}\to j}(c)$.
2. Each other party P_i (for $i \in [n]$, $i \neq j$) proceeds as follows:
   1) Verify whether $\Delta_{\mathsf{Sender}\to i}(0) = 0$; if so, proceed.
   2) Compute $\Delta_{\mathsf{Sender}\to i}(c)$ and send it to the *Acceptor* P_j.
   3) Send $(\mathsf{Request}, \mathsf{AMC}, P_j, (c^1, \ldots, c^m))$ to $\mathcal{F}_{\mathsf{AMC}}(P_{\mathsf{Sender}}, I)$.
3. Upon receiving the message from Sender, the values from all P_i, and the authenticated evaluation handle $\boldsymbol{g}_{\mathsf{Sender}} = (g_{\mathsf{Sender}}(\alpha_1), ..., g_{\mathsf{Sender}}(\alpha_n))$ from $\mathcal{F}_{\mathsf{AMC}}(P_{\mathsf{Sender}})$, the *Acceptor* P_j performs:
   1) Check whether $\Delta_{\mathsf{Sender}\to j}(c) = g_{\mathsf{Sender}}(\alpha_j)$; if so, continue.
   2) Check whether $\Delta_{\mathsf{Sender}\to i}(c) = g_{\mathsf{Sender}}(\alpha_i)$ for each $i \in [n]$.
   3) If at least $2t+1$ distinct indices i satisfy the previous condition, accept $\Delta_{\mathsf{Sender}\to j}(r)$ and output $\Delta_{\mathsf{Sender}\to j}(r)$.

Fig. 11: The protocol of the Π_{Acc}.

$\Pi_{\mathsf{AcceptPoly}}$ is presented in Fig. 12. Its communication complexity is $\mathcal{O}(mn\kappa + nT'\kappa)$ bits, we will give a detailed complexity analysis in Appendix C.2.

Protocol $\Pi_{\mathsf{AcceptPoly}}$

Parameter: The identity of sender *Sender* and acceptor *Acceptor*, packed secrets number m, each secret vector length L and security parameter κ.

Initialization Phase: Init(T')

1. All parties participate in Π_{Init}.
2. All parties initialize a counter $\mathsf{count} = T'$.

Acceptance Phase: Acc$(Acceptor)$

1. All parties participate in Π_{Acc} with inputs *Acceptor* and replace count by $\mathsf{count} - 1$.

Fig. 12: The protocol of the $\Pi_{\mathsf{AcceptPoly}}$.

Theorem 3. *The protocol $\Pi_{\mathsf{AcceptPoly}}$ realizes $\mathcal{F}_{\mathsf{AcceptPoly}}$ in the $\mathcal{F}_{\mathsf{AMC}}$-hybrid model with statistically secure and $\mathcal{O}(mn\kappa + nT'\kappa)$-bit communication.*

We prove Theorem 3 in Appendix C.1.

5 The Asynchronous Secret Sharing Protocol (ACSS)

In this section, we present the ACSS protocol Π_{ACSS}, which achieves both linear communication complexity and guaranteed termination. In this protocol, the dealer sends N degree-t sharing polynomials, denoted $q_1(x), \ldots, q_N(x)$, to $\mathcal{F}_{\mathsf{ACSS}}$. If all submitted polynomials are indeed of degree t, then $\mathcal{F}_{\mathsf{ACSS}}$ distributes the corresponding shares to all honest parties.

5.1 Instantiation of ACSS

To achieve the proposed Π_{ACSS} protocol, all parties execute the protocols Π_{Sh}, Π_{Ver}, $\Pi_{\mathsf{Accept}_{\mathsf{Share}}}$, and Π_{Comp} in sequence. As described in Section 2.3, Π_{ACSS} consists of four distinct phases (see Fig. 26), which we detail below. The parameters used in these protocols are defined at the beginning of the sharing phase.

Sharing Phase Π_{Sh}. As shown in Fig. 13, this phase involves the dealer distributing secret shares to the parties. Specifically, the dealer D encodes each batch of $t+1$ degree-t polynomials into a degree-$(t, 2t)$ bivariate polynomial and distributes all degree-$2t$ column polynomials to all parties. For each column polynomial $g_\ell(y)$, the secret shares for each party are given by $g_\ell(\alpha_{-t}), \ldots, g_\ell(\alpha_0)$. Subsequently, each party invokes $\mathcal{F}_{\mathsf{AMC}}$ to generate signatures for each received column polynomial and sends the signatures along with the polynomials back to D. At the same time, each party also invokes $\mathcal{F}_{\mathsf{AcceptPoly}}$. When the set $\mathcal{M}$ reaches a size of $2t+1$, the dealer broadcasts $\mathcal{M}$ for verification by all parties. Disregarding the communication cost of $\mathcal{F}_{\mathsf{AMC}}$ and $\mathcal{F}_{\mathsf{AcceptPoly}}$, the communication complexity of Π_{Sh} is $\mathcal{O}(mn^2\kappa + n^3 \log n)$ bits.

Protocol Π_{Sh}

Parameter: All parties agree on distinct public field elements $\alpha_{-t}, \ldots, \alpha_{-1}, \alpha_0, \alpha_1, \ldots, \alpha_n$ in $\mathbb{F}$, number of polynomials N. Let $\mathcal{F}_{\mathsf{AMC}}(Sig, I)$ denote $\mathcal{F}_{\mathsf{AMC}}$ with dealer Sig and intermediary I. Let $\mathcal{F}_{\mathsf{AcceptPoly}}(Sender)$ denote $\mathcal{F}_{\mathsf{AcceptPoly}}$ with sender $Sender$. Let $L = n$ denote the vector length in $\mathcal{F}_{\mathsf{AMC}}$ and $\mathcal{F}_{\mathsf{AcceptPoly}}$, $m' = \lceil \frac{N}{t+1} \rceil$.

Initialization: All polynomials are divided into m groups of size $t+1$. Let $T = 2n, T' = n$, $m = m' + T \cdot T'$. Later on, T will be the number of revelations in $\mathcal{F}_{\mathsf{AMC}}$, T' will be the number of revelations in $\mathcal{F}_{\mathsf{AcceptPoly}}$.

Sharing Phase

Distributing Column Polynomials. Upon receiving the input polynomials $q_1(x), \ldots, q_N(x)$, the dealer D performs the following steps for each $\ell \in [m]$:

1. Set $\mathsf{idx} \leftarrow (\ell - 1)(t+1) + 1$.
2. **Case A:** If $(t+1) \mid N$ then

1) for each $\ell \in [m']$ choose a *uniformly random* bivariate polynomial $F_\ell(x, y)$ of degree $(t, 2t)$ s.t. $F_\ell(x, \alpha_{-i}) = q_{\text{idx}+i}(x)$ for all $i \in [0, t]$;
2) for each $\ell \in [m'+1, m]$ randomly sample $(m-m')(t+1)$ degree-t polynomials $q_{m'(t+1)+1}(x), \ldots, q_{m(t+1)}(x)$ and construct $F_\ell(x, y)$ as in 1).

3. **Case B:** Otherwise (i.e. $(t+1) \nmid N$),
   1) let $m'(t+1) \coloneqq \lceil N/(t+1) \rceil (t+1)$ and pad $\{q_1(x), \ldots, q_N(x)\}$ with $N' - N$ independent degree-t polynomials $q_{N+1}(x), \ldots, q_{m'(t+1)}(x)$. For each $\ell \in [m']$, construct each $F_\ell(x, y)$ exactly as in Case A 1).
   2) for each $\ell \in [m'+1, m]$, construct each $F_\ell(x, y)$ exactly as in Case A 2).
4. Send the column polynomials $\{g_{\ell,i}(y) = F_\ell(\alpha_i, y)\}_{\ell \in [m]}$ and the row polynomials $\{f_{\ell,i}(x) = F_\ell(x, \alpha_i)\}_{\ell \in [m]}$ to each $P_i \in \mathcal{P}$.

Committing to the column polynomials: Each $P_i \in \mathcal{P}$ executes the following code:

1. Wait to receive $\{g_{\ell,i}(y)\}_{\ell \in [m]}$ from D.
2. If $\{g_{\ell,i}(y)\}_{\ell \in [m]}$ is degree-$2t$ and $\{f_{\ell,i}(x)\}_{\ell \in [m]}$ is degree-t, broadcast YES_i, and set $\boldsymbol{g_{\ell,i}} = (g_{\ell,i}(\alpha_1), \ldots, g_{\ell,i}(\alpha_n))$ for each $\ell \in [m]$. Then $\boldsymbol{g_{\ell,i}}$ is a vector of size L.
3. Send $(\mathsf{Init}, \mathsf{AMC}, T, (\boldsymbol{g_{1,i}}, \ldots, \boldsymbol{g_{m,i}}))$ to $\mathcal{F}_{\mathsf{AMC}}(P_i, D)$.
4. Send $(\mathsf{Init}, \mathsf{AcceptPoly}, T', (\boldsymbol{g_{1,i}}, \ldots, \boldsymbol{g_{m,i}}))$ to $\mathcal{F}_{\mathsf{AcceptPoly}}(P_i)$.

Confirming column polynomials: D executes the following code:

1. Initialize a set $\mathcal{M}$ to $\emptyset$.
2. If $|\mathcal{M}| < 2t+1$, include P_i into $\mathcal{M}$ when:
   1) YES_i is received from P_i.
   2) $(P_i, \mathsf{AMC}, (\boldsymbol{g_{1,i}}, \ldots, \boldsymbol{g_{m,i}}))$ is received from $\mathcal{F}_{\mathsf{AMC}}(P_i, D)$ and $g_{\ell,i}(y) = F_\ell(\alpha_i, y)$ for each $\ell \in [m]$.
3. Broadcast $\mathcal{M}$ when $|\mathcal{M}| = 2t+1$.

Verifying the set $\mathcal{M}$: Each party moves to the next phase if the following conditions are met:

1. $\mathcal{M}$ is received from D and $|\mathcal{M}| = 2t+1$.
2. YES_h is received from all $P_h \in \mathcal{M}$.

Fig. 13: The protocol of the Π_{Sh}.

Verification Phase Π_{Ver}. As depicted in Fig. 14, this phase requires each party P_i to verify the validity of its received column polynomials. Upon receiving its column polynomials from D, P_i validates their consistency with randomly chosen linear combinations of bivariate polynomials selected by D. Ignoring the communication cost of $\mathcal{F}_{\mathsf{AMC}}$, the communication complexity of Π_{Ver} is $\mathcal{O}(n^2\kappa + n^3 \log n)$ bits.

Protocol Π_{Ver}

Verification Phase

For each $P_i \in \mathcal{P}$:

1. Upon receiving $\{g_{\ell,i}(y)\}_{\ell \in [m]}$ from D and these polynomials are all degree-$2t$, P_i broadcasts a random $r_i \in \mathbb{F}$, and computes $g_i(y) = \sum_{\ell=1}^{m} r_i^\ell g_{\ell,i}(y)$ and $f_i(x) = \sum_{\ell=1}^{m} r_i^\ell f_{\ell,i}(x)$.

2. Upon receiving r_i from P_i, each party sends $(\mathsf{Request}, \mathsf{AMC}, P_i, (r_i^1, \ldots, r_i^m))$ to $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$ for all $P_h \in \mathcal{M}$.
3. Upon receiving $\boldsymbol{g}_h$ from $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$ for all $P_h \in \mathcal{M}$, P_i proceeds as follows:
   1) For each $P_h \in \mathcal{M}$, parse the vector $\boldsymbol{g}_h = (g_h(\alpha_1), \ldots, g_h(\alpha_n))$ into a degree-$2t$ univariate polynomial $g_h(y)$.
   2) There exists a bivariate polynomial $F(x, y)$ with $\deg_x F \leq t$ and $\deg_y F \leq 2t$ such that $F(\alpha_h, y) = g_h(y)$ and $F(x, \alpha_h) = f_h(x)$ for every $P_h \in \mathcal{M}$. Moreover, $F(\alpha_i, y) = g_i(y)$ and $F(x, \alpha_i) = f_i(x)$.

Fig. 14: The protocol of the Π_{Ver}.

AcceptPoly Phase $\Pi_{\mathsf{AcceptPoly}}$. As shown in Fig. 15, this phase involves each party P_i verifying the validity of the polynomials it receives. When P_i receives its polynomials from the sender *Sender*, it validates their consistency with randomly chosen linear combinations of polynomials. Other parties assist P_i in verifying the validity of the polynomials. Disregarding the communication cost of $\mathcal{F}_{\mathsf{AMC}}$, the communication complexity of $\Pi_{\mathsf{AcceptPoly}}$ is $\mathcal{O}(n^4\kappa)$ bits.

Protocol $\Pi_{\mathsf{AcceptPoly}}$

AcceptPoly Phase

For each $P_i \in \mathcal{P}/\mathcal{M}$:
1. Each online P_j sends $(\mathsf{Request}, \mathsf{RandCoin}, \mathcal{P})$ to $\mathcal{F}_{\mathsf{coin}}$.
2. Upon receiving c_i from $\mathcal{F}_{\mathsf{coin}}$, each online party sends $(\mathsf{Request}, \mathsf{AMC}, P_i, (c_i^1, \ldots, c_i^m))$ to $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$, and sends $(\mathsf{Request}, \mathsf{AcceptPoly}, P_i)$ to $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$ for all $P_h \in \mathcal{M}$.
3. Receive $\Delta_{\mathsf{h}\to i}(r) = \sum_{\ell=1}^{m} r^\ell g_{\ell,h}(\alpha_i)$ from $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$ for all $P_h \in \mathcal{M}$.

Fig. 15: The protocol of the $\Pi_{\mathsf{AcceptPoly}}$.

Completion Phase Π_{Comp}. As depicted in Fig. 16, this phase describes how each honest party ultimately reconstructs its output shares. Each party first reconstructs its degree-t row polynomials, and then reconstructs its degree-$2t$ column polynomials to derive its final shares. Including the communication cost of implementing $\mathcal{F}_{\mathsf{coin}}$, the communication complexity of Π_{Comp} is $\mathcal{O}(mn^2\kappa + n^4\kappa + n^4 \log n)$ bits.

Protocol Π_{Comp}

Completion Phase

Preparing:
1. For each $P_i \in \mathcal{P}/\mathcal{M}$, do the following:
   1) Upon receiving $\{\Delta_{h\to i}(r)\}_{P_h \in \mathcal{M}}$ from $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$, party P_i gets the coefficient vector $\boldsymbol{g}_{h\to i} = \big(g_{1,h}(\alpha_i), \ldots, g_{m,h}(\alpha_i)\big)$, i.e., the entire set $\{g_{\ell,h}(\alpha_i)\}_{\ell\in[m],\, P_h\in\mathcal{M}}$.

Reconstructing Row Polynomials:
2. For each $P_v \in \mathcal{P}/\mathcal{M}$, perform the following steps:
   1) Each $P_h \in \mathcal{M}$ sends $\{g_{\ell,h}(\alpha_v)\}_{\ell\in[m]}$ to P_v.

2) P_v accepts $\{g_{\ell,h}(\alpha_v)\}_{\ell\in[m]}$ if the following condition holds: the values $\{g_{\ell,h}(\alpha_v)\}_{\ell\in[m]}$ match the secret obtained during the preparing phase.
3) Upon accepting $t+1$ different sets $\{g_{\ell,h}(\alpha_v)\}_{\ell\in[m]}$ from distinct parties P_h, P_v reconstructs m degree-t polynomials $\{f_{\ell,v}(x)\}_{\ell\in[m]}$ such that $f_{\ell,v}(\alpha_h) = g_{\ell,h}(\alpha_v)$ for all $\ell \in [m]$, $P_h \in \mathcal{M}$.

Reconstructing Column Polynomials:

3. For each $P_w \in \mathcal{P}$, perform the following steps:
   1) For each $P_j \in \mathcal{P}$, P_w computes $\{f_{\ell,w}(\alpha_j)\}_{\ell\in[m]}$ and sends them to P_j.
   2) For each party $P_{o'}$ that has accepted row polynomials, and for each party P_o that has accepted both column and row polynomials, they send $\{f_{\ell,o'}(\alpha_w)\}_{\ell\in[m]}$ and $\{f_{\ell,o}(\alpha_w)\}_{\ell\in[m]}$ respectively to P_w. P_w collects these into the set $\mathcal{W}_w = \{\mathcal{W}_{1,w}, \ldots, \mathcal{W}_{m,w}\}$, once $|\mathcal{W}_{\ell,w}| \geq 2t+1$ for each $\ell \in [m]$, party P_w broadcasts Yes'_w.
   3) Upon receiving Yes'_j from at least $2t+1$ distinct parties P_j, P_w sends $(\mathsf{Request}, \mathsf{RandCoin}, \mathcal{P})$ to $\mathcal{F}_{\mathsf{coin}}$.
   4) Upon receiving $d \in \mathbb{F}$ from $\mathcal{F}_{\mathsf{coin}}$, P_w computes the polynomial $f_w(x) := \sum_{\ell=1}^{m} d^\ell f_{\ell,w}(x)$ with $\deg f_w = t$, and broadcasts $f_w(x)$. If P_w has also accepted the column polynomials, it computes $g_w(y) := \sum_{\ell=1}^{m} d^\ell g_{\ell,w}(y)$ with $\deg g_w = 2t$, and broadcasts $g_w(y)$ as well.
   5) Upon receiving, from each party P_o that has accepted both the row *and* column polynomials, the pair $(g_o(y), f_o(x))$ and the evaluations $\{f_{\ell,o}(\alpha_w)\}_{\ell\in[m]}$, and from each party $P_{o'}$ that has accepted only the row polynomials the polynomial $f_{o'}(x)$ together with $\{f_{\ell,o'}(\alpha_w)\}_{\ell\in[m]}$, P_w performs the following checks:
      i. Verify that $\deg g_o = 2t$, $\deg f_o = t$, and $\deg f_{o'} = t$. If so, continue.
      ii. Compute $\Delta'_{o\to w} := \sum_{\ell=1}^{m} d^\ell g_{\ell,o}(\alpha_w)$ and check whether $\Delta'_{o\to w} = f_o(\alpha_w)$. If so, continue.
      iii. Compute $\Delta'_{w\to o} := \sum_{\ell=1}^{m} d^\ell g_{\ell,w}(\alpha_o)$ and check whether $\Delta'_{w\to o} = f_w(\alpha_o)$. If so, continue.
      iv. For each pair $(P_o, P_{o'})$, check whether $f_{o'}(\alpha_o) = f_o(\alpha_{o'})$. If so, continue.
      v. If all of (i)–(iv) hold, set $V(\alpha_o, y) = g_o(y) + f_o(y)$ and broadcast OK_{V_o}.
   6) Upon receiving OK_{V_o} from at least $2t+1$ distinct parties, P_w accepts the polynomial $V(\alpha_o, y)$.
   7) Upon accepting $t+1$ different such polynomials $V(\alpha_o, y)$, P_w reconstructs a degree-$(2t, 2t)$ symmetric bivariate polynomial $V(x, y)$. P_w can then compute its column polynomial as $g_w(y) = V(\alpha_w, y) - f_w(y)$.
   8) P_w accepts the column polynomial $g_w(y)$ if the shares $\{f_{\ell,i}(\alpha_w)\}_{\ell\in[m]}$ received from at least $2t+1$ parties $P_i \in \mathcal{W}_w$ are consistent, i.e., $g_w(\alpha_i) = f_i(\alpha_w)$ for those i. Using any such set of at least $2t+1$ consistent parties, P_w interpolates each $g_{\ell,w}(y)$ from the points $\{(\alpha_i, f_{\ell,i}(\alpha_w))\}$ and thereby reconstructs the family $\{g_{\ell,w}(y)\}_{\ell\in[m]}$.
   9) If all of the above conditions are satisfied, P_w broadcasts Done_w.

Terminating:

4. For each party P_k that has accepted the column polynomials $\{g_{\ell,k}(y)\}_{\ell\in[m]}$ and the row polynomials $\{f_{\ell,k}(x)\}_{\ell\in[m]}$, perform the following:
   1) Upon receiving Done_w messages from at least $2t+1$ distinct parties P_w, P_k outputs all its reconstructed shares $\{g_{\ell,k}(\alpha_i)\}_{\ell\in[m],\, i\in[-t,0]}$ and terminates.

Fig. 16: The protocol of the Π_{Comp}.

Lemma 4. *The protocol Π_{ACSS} t-securely realizes $\mathcal{F}_{\mathsf{ACSS}}$ in the $(\mathcal{F}_{\mathsf{AMC}}, \mathcal{F}_{\mathsf{AcceptPoly}}, \mathcal{F}_{\mathsf{coin}})$-hybrid model with statistical security.*

We now use the instances of $\mathcal{F}_{\mathsf{AMC}}$ (Π_{AMC} in Section 4), $\mathcal{F}_{\mathsf{AcceptPoly}}$ ($\Pi_{\mathsf{AcceptPoly}}$ in Section 4.2) and $\mathcal{F}_{\mathsf{coin}}$ instead of the functionalities to realize $\mathcal{F}_{\mathsf{ACSS}}$. Taking $m = n^3$, we obtain our main Theorem 1.
The proof of Lemma 4 is given in Appendix E.2.

Communication at a glance. We batch N input polynomials into $m' = N/(t+1)$ groups and add $TT' = 2n^2$ random groups with $T = 2n$, $T' = n$, so $m = m' + TT'$ and each AMC call handles m length-n vectors. Aggregating the dominant costs across phases, the overall communication of Π_{ACSS} is $\mathcal{O}(mn^2\kappa + n^5\kappa + n^2\kappa^2)$ bits. Since $mn = (m' + TT')n = \Theta(N)$, setting $m = \Theta(n^3)$ gives $\mathcal{O}(Nn\kappa + n^5\kappa + n^2\kappa^2)$ bits. For large N (e.g., $N = \Omega(n^4 + n\kappa)$), the amortized per-secret cost approaches $\mathcal{O}(n\kappa)$. Detailed analysis of the complexity of our Π_{ACSS} is given in Appendix E.3.

Acknowledgmnts

This work was supported by the National Key Research and Development Program of China (No. 2021YFA1000600), the National Natural Science Foundation of China (No. 12441104), the Singapore Ministry of Education (MOE) Academic Research Fund (AcRF) Tier 1 grants (Proposal ID: 23-SIS-SMU-035, 24-SIS-SMU-052).

References

1. Abraham, I., Malkhi, D., Spiegelman, A.: Asymptotically optimal validated asynchronous byzantine agreement. In: PODC 2019. pp. 337–346 (2019)
2. Abraham, I., Asharov, G., Patra, A., Stern, G.: Asynchronous agreement on a core set in constant expected time and more efficient asynchronous vss and mpc. In: TCC 2025. pp. 451–482. Springer (2025)
3. Abraham, I., Asharov, G., Yanai, A.: Efficient perfectly secure computation with optimal resilience. J. Cryptol. **35**(4), 1–43 (2022)
4. Abraham, I., Dolev, D., Halpern, J.Y.: An almost-surely terminating polynomial protocol for asynchronous byzantine agreement with optimal resilience. In: PODC 2008. pp. 405–414 (2008)
5. Abraham, I., Jovanovic, P., Maller, M., Meiklejohn, S., Stern, G.: Bingo: Adaptivity and asynchrony in verifiable secret sharing and distributed key generation. In: CRYPTO 2023. pp. 39–70. Springer (2023)

6. Alhaddad, N., Das, S., Duan, S., Ren, L., Varia, M., Xiang, Z., Zhang, H.: Balanced byzantine reliable broadcast with near-optimal communication and improved computation. In: PODC 2022. p. 399–417 (2022)
7. Alhaddad, N., Varia, M., Yang, Z.: Haven++: Batched and packed dual-threshold asynchronous complete secret sharing with applications. Cryptology ePrint Archive (2024)
8. Bacho, R., Loss, J., Stern, G., Wagner, B.: Harts: High-threshold, adaptively secure, and robust threshold schnorr signatures. In: ASIACRYPT 2024. pp. 104–140. Springer (2025)
9. Beerliová-Trubíniová, Z., Hirt, M.: Efficient multi-party computation with dispute control. In: TCC 2006. pp. 305–328. Springer (2006)
10. Beerliová-Trubíniová, Z., Hirt, M.: Simple and efficient perfectly-secure asynchronous mpc. In: ASIACRYPT 2007. pp. 376–392. Springer (2007)
11. Beerliová-Trubíniová, Z., Hirt, M.: Perfectly-secure mpc with linear communication complexity. In: TCC 2008. pp. 213–230 (2008)
12. Ben-Or, M., Canetti, R., Goldreich, O.: Asynchronous secure computation. In: STOC 1993. pp. 52–61 (1993)
13. Ben-Or, M., Kelmer, B., Rabin, T.: Asynchronous secure computations with optimal resilience (extended abstract). In: PODC 1994. pp. 183–192 (1994)
14. Ben-Sasson, E., Fehr, S., Ostrovsky, R.: Near-linear unconditionally-secure multiparty computation with a dishonest minority. In: CRYPTO 2012. pp. 663–680. Springer (2012)
15. Ben-Or, M., Goldwasser, S., Wigderson, A.: Completeness theorems for non-cryptographic fault-tolerant distributed computation. In: Providing Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali, pp. 351–371. Springer (2019)
16. Bracha, G.: An asynchronous $(n-1)/3$-resilient consensus protocol. In: PODC 1984. pp. 154–162 (1984)
17. Cachin, C., Kursawe, K., Shoup, V.: Random oracles in constantinople: Practical asynchronous byzantine agreement using cryptography. J. Cryptol. **18**(2), 109–147 (2000)
18. Cai, Y., Qin, C., Wang, M.: Guaranteed termination asynchronous complete secret sharing with lower communication and optimal resilience. In: ACISP 2025. vol. 15659 (2025)
19. Canetti, R.: Studies in secure multiparty computation and applications. Tech. rep., Scientific Council of The Weizmann Institute of Science (1996)
20. Canetti, R.: Security and composition of multiparty cryptographic protocols. J. Cryptol. **13**, 143–202 (2000)
21. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: FOCS 2001. pp. 136–145 (2001)
22. Canetti, R.: Universally composable security. J. ACM **67**(5), 1–94 (2020)
23. Canetti, R., Rabin, T.: Fast asynchronous byzantine agreement with optimal resilience. In: STOC 1993. pp. 42–51 (1993)
24. Choc, B., Goldwasser, S., Micali, S., Awerbuch, B.: Verifiable secret sharing and achieving simultaneity in the presence of faults. In: FOCS 1985. pp. 383–395 (1985)
25. Choudhury, A., Hirt, M., Patra, A.: Asynchronous multiparty computation with linear communication complexity. In: DISC 2013. pp. 388–402. Springer (2013)
26. Choudhury, A., Patra, A.: An efficient framework for unconditionally secure multiparty computation. IEEE Trans. Inf. Theory **63**(1), 428–468 (2017)
27. Choudhury, A., Patra, A.: On the communication efficiency of statistically secure asynchronous mpc with optimal resilience. J. Cryptol. **36**(2), 13 (2023)

28. Cohen, R.: Asynchronous secure multiparty computation in constant time. In: PKC 2016. pp. 183–207. Springer (2016)
29. Cohen, R., Forghani, P., Garay, J., Patel, R., Zikas, V.: Concurrent asynchronous byzantine agreement in expected-constant rounds, revisited. In: TCC 2023. pp. 422–451. Springer (2023)
30. Coretti, S., Garay, J., Hirt, M., Zikas, V.: Constant-round asynchronous multiparty computation based on one-way functions. In: ASIACRYPT 2016. pp. 998–1021. Springer (2016)
31. Daian, P., Goldfeder, S., Kell, T., Li, Y., Zhao, X., Bentov, I., Breidenbach, L., Juels, A.: Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In: IEEE S&P 2020. pp. 910–927. IEEE (2020)
32. Feldman, P.N., Micali, S.: An optimal algorithm for synchronous byzantine agreement. In: STOC 1988. pp. 639–648 (1988)
33. Feldman, P.N., Micali, S.: An optimal probabilistic protocol for synchronous byzantine agreement. SIAM J. Comput. **26**(4), 873–933 (1997)
34. Goyal, V., Liu-Zhang, C.D., Song, Y.: Towards achieving asynchronous mpc with linear communication and optimal resilience. In: CRYPTO 2024. pp. 170–206. Springer (2024)
35. Goyal, V., Song, Y., Zhu, C.: Guaranteed output delivery comes free in honest majority mpc. In: CRYPTO 2020. pp. 618–646. Springer (2020)
36. Ji, X., Li, J., Song, Y.: Linear-communication asynchronous complete secret sharing with optimal resilience. In: CRYPTO 2024. pp. 418–453. Springer (2024)
37. Katz, J., Maurer, U., Tackmann, B., Zikas, V.: Universally composable synchronous computation. In: TCC 2013. pp. 477–498. Springer (2013)
38. Patra, A., Choudhary, A., Pandu Rangan, C.: Efficient statistical asynchronous verifiable secret sharing with optimal resilience. In: ICITS 2009. pp. 74–92. Springer (2009)
39. Patra, A., Choudhary, A., Pandu Rangan, C.: Simple and efficient asynchronous byzantine agreement with optimal resilience. In: PODC 2009. pp. 92–101 (2009)
40. Patra, A., Choudhury, A., Pandu Rangan, C.: Asynchronous byzantine agreement with optimal resilience. Distrib. Comput. **27**(2), 111–146 (2014)
41. Patra, A., Choudhury, A., Pandu Rangan, C.: Efficient asynchronous verifiable secret sharing and multiparty computation. J. Cryptol. **28**, 49–109 (2015)
42. Pedersen, T.P.: Non-interactive and information-theoretic secure verifiable secret sharing. In: CRYPTO 1991. pp. 129–140. Springer (1992)
43. Rabin, T., Ben-Or, M.: Verifiable secret sharing and multiparty protocols with honest majority. In: STOC 1989. pp. 73–85 (1989)
44. Schwartz, J.T.: Fast probabilistic algorithms for verification of polynomial identities. vol. 27, pp. 701–717 (1980)
45. Shoup, V., Smart, N.P.: Lightweight asynchronous verifiable secret sharing with optimal resilience. J. Cryptol. **37**(3), 27 (2024)
46. Srinathan, K., Pandu Rangan, C.: Efficient asynchronous secure multiparty distributed computation. In: INDOCRYPT 2000. pp. 117–129. Springer (2000)
47. Zhang, Z., Li, W., Guo, Y., Shi, K., Chow, S.S.M., Liu, X., Dong, J.: Fast rs-iop multivariate polynomial commitments and verifiable secret sharing. In: USENIX Security 2024. pp. 3187–3204 (2024)
48. Zippel, R.: Probabilistic algorithms for sparse polynomials. pp. 216–226 (1979)

A Additional Definitions

A.1 Universal Composability

Here, we provide the complete formal details of the Universal Composability(UC) Security Model.

The Real-World Model. The real-world setting comprises n parties, denoted as $P_1, \ldots, P_n$, an adversary $\mathcal{A}$, and an environment $\mathcal{Z}$. The environment provides inputs to the honest parties, receives their outputs, and communicates directly with $\mathcal{A}$. The adversary $\mathcal{A}$ is assumed to be fully malicious, capable of corrupting up to $t < n/3$ parties, and exerting complete control over their behavior. The remaining parties, referred to as honest, are unaffected by $\mathcal{A}$. For simplicity, we consider a static adversary that determines the set of corrupted parties at the onset of the protocol.
Each party and the adversary are modeled as *interactive Turing machines* (ITMs), initialized with random coins and input values. Protocol execution proceeds as a sequence of *activations*, where only one ITM is active at any given moment. During activation, an ITM can perform local computations, generate outputs, or send messages to other parties. The adversary, when activated, can send messages on behalf of the corrupted parties.
Communication among the parties is facilitated by point-to-point asynchronous and secure channels. These asynchronous channels guarantee *eventual delivery* [23], meaning that messages sent are eventually delivered, albeit under adversarial scheduling. The adversary determines the delivery timing of messages but cannot alter, drop, or inject messages exchanged between honest parties. The behavior of such channels is formalized in the UC framework using the *eventual-delivery secure message-transmission* ideal functionality [37,30]. The protocol concludes when the environment $\mathcal{Z}$ outputs a single bit.
The output of the real-world execution is represented by the random variable $\mathsf{REAL}_{\Pi,\mathcal{A},\mathcal{Z}}(\kappa, z, \boldsymbol{r})$, where κ is the security parameter, z is the environment's input, and $\boldsymbol{r}$ represents the random tapes of all parties, the adversary, and the environment. For uniformly random $\boldsymbol{r}$, we simplify the notation as $\mathsf{REAL}_{\Pi,\mathcal{A},\mathcal{Z}}(\kappa, z)$.

The Ideal-World Model. In the idealized setting, there are n dummy parties, a simulator $\mathcal{S}$, an environment $\mathcal{Z}$, and a trusted party modeled as the ideal functionality $\mathcal{F}$. The environment interacts with the honest parties and the simulator $\mathcal{S}$, providing inputs and receiving outputs. The computation terminates once $\mathcal{Z}$ outputs a single bit.
The ideal functionality $\mathcal{F}$ represents the intended behavior of the computation. It accepts inputs from the parties and $\mathcal{S}$, computes outputs for the honest parties, and ensures that $\mathcal{S}$ cannot observe or interfere with the communication between $\mathcal{F}$ and the honest parties. To accommodate adversarial influence on output timing, we adopt a *request-based delay output model* [37,28,29]. In this model, honest parties explicitly request outputs from $\mathcal{F}$, and the adversary can delay these outputs for a polynomially bounded duration. This ensures that outputs are eventually delivered after sufficient requests.

The output of the ideal-world execution is denoted by $\mathsf{IDEAL}_{\Pi,\mathcal{S},\mathcal{Z}}(\kappa, z, \boldsymbol{r})$, where κ is the security parameter, z is the environment's input, and $\boldsymbol{r}$ includes the random tapes of $\mathcal{S}$ and $\mathcal{Z}$. For uniformly random $\boldsymbol{r}$, we denote this as $\mathsf{IDEAL}_{\Pi,\mathcal{S},\mathcal{Z}}(\kappa, z)$.

Defining Security: Perfect and Statistical. A protocol Π is said to t-securely realize $\mathcal{F}$ if, for any adversary $\mathcal{A}$, there exists a simulator $\mathcal{S}$ in the ideal world such that, for any environment $\mathcal{Z}$, the outputs of the real and ideal executions are indistinguishable:

$$\mathsf{REAL}_{\Pi,\mathcal{A},\mathcal{Z}}(\kappa, z) \equiv \mathsf{IDEAL}_{\Pi,\mathcal{S},\mathcal{Z}}(\kappa, z).$$

For statistical security, the outputs of the real and ideal executions are required to be statistically close. Specifically, the total variation distance between their distributions must not exceed $\epsilon = \mathrm{negl}(\kappa)$, where $\mathrm{negl}(\kappa)$ represents a negligible function.

The Hybrid Model. In a $\mathcal{G}$-hybrid model, the protocol execution is identical to the real-world setting, except that the parties have access to an ideal functionality $\mathcal{G}$ for specific tasks. During execution, parties can query $\mathcal{G}$ as they would in the ideal world. The UC framework guarantees composability: any ideal functionality $\mathcal{G}$ in the hybrid model can be replaced with a protocol that UC-securely realizes $\mathcal{G}$, as formalized in [21,22].

Theorem A.1 *([21,22]) Let Π be a protocol that UC-securely realizes a functionality $\mathcal{F}$ within the $\mathcal{G}$-hybrid model, and let ρ be a protocol that UC-securely realizes the ideal functionality $\mathcal{G}$. Define Π^{ρ} as the protocol derived from Π by substituting every invocation of the ideal functionality $\mathcal{G}$ with the implementation provided by ρ. Then, the protocol Π^{ρ} UC-securely realizes $\mathcal{F}$ in an execution environment where the ideal functionality $\mathcal{G}$ is no longer directly accessible to the parties.*

Hybrid Arguments. To demonstrate that the distributions of $\mathsf{REAL}_{\Pi,\mathcal{A},\mathcal{Z}}(\kappa, z)$ and $\mathsf{IDEAL}_{\Pi,\mathcal{S},\mathcal{Z}}(\kappa, z)$ are equivalent (or statistically indistinguishable), we employ hybrid argumentation. In this technique, we iteratively transform the real-world protocol execution into the ideal-world execution via a series of intermediate "hybrid worlds." Each hybrid represents a slight modification of the previous one, transitioning from real-world behavior to ideal-world behavior.
In every hybrid, the "output distribution" corresponds to the distribution of the random variable representing the output of $\mathcal{Z}$ on input z, given uniformly random $\boldsymbol{r}$. By constructing a sequence of hybrid scenarios that connect the real-world protocol to the ideal-world protocol, we ensure that the output distributions of any two consecutive hybrids are either identical or statistically close. Consequently, if the distributions of all adjacent hybrids satisfy this property, it follows that the distributions of $\mathsf{REAL}_{\Pi,\mathcal{A},\mathcal{Z}}(\kappa, z)$ and $\mathsf{IDEAL}_{\Pi,\mathcal{S},\mathcal{Z}}(\kappa, z)$ must also be equivalent (or statistically indistinguishable).

A.2 Agreement and Asynchronous Secret-Sharing Primitives

Below, we provide the formal definitions of agreement and asynchronous secret-sharing primitives.

Definition 4 (A-Cast[16]). *A-Cast is an asynchronous broadcast primitive specifically designed for networks with $n = 3t + 1$ parties. Let Π denote an asynchronous protocol initiated by a designated sender with an input message m to be broadcast. The protocol Π is considered a t-resilient A-Cast protocol if the following properties are satisfied for any adversary $\mathcal{A}_t$:*

- ***Termination:*** *The protocol ensures eventual termination under the following conditions:*
  1) *If the sender is honest and all honest parties participate in the protocol, then every honest party will eventually terminate the protocol execution.*
  2) *Regardless of the sender's behavior, if any honest party terminates the protocol, then all honest parties will eventually terminate the protocol.*
- ***Correctness:*** *Upon termination, all honest parties agree on a common output m^*. Moreover, if the sender is honest, then m^* corresponds to the sender's input message m.*

Definition 5 (Asynchronous Weak Secret Sharing (AWSS)[39]). *Let $(\mathsf{Sh}, \mathsf{Rec})$ denote a pair of protocols where a dealer $D \in \mathcal{P}$ shares a secret s using protocol Sh. The pair (Sh[13], Rec[14]) is defined as a t-resilient statistical AWSS scheme if the following conditions are satisfied:*

- ***Termination:*** *With probability at least $1-\epsilon$, the following conditions hold:*
  1) *If D is honest, every honest party will eventually terminate the execution of protocol Sh.*
  2) *If any honest party successfully terminates protocol Sh, then regardless of D's behavior, all other honest parties will also eventually terminate Sh.*
  3) *Once all honest parties have completed protocol Sh and initiated protocol Rec, every honest party will eventually complete Rec.*
- ***Correctness:*** *With probability at least $1-\epsilon$, the following correctness properties hold:*
  1) ***Correctness 1 (AWSS):*** *If D is honest, every honest party, upon completing Rec, will reconstruct the originally shared secret s.*
  2) ***Correctness 2 (AWSS):*** *If D is malicious and at least one honest party has completed Sh, then there exists a unique value $s' \in \mathbb{F} \cup \{NULL\}$ such that, upon completing Rec, each honest party will output either s' or $NULL$. This property is often referred to as weak commitment.*
- ***Secrecy:*** *If D is honest and no honest party has initiated protocol Rec, then the adversary $\mathcal{A}_t$ has no knowledge of the secret s.*

[13] Sh refers to the sharing phase protocol of the AWSS scheme.
[14] Rec refers to the reconstruction phase protocol of the AWSS scheme.

Definition 6 (AVSS [12,19]). *The definition of statistical AVSS is similar to that of statistical AWSS, with the exception that the* ***Correctness 2 (AWSS)*** *property is replaced by the following stronger property:*

- ***Correctness 2 (AVSS):*** *If the dealer D is corrupted and at least one honest party has successfully completed the protocol* Sh*, then there exists a unique value $s' \in \mathbb{F}$, such that every honest party, upon completing* Rec*, outputs exactly s'.*

Definition 1 (Asynchronous Complete Secret Sharing (ACSS)[13]). *The termination, correctness, and secrecy properties of ACSS are identical to those of AVSS. Additionally, ACSS satisfies the following completeness property at the conclusion of* Sh*, with a probability of at least $(1 - \epsilon)$:*

- ***Completeness:*** *If at least one honest party successfully completes* Sh*, then there exists a random degree-t polynomial $f(x)$ over $\mathbb{F}$, such that $f(0) = s'$ and each honest party $P_i \in \mathcal{P}$ eventually holds their share $s_i = f(i)$ of the secret s'. Furthermore, if the dealer D is honest, then $s' = s$.*

B The Asynchronous Message Checking (AMC)

B.1 The proof of Π_{AMC}

Proof. To establish the security of the AMC protocol, we introduce an ideal adversary $\mathcal{S}$ that interacts with both the environment $\mathcal{Z}$ and the ideal functionalities. The role of $\mathcal{S}$ is to simulate honest parties in the real world and concurrently execute the real-world adversary $\mathcal{A}$. For clarity and simplicity in our proof, $\mathcal{S}$ acts as a proxy for communication between honest parties and the ideal functionalities of sub-protocols.
In this simulation framework, all communications from $\mathcal{Z}$ are relayed by $\mathcal{S}$ to $\mathcal{A}$, and vice versa, ensuring accurate message flow representation. When an honest party needs to communicate with another, $\mathcal{S}$ informs $\mathcal{A}$ of the message delivery, which allows $\mathcal{A}$ to provide feedback on the message's arrival time. This interaction assists $\mathcal{S}$ in instructing the ideal functionalities to appropriately delay outputs within the ideal world. Specifically, for each output that should be delayed before being sent to an honest party, $\mathcal{S}$ maintains this delay by default until it authorizes the functionality to proceed with the transmission. Through hybrid arguments, we will demonstrate that the distribution of outputs in the ideal world is indistinguishable from those in the real world.
The construction of the ideal adversary $\mathcal{S}$ proceeds as follows: upon claiming a message has been delivered, $\mathcal{S}$ merely indicates to $\mathcal{A}$ that such delivery has taken place. It is important to note that $\mathcal{S}$ does not necessarily have access to the content of these messages, thereby preserving the confidentiality of the communications.
Furthermore, to enhance the rigor of our analysis, we employ a series of hybrid steps, where each step incrementally refines the simulation closer to the ideal execution scenario. Each hybrid step minimizes the distinguishability between

consecutive stages, culminating in a comprehensive proof that $\mathcal{S}$'s behavior accurately models the real-world adversary $\mathcal{A}$ under ideal conditions. This structured approach substantiates the robustness of the AMC protocol's security claims.

When Sig and I are honest:

Simulator $\mathcal{S}$

1. For each corrupted party $P_i \in \mathcal{C}$, $\mathcal{S}$ samples κ verification points $\{(\alpha_u^i, y_u^i)\}_{u\in[\kappa]} \xleftarrow{\$} \mathbb{F}^{2\kappa}$ and, on behalf of Sig, sends them to P_i. If these verification points are not pairwise distinct as required by the protocol, $\mathcal{S}$ aborts the simulation.
2. For each corrupted party $P_i \in \mathcal{C}$, $\mathcal{S}$ samples a fake share vector $[\boldsymbol{\alpha}]_{t\to i} \xleftarrow{\$} \mathbb{F}^n$ (component-wise, uniformly at random) and sends it to P_i on behalf of Sig. If any distinctness condition mandated by the protocol is violated (e.g., duplicated coordinates where forbidden), $\mathcal{S}$ aborts.
3. For each corrupted party $P_i \in \mathcal{C}$, $\mathcal{S}$ samples T triples of fake shares $\big\{([\boldsymbol{a}_r]_{t\to i}, [\boldsymbol{b}_r]_{t\to i}, [c_r]_{t\to i})\big\}_{r\in[T]} \xleftarrow{\$} \mathbb{F}^{2nT} \times \mathbb{F}^T$ and sends them to P_i on behalf of Sig. If these values fail any required pairwise-distinctness condition, $\mathcal{S}$ aborts.
4. For each corrupted party $P_i \in \mathcal{C}$, $\mathcal{S}$ samples m authenticated fake shares $\{[\tau_\ell]_{t\to i}\}_{\ell\in[m]} \xleftarrow{\$} \mathbb{F}^m$ and sends them to P_i on behalf of Sig. If any mandated distinctness requirement is violated, $\mathcal{S}$ aborts.
5. For each corrupted party $P_i \in \mathcal{C}$, $\mathcal{S}$ samples two independent weights $a^{(i)}, b^{(i)} \xleftarrow{\$} \mathbb{F}^2$ and, on behalf of I, broadcasts the pair $(a^{(i)}, b^{(i)})$.
6. For each $P_i \in \mathcal{P}$: if P_i is corrupted, $\mathcal{S}$ waits to receive an aggregate set $\big(\Sigma_i, \{\alpha_u^i\}_{u\in[\kappa]}\big)$ from P_i and checks that all entries are consistent with the values previously sent by $\mathcal{S}$ to corrupted parties; otherwise, it aborts. If P_i is honest, when I receives the aggregate set purportedly from P_i, $\mathcal{S}$ treats it as a correct verification set.
7. When I has received at least $2t{+}1$ correct verification sets, $\mathcal{S}$ initializes a counter $\mathsf{count} := T$ and a working set $\mathcal{W} := \emptyset$, and then allows $\mathcal{F}_{\mathsf{AMC}}$ to deliver its output to I.
8. For each revelation round while $\mathsf{count} > 0$, $\mathcal{S}$ executes the following and then sets $\mathsf{count} \leftarrow \mathsf{count} - 1$:
   - *Case R honest.*
     (a) $\mathcal{S}$ checks whether the vectors $\boldsymbol{c}, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_k$(from $\mathcal{W} = \{(\boldsymbol{c}_1, \boldsymbol{s}^1), \ldots, (\boldsymbol{c}_k, \boldsymbol{s}^k)\}$) are linearly dependent. If $\boldsymbol{c} = \sum_{j=1}^k a_j \boldsymbol{c}_j$, then it computes $\boldsymbol{s} = \sum_{j=1}^k a_j \boldsymbol{s}^j$ and proceeds according to the protocol: for each corrupted P_i, $\mathcal{S}$ sends $[\boldsymbol{s}]_{t\to i}$.
     (b) After obtaining the output from $\mathcal{F}_{\mathsf{AMC}}$, $\mathcal{S}$ samples a vector $\boldsymbol{\alpha} \xleftarrow{\$} \mathbb{F}^n$ and chooses the already sent $\{[\boldsymbol{\alpha}]_{t\to i}\}_{P_i\in\mathcal{C}}$ so that together they define a valid (t, n)-sharing with secret $\boldsymbol{\alpha}$. $\mathcal{S}$ pre-computes, for every corrupted P_i, the differences $\delta\boldsymbol{\alpha}$ and $\delta\boldsymbol{s}$ that P_i should output, waits to receive the reported $\delta'\boldsymbol{\alpha}$ and $\delta'\boldsymbol{s}$ from all cor-

rupted parties, and verifies equality with its own values. It then computes $[\boldsymbol{\alpha}]_{t\to j}$ for each honest P_j and, on behalf of each honest P_j, sends $\delta\boldsymbol{\alpha}$ and $\delta\boldsymbol{s}$ to all corrupted parties P_i. (The same procedure is applied for the T triples $\{(\boldsymbol{a}_r, \boldsymbol{b}_r, c_r)\}_{r\in[T]}$.)

(c) Based on the shares previously sent to corrupted parties, $\mathcal{S}$ pre-computes for every corrupted P_i the values $\Delta\boldsymbol{\alpha}$, $\Delta\boldsymbol{s}$, v'_i, and τ'_i. For any corrupted P_i, $\mathcal{S}$ waits to receive $\Delta'\boldsymbol{\alpha}$, $\Delta'\boldsymbol{s}$, v''_i, and τ''_i and checks that they match its pre-computed values. If the check passes, $\mathcal{S}$ deems that R has received a correct verification set. If P_i is honest, $\mathcal{S}$ regards R as having received correct shares from P_i once all shares arrive.

- *Case R corrupted.*

(a) $\mathcal{S}$ checks linear dependence of $\boldsymbol{c}, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_k$(from $\mathcal{W} = \{(\boldsymbol{c}_1, \boldsymbol{s}^1), \ldots, (\boldsymbol{c}_k, \boldsymbol{s}^k)\}$); if $\boldsymbol{c} = \sum_{j=1}^k a_j \boldsymbol{c}_j$, it computes $\boldsymbol{s} = \sum_{j=1}^k a_j \boldsymbol{s}^j$ and, as per the protocol, sends $[\boldsymbol{s}]_{t\to i}$ to each corrupted P_i, for corrupted R, $\mathcal{S}$ sends $\boldsymbol{s}$.

(b) Using the output from $\mathcal{F}_{\mathsf{AMC}}$, $\mathcal{S}$ sends $\boldsymbol{s}$ to R, samples $\boldsymbol{\alpha} \xleftarrow{\$} \mathbb{F}^n$ and ensures that the already sent $\{[\boldsymbol{\alpha}]_{t\to i}\}_{P_i\in\mathcal{C}}$ jointly define a (t, n)-sharing with secret $\boldsymbol{\alpha}$. It pre-computes $\delta\boldsymbol{\alpha}$ and $\delta\boldsymbol{s}$ for all corrupted parties, waits for their reported $\delta'\boldsymbol{\alpha}$ and $\delta'\boldsymbol{s}$, and checks consistency. It also computes $[\boldsymbol{\alpha}]_{t\to j}$ for every honest P_j and, on behalf of each honest P_j, sends $\delta\boldsymbol{\alpha}$ and $\delta\boldsymbol{s}$ to all corrupted parties. (The same procedure is applied for the T triples $\{(\boldsymbol{a}_r, \boldsymbol{b}_r, c_r)\}_{r\in[T]}$.)

(c) Based on the shares previously sent to corrupted parties, $\mathcal{S}$ pre-computes for every corrupted P_i the values $\Delta\boldsymbol{\alpha}$, $\Delta\boldsymbol{s}$, v'_i, and τ'_i.

(d) Using the output $\boldsymbol{s}$ of $\mathcal{F}_{\mathsf{AMC}}$ and the vector $\boldsymbol{\alpha} \in \mathbb{F}^n$, the simulator $\mathcal{S}$ computes, for every $\ell \in [m]$, it samples $m-1$ vectors uniformly at random,

$$\boldsymbol{s}_1, \ldots, \boldsymbol{s}_{m-1} \xleftarrow{\$} \mathbb{F}^n,$$

and chooses an index j with $c_j \neq 0$ (without loss of generality, take $j = m$). It then defines

$$\boldsymbol{s}_m \;:=\; \frac{1}{c_m}\left(\boldsymbol{s} - \sum_{\ell=1}^{m-1} c_\ell\, \boldsymbol{s}_\ell\right),$$

so that $\sum_{\ell=1}^m c_\ell\, \boldsymbol{s}_\ell = \boldsymbol{s}$. $\tau_\ell := \langle \boldsymbol{\alpha}, \boldsymbol{s}_\ell \rangle$ and then $\tau := \sum_{\ell=1}^m c_\ell\, \tau_\ell$.

(e) Since τ is the secret of a degree-t Shamir sharing, and $\mathcal{S}$ knows the t corrupted parties' shares $\{[\tau]_{t\to i}\}_{P_i\in\mathcal{C}}$ together with the secret τ itself, it has $t{+}1$ points on the sharing polynomial. Hence $\mathcal{S}$ interpolates the polynomial and evaluates it to obtain $[\tau]_{t\to j}$ for every honest party P_j.

(f) Recall that by linearity the quantities $v'_i := [c_r]_{t\to i} + \langle \Delta\boldsymbol{\alpha}, [\boldsymbol{b}_r]_{t\to i} \rangle + \langle \Delta\boldsymbol{s}, [\boldsymbol{a}_r]_{t\to i} \rangle$ form a degree-t Shamir sharing

of $v' := c_r + \langle \Delta\boldsymbol{\alpha}, \boldsymbol{b}_r \rangle + \langle \Delta\boldsymbol{s}, \boldsymbol{a}_r \rangle$. Given $v' = \tau$ (from step (1)) and the t corrupted shares $\{v'_i\}_{P_i \in \mathcal{C}}$, $\mathcal{S}$ reconstructs the sharing polynomial of v' and evaluates it to obtain v'_j for every honest P_j. Finally, using the relation

$$[c_r]_{t\to j} \;=\; v'_j \;-\; \langle \Delta\boldsymbol{\alpha}, [\boldsymbol{b}_r]_{t\to j} \rangle \;-\; \langle \Delta\boldsymbol{s}, [\boldsymbol{a}_r]_{t\to j} \rangle,$$

and the already determined shares $[\boldsymbol{a}_r]_{t\to j}$ and $[\boldsymbol{b}_r]_{t\to j}$, $\mathcal{S}$ computes $[c_r]_{t\to j}$ for every honest party P_j.

(g) For each honest P_j, the simulator $\mathcal{S}$ emulates the prescribed behavior and transmits to R the tuple $(\Delta\boldsymbol{\alpha},\ \Delta\boldsymbol{s},\ v'_j,\ \tau'_j)$.

Fig. 17: Simulator for the $\mathcal{F}_{\mathsf{AMC}}$ when both Sig and I are honest.

Hybrid Arguments:

Hyb0. In this hybrid, the simulator $\mathcal{S}$ receives the honest parties' inputs and executes the protocol faithfully. This coincides with the real-world execution.

Hyb1. In this hybrid, the simulator $\mathcal{S}$ first samples, for each corrupted party P_i, κ verification points $\{(\alpha_u^i, y_u^i)\}_{u\in[\kappa]}$ uniformly at random from $\mathbb{F} \times \mathbb{F}$ (respecting the protocol's distinctness constraints on the α's). Equivalently, across the t corrupted parties there are κt such points. In **Hyb0**, the corrupted parties' values $\{y_u^i\}$ are obtained as aggregated evaluations $y_u^i = \sum_{\ell=1}^{m} H_\ell(\alpha_u^i)\, \rho_i^{\ell-1}$, where the H_ℓ are random (due to their random masking terms) and the ρ_i's are public. Conditioned on the $\{\alpha_u^i\}$, the vector $(y_u^i)_{u\in[\kappa]}$ is therefore uniform over $\mathbb{F}^\kappa$ (and independent across corrupted parties). Thus, **Hyb1** merely changes the order of generation—sampling the verification points directly and then generating the polynomials—without affecting the joint output distribution. Consequently, **Hyb1** and **Hyb0** have identical output distributions.

Hyb2. In this hybrid, the simulator $\mathcal{S}$ first samples, for all corrupted parties, a total of nt Shamir shares uniformly at random from $\mathbb{F}$ (one share per coordinate and corrupted party), subject only to consistency with some degree-t sharing polynomial per coordinate. In **Hyb1**, $\boldsymbol{\alpha} \xleftarrow{\$} \mathbb{F}^n$ is sampled uniformly and then Shamir (t, n)-sharing is applied to obtain $[\boldsymbol{\alpha}]_t$; by the standard property of Shamir sharing, each corrupted share $[\alpha_j]_{t\to i}$ is marginally uniform over $\mathbb{F}$ (and independent across coordinates given the public evaluation points). Thus **Hyb2** merely swaps the order of generation—sampling the corrupted shares first and (if needed) completing them to a valid sharing of a uniformly random $\boldsymbol{\alpha}$—without affecting the joint output distribution. Consequently, **Hyb2** and **Hyb1** have identical output distributions.

Hyb3. In this hybrid, the simulator $\mathcal{S}$ first samples uniformly at random, from $\mathbb{F}$, a total of $(2n{+}1)Tt$ Shamir shares for all corrupted parties—namely, the t corrupted parties' shares of the T triples $\{(\boldsymbol{a}_r, \boldsymbol{b}_r, c_r)\}_{r\in[T]}$, where $\boldsymbol{a}_r, \boldsymbol{b}_r \in \mathbb{F}^n$ and $c_r \in \mathbb{F}$. (Equivalently: $2nTt$ vector-coordinate shares plus Tt scalar shares.) In **Hyb2**, the dealer Sig first samples T independent triples $\{(\boldsymbol{a}_r, \boldsymbol{b}_r, c_r)\}_{r\in[T]}$ and then applies Shamir (t, n)-sharing to obtain $\big\{([\boldsymbol{a}_r]_{t\to i}, [\boldsymbol{b}_r]_{t\to i}, [c_r]_{t\to i})\big\}_{r\in[T]}$. By the standard property of Shamir sharing, each corrupted share is marginally

uniform over $\mathbb{F}$ (conditional on the public evaluation points). Thus **Hyb3** merely swaps the order of generation—sampling the corrupted shares first and later completing them to a valid sharing—without affecting the joint output distribution. Consequently, **Hyb3** and **Hyb2** have identical output distributions.

Hyb4. In this hybrid, the simulator $\mathcal{S}$ first samples, uniformly at random from $\mathbb{F}$, a total of mt Shamir shares for the t corrupted parties—one share of each τ_ℓ for every corrupted party ($\ell \in [m]$). In **Hyb3**, the dealer Sig computes $\tau_\ell = \langle \boldsymbol{\alpha}, \boldsymbol{s}_\ell \rangle = \sum_{i=1}^{n} \alpha_i \, s_{\ell,i}$ and then applies Shamir (t, n)-sharing to obtain $[\tau_\ell]_t$. By the standard property of Shamir sharing, each corrupted share is marginally uniform over $\mathbb{F}$ (given the public evaluation points). Hence **Hyb4** merely swaps the order of generation— sampling the corrupted shares first and later completing them to a valid sharing—without affecting the joint output distribution. Consequently, **Hyb4** and **Hyb3** have identical output distributions.

Hyb5. In this hybrid, if any of the verification points or any of the randomly sampled shares for corrupted parties fail the required distinctness properties, the simulator $\mathcal{S}$ aborts the simulation. By a union bound, the collision probability is bounded as follows. Let there be κt verification points, nt vector-coordinate shares (for $\boldsymbol{\alpha}$), $(2n{+}1)Tt$ triple shares, and mt authenticated shares for corrupted parties. Since any two independent field elements collide with probability $1/|\mathbb{F}|$, we have

$$\varepsilon_1 \;\le\; \frac{1}{|\mathbb{F}|}\binom{N}{2} \;\le\; \frac{N^2}{2\,|\mathbb{F}|} \;=\; \frac{t^2\big(\kappa + n + (2n{+}1)T + m\big)^2}{2\,|\mathbb{F}|}.$$

In particular, if $|\mathbb{F}| = 2^\kappa$, then

$$\varepsilon_1 \;<\; \frac{\kappa^2 t^2 + n^2 t^2 + (2n{+}1)^2 T^2 t^2 + m^2 t^2}{2^{\kappa+1}},$$

which is negligible in κ. Hence the output distributions of **Hyb5** and **Hyb4** are statistically close (within advantage at most ε_1).

Hyb6. In this hybrid, the simulator $\mathcal{S}$ does not re-verify, on behalf of I, the verification sets sent by honest parties. Instead, whenever I receives a verification set from an honest P_i, $\mathcal{S}$ treats it as accepted. Since in these interactions both Sig and P_i are honest, the set is always correct and would be accepted by I in **Hyb5** as well. Hence **Hyb6** only changes the bookkeeping of the check and leaves the joint output distribution unchanged. Consequently, **Hyb6** and **Hyb5** have identical output distributions.

Hyb7. In this hybrid, the simulator $\mathcal{S}$ no longer re-checks, on behalf of I, the verification sets sent by corrupted parties. Instead, for each corrupted P_i it only verifies that the reported aggregate set $\big(\Sigma_i, \{\alpha_u^i\}_{u\in[\kappa]}\big)$ is consistent with the verification points previously generated (on behalf of Sig) for P_i. A deviation can affect the distribution only if a corrupted party submits an aggregate $\big(\Sigma_i', \{\alpha_u'^i\}_{u\in[\kappa]}\big)$ that is inconsistent with those pre-generated points yet still passes I's check. Conditioned on the (distinct) abscissae $\{\alpha_u'^i\}_{u\in[\kappa]}$ not all matching the pre-generated ones, the vector of evaluations $(y_1'^i, \ldots, y_\kappa'^i)$ is uniform over $\mathbb{F}^\kappa$ from I's viewpoint. Consequently, the forged scalar aggregate $\Sigma_i' \;=\; \sum_{u=1}^{\kappa} w_u^{(i)}\, y_u'^i$ with

$w_u^{(i)} = a^{(i)} \cdot u + b^{(i)}$ matches the honest value with probability at most $1/|\mathbb{F}|$. By a union bound over all corrupted submissions (at most $t\kappa$ opportunities), we get

$$\varepsilon_2 \;\le\; \frac{t\kappa}{|\mathbb{F}|}.$$

In particular, for $|\mathbb{F}| = 2^\kappa$ this yields $\varepsilon_2 \le (t\kappa)/2^\kappa$, which is negligible in κ. Hence the output distributions of **Hyb7** and **Hyb6** are statistically close.

Hyb8. In this hybrid, the simulator $\mathcal{S}$ maintains a set $\mathcal{W}$ capturing the adversary's knowledge of linear queries and their corresponding aggregate secrets revealed so far: each element is a pair $(\boldsymbol{c}, \boldsymbol{s})$, where $\boldsymbol{c}$ is the query vector (from a revelation) and $\boldsymbol{s}$ is the aggregate secret sent from I to R. Initialize $\mathcal{W} \leftarrow \emptyset$. For each revelation in which R is corrupted, let the current query be $\boldsymbol{c}$ with response $\boldsymbol{s}$. Write $\mathcal{W} = \{(\boldsymbol{c}_1, \boldsymbol{s}^1), \ldots, (\boldsymbol{c}_k, \boldsymbol{s}^k)\}$. If $\boldsymbol{c} \in \mathrm{span}_{\mathbb{F}}\{\boldsymbol{c}_1, \ldots, \boldsymbol{c}_k\}$, say $\boldsymbol{c} = \sum_{j=1}^k a_j \boldsymbol{c}_j$, then by linearity the response is already determined as $\boldsymbol{s} = \sum_{j=1}^k a_j \boldsymbol{s}^j$, so $\mathcal{S}$ does not modify $\mathcal{W}$. Otherwise (i.e., if $\boldsymbol{c}, \boldsymbol{c}_1, \ldots, \boldsymbol{c}_k$ are linearly independent), $\mathcal{S}$ augments the knowledge set by inserting the new pair: $\mathcal{W} \leftarrow \mathcal{W} \cup \{(\boldsymbol{c}, \boldsymbol{s})\}$. This change is purely bookkeeping and does not affect any messages delivered to the environment or the adversary. Therefore, **Hyb8** and **Hyb7** have identical output distributions.

Hyb9. In this hybrid, the simulator $\mathcal{S}$ no longer samples the global vector $\boldsymbol{\alpha}$ at the outset. Instead, in each revelation round it samples $\boldsymbol{\alpha} \xleftarrow{\$} \mathbb{F}^n$, computes the corresponding Shamir shares $[\boldsymbol{\alpha}]_{t\to j}$ for every honest party P_j, and—on behalf of each honest P_j— sends the prescribed messages $\delta\boldsymbol{\alpha}$ and $\delta\boldsymbol{s}$ to all corrupted parties P_i. Because $\boldsymbol{\alpha}$ is uniformly random in both hybrids and (prior to the revelation) independent of the adversary's view, deferring its sampling to the revelation step does not change the joint output distribution. Consequently, **Hyb9** and **Hyb8** have identical output distributions.

Hyb10. In this hybrid, the simulator $\mathcal{S}$ no longer samples the triples $\{(\boldsymbol{a}_r, \boldsymbol{b}_r, c_r)\}_{r\in[T]}$ at setup. Instead, in each revelation round and for every corrupted party P_i, it samples the corresponding Shamir shares $\big\{([\boldsymbol{a}_r]_{t\to i}, [\boldsymbol{b}_r]_{t\to i}, [c_r]_{t\to i})\big\}_{r\in[T]} \xleftarrow{\$} \mathbb{F}^{2nT} \times \mathbb{F}^T$, and delivers them on behalf of Sig. By the standard marginal uniformity of Shamir sharing, each corrupted share is uniformly distributed over $\mathbb{F}$ (conditioned on the public evaluation points), regardless of *when* it is sampled. Thus this modification only changes the sampling order/timing and does not affect the joint output distribution. Consequently, **Hyb10** and **Hyb9** have identical output distributions.

Hyb11. In this hybrid, for each revelation in which R is corrupted and for every honest party P_j, the dealer Sig no longer precomputes the per-ℓ shares $\{[\tau_\ell]_{t\to j}\}_{\ell\in[m]}$ at setup. Instead, given $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$ and $\boldsymbol{\alpha} \in \mathbb{F}^n$, the simulator $\mathcal{S}$ computes $\tau_\ell := \langle \boldsymbol{\alpha}, \boldsymbol{s}_\ell \rangle \quad (\ell \in [m])$, $\tau := \sum_{\ell=1}^m c_\ell\, \tau_\ell$. Using the secret τ together with the t corrupted parties' shares $\{[\tau]_{t\to i}\}_{P_i\in\mathcal{C}}$, $\mathcal{S}$ interpolates the degree-t sharing polynomial of τ and evaluates it to obtain $[\tau]_{t\to j}$ for every honest party P_j; it then sends $[\tau]_{t\to j}$ to R on behalf of P_j. Note that $[\tau]_{t\to j}$ equals $\sum_{\ell=1}^m c_\ell\, [\tau_\ell]_{t\to j}$ by linearity, so the message sent by P_j to R is distributionally identical to that in **Hyb10**. This modification only changes the time/order in

which shares are derived and does not affect the joint output distribution. Consequently, **Hyb11** and **Hyb10** have identical output distributions.

Hyb12. In this hybrid, for each revelation in which R is corrupted and for every honest party P_j, the dealer Sig no longer precomputes v'_j at setup. Instead, using $v' = \tau$ and the t corrupted parties' shares $\{v'_i\}_{P_i \in \mathcal{C}}$, the simulator $\mathcal{S}$ interpolates the degree-t sharing polynomial of v' and evaluates it to obtain v'_j for each honest P_j; it then sends v'_j to R on behalf of P_j. By the marginal uniformity of Shamir shares (conditioned on the public evaluation points), each v'_j is distributionally identical to the value produced in **Hyb11**; this change only affects the sampling order/timing and not the joint output distribution. Consequently, **Hyb12** and **Hyb11** have identical output distributions.

Hyb13. In this hybrid, for each revelation in which R is honest and for every honest party P_i, the simulator $\mathcal{S}$ does not re-verify P_i's messages on behalf of R. Instead, whenever R receives a message from P_i, $\mathcal{S}$ treats it as accepted. Since both Sig and P_i are honest, P_i always sends correct values, and thus R would accept them in **Hyb12** as well. Therefore, **Hyb13** and **Hyb12** have identical output distributions.

Hyb14. In this hybrid, for each revelation the simulator $\mathcal{S}$ does not re-check (on behalf of R) whether the scalar shares sent by corrupted parties P_i to honest parties are correct. Instead, $\mathcal{S}$ only verifies consistency with the pre-generated shares that Sig previously sent to corrupted parties. The output distribution can change only if some corrupted P_i submits a forged verification point $(\alpha'_i, \delta\boldsymbol{\alpha}, \delta\boldsymbol{s})$ whose abscissa α'_i is *not* among the corrupted parties' pre-generated abscissae, yet whose (per-coordinate) share values nevertheless match the honest evaluations at α'_i, i.e., $\delta\boldsymbol{\alpha}$ equals the correct share of $\Delta\boldsymbol{\alpha}$ at α'_i and likewise for $\delta\boldsymbol{s}$. Conditioned on all information available to the adversary, the (per-coordinate) share at any fresh point is uniform over $\mathbb{F}$; thus each such scalar equality holds with probability at most $1/|\mathbb{F}|$. Since at most $2nt$ scalar shares (for $\Delta\boldsymbol{\alpha}$ and $\Delta\boldsymbol{s}$) can be sent from corrupted parties to R across the revelation, a union bound gives

$$\varepsilon_3 \;\le\; \frac{2nt}{|\mathbb{F}|} \;=\; \frac{2nt}{2^{\kappa}},$$

which is negligible in κ. Therefore, the output distributions of **Hyb14** and **Hyb13** are statistically close.

Hyb15. In this hybrid, for each revelation in which R is honest, the simulator $\mathcal{S}$ does not re-check whether the shares sent by corrupted parties P_i are correct. Instead, it only verifies consistency with the pre-generated shares that Sig previously sent to the corrupted parties. The output distribution can change only if some corrupted P_i submits a forged verification point $(\alpha'_i, (v''_i, \tau''_i))$ whose abscissa α'_i is *not* among the pre-generated abscissae for corrupted parties, yet the reported values nevertheless match the honest evaluations at that fresh point, i.e., v''_i equals the correct share of v' at α'_i and τ''_i equals the correct share of τ at α'_i. Conditioned on the adversary's view, the share of either v' or τ at any fresh point is uniform over $\mathbb{F}$; hence each scalar equality holds with probability at most $1/|\mathbb{F}|$. Since at most $2t$ such scalars (one for v' and one for τ per corrupted

party) can be sent to R in a revelation, a union bound yields

$$\varepsilon_4 \;\le\; \frac{2t}{|\mathbb{F}|} \;=\; \frac{2t}{2^{\kappa}},$$

which is negligible in κ. Therefore, the output distributions of **Hyb15** and **Hyb14** are statistically close.

Hyb16. In this hybrid, the simulator $\mathcal{S}$ does not obtain $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$ at setup nor use them to precompute $\boldsymbol{s} = \sum_{\ell=1}^{m} c_\ell \boldsymbol{s}_\ell$ for later revelations. Instead, in each revelation where R is corrupted, $\mathcal{S}$ obtains the aggregate vector $\boldsymbol{s}$ directly from the ideal functionality's output and proceeds accordingly. Since the construction of the masking polynomials $L_1(x), \ldots, L_m(x)$ is not otherwise needed by $\mathcal{S}$, deferring knowledge of $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$ (and hence of $\boldsymbol{s}$) to the moment of revelation only changes the timing of when $\boldsymbol{s}$ is fixed, not its distribution. Consequently, **Hyb16** and **Hyb15** have identical output distributions.

Note that **Hyb16** is the ideal-world scenario,Π_{AMC} statistically-securely computes $\mathcal{F}_{\mathsf{AMC}}$.

When Sig and I are corrupted:

Simulator $\mathcal{S}$

1. For each honest $P_i \in \mathcal{H}$, the simulator $\mathcal{S}$ obtains on behalf of P_i $\{(\alpha_u^i, y_u^i)\}_{u\in[\kappa]}$, $[\boldsymbol{\alpha}]_{t\to i}$, $\{[\tau_\ell]_{t\to i}\}_{\ell\in[m]}$, $\{([\boldsymbol{a}_r]_{t\to i}, [\boldsymbol{b}_r]_{t\to i}, [c_r]_{t\to i})\}_{r\in[T]}$, $(a^{(i)}, b^{(i)})$, and then sends $(\Sigma_i, \{\alpha_u^i\}_{u\in[\kappa]})$ to I on behalf of P_i.
2. $\mathcal{S}$ sets $\boldsymbol{s}_1 = \cdots = \boldsymbol{s}_m = \mathbf{0}$ (the all-zero vector in $\mathbb{F}^n$) and, on behalf of Sig, sends (Init, AMC, T, $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$) to $\mathcal{F}_{\mathsf{AMC}}$.
3. $\mathcal{S}$ initializes a counter $\mathsf{count} := T$.
4. For each revelation, while $\mathsf{count} > 0$, $\mathcal{S}$ executes the following and then sets $\mathsf{count} \leftarrow \mathsf{count} - 1$:
   1) **Case R honest.**
      i. Acting on behalf of R, $\mathcal{S}$ receives $\boldsymbol{s}'$ from I.
      ii. For every honest $P_i \in \mathcal{H}$, $\mathcal{S}$ (on behalf of P_i) receives $[\boldsymbol{s}]_{t\to i}$ from I and sends $\delta\boldsymbol{\alpha}$ and $\delta\boldsymbol{s}$ to all corrupted parties.
      iii. For each corrupted $P_i \in \mathcal{C}$, $\mathcal{S}$ (on behalf of honest parties) waits to receive $\delta\boldsymbol{\alpha}$ and $\delta\boldsymbol{s}$; and (on behalf of honest R) receives from corrupted parties $\Delta\boldsymbol{\alpha}$, $\Delta\boldsymbol{s}$, v_i', and τ_i'. For each honest $P_i \in \mathcal{H}$, $\mathcal{S}$ uses the messages from Sig and from all corrupted parties to compute the corresponding $\Delta\boldsymbol{\alpha}$, $\Delta\boldsymbol{s}$, v_i', and τ_i'. In both cases, $\mathcal{S}$ checks whether the reconstructed values satisfy $v = \tau$. If the check passes, $\mathcal{S}$ deems that R has received correct shares.
      iv. Once shares from at least $2t{+}1$ parties satisfy $v = \tau$, $\mathcal{S}$ sends Proceed and $\boldsymbol{s}'$ to $\mathcal{F}_{\mathsf{AMC}}$ and allows $\mathcal{F}_{\mathsf{AMC}}$ to deliver its output to R. If $\mathcal{S}$ observes $2t{+}1$ inconsistent verification shares, it sends Ignore to $\mathcal{F}_{\mathsf{AMC}}$.
   2) **Case R corrupted.**

i. For every honest $P_i \in \mathcal{H}$, $\mathcal{S}$ (on behalf of P_i) receives $[\boldsymbol{s}]_{t\to i}$ from I and sends $\delta\boldsymbol{\alpha}$ and $\delta\boldsymbol{s}$ to all corrupted parties.
ii. $\mathcal{S}$ (on behalf of honest parties) waits to receive $\delta\boldsymbol{\alpha}$ and $\delta\boldsymbol{s}$ from all corrupted parties.
iii. For each honest $P_i \in \mathcal{H}$, $\mathcal{S}$ uses the messages from Sig together with all messages sent by corrupted parties to compute $\Delta\boldsymbol{\alpha}$, $\Delta\boldsymbol{s}$, v_i', and τ_i', and then sends these to the corrupted R on behalf of P_i.

Fig. 18: Simulator for the $\mathcal{F}_{\mathsf{AMC}}$ when Sig and I are corrupted.

Hybrid arguments:

Hyb0. In this hybrid, the simulator $\mathcal{S}$ receives the honest parties' inputs and executes the protocol faithfully. This coincides with the real-world execution.

Hyb1. In this hybrid, we change only how R obtains $\boldsymbol{s}'$. After receiving at least $2t+1$ correct verification shares, $\mathcal{S}$ sends $\mathsf{Proceed}$ and $\boldsymbol{s}'$ to $\mathcal{F}_{\mathsf{AMC}}$ and lets R receive $\boldsymbol{s}'$ as the functionality's output. Thus the only difference from **Hyb0** is whether R obtains $\boldsymbol{s}'$ directly (from $\mathcal{S}$) or via $\mathcal{F}_{\mathsf{AMC}}$; this change in delivery path does not affect the joint output distribution. Consequently, **Hyb1** and **Hyb0** have identical output distributions.

Note that **Hyb1** is the ideal-world scenario,Π_{AMC} statistically-securely computes $\mathcal{F}_{\mathsf{AMC}}$.

When Sig is honest and I is corrupted:

Simulator $\mathcal{S}$

1. $\mathcal{S}$ waits to receive $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$ from $\mathcal{F}_{\mathsf{AMC}}$. For each $\ell \in [m]$, $\mathcal{S}$ samples a random polynomial $L_\ell(x)$ of degree at most n whose highest n coefficients form the vector $\boldsymbol{s}_\ell = (s_{\ell,1}, \ldots, s_{\ell,n})$.
2. For each corrupted P_i, $\mathcal{S}$ samples κ distinct elements $(\alpha_1^i, \ldots, \alpha_\kappa^i) \xleftarrow{\$} \mathbb{F}$. Using these points and the polynomials L_ℓ from step (1), $\mathcal{S}$ constructs, for every $\ell \in [m]$, a masking polynomial $H_\ell(x)$ of degree at most $(n+n\kappa)$ (as prescribed by the protocol), and sends to P_i the verification set $\{(\alpha_u^i,\, y_u^i)\}_{u\in[\kappa]}$ where $y_u^i := \sum_{\ell=1}^m L_\ell(\alpha_u^i)\, \rho_i^{\ell-1}$, acting on behalf of Sig.
3. For each corrupted P_i, $\mathcal{S}$ follows the protocol to sample $\boldsymbol{\alpha} \xleftarrow{\$} \mathbb{F}^n$ and $\{(\boldsymbol{a}_r, \boldsymbol{b}_r, c_r)\}_{r\in[T]}$ with $\boldsymbol{a}_r, \boldsymbol{b}_r \xleftarrow{\$} \mathbb{F}^n$ and $c_r \xleftarrow{\$} \mathbb{F}$, compute $\tau_\ell := \langle \boldsymbol{\alpha}, \boldsymbol{s}_\ell \rangle$ for all $\ell \in [m]$, and Shamir-share all required quantities to the corresponding corrupted parties: $[\boldsymbol{\alpha}]_{t\to i}$, $\{[\tau_\ell]_{t\to i}\}_{\ell\in[m]}$, and $\{([\boldsymbol{a}_r]_{t\to i}, [\boldsymbol{b}_r]_{t\to i}, [c_r]_{t\to i})\}_{r\in[T]}$.
4. Acting on behalf of Sig, $\mathcal{S}$ sends $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$ to I.
5. For each honest P_i, upon receiving public weights $(a^{(i)}, b^{(i)})$ from I, $\mathcal{S}$ computes the linear aggregate $\Sigma_i := \sum_{u=1}^\kappa w_u^{(i)}\, y_u^i$ with $w_u^{(i)} := a^{(i)}u + b^{(i)}$, and sends $\big(\Sigma_i, \{\alpha_u^i\}_{u\in[\kappa]}\big)$ to I on behalf of P_i.
6. $\mathcal{S}$ initializes a counter $\mathsf{count} := T$.
7. For each revelation, $\mathcal{S}$ computes the aggregate $\boldsymbol{s} := \sum_{\ell=1}^m c_\ell \boldsymbol{s}_\ell$. If $\mathsf{count} > 0$, it performs the following and then sets $\mathsf{count} \leftarrow \mathsf{count} - 1$:

1) **Case R honest.**
   i. On behalf of R, $\mathcal{S}$ receives $\boldsymbol{s}'$ from I.
   ii. $\mathcal{S}$ checks whether $\boldsymbol{s} = \boldsymbol{s}'$. If so, it sends Proceed and $\boldsymbol{s}$ to $\mathcal{F}_{\mathsf{AMC}}$; otherwise, it sends Ignore.
   iii. If $\boldsymbol{s} = \boldsymbol{s}'$, then for every honest P_i the simulator (on behalf of P_i) receives $[\boldsymbol{s}]_{t\to i}$ from I and sends $\delta\boldsymbol{\alpha}$ and $\delta\boldsymbol{s}$ to all corrupted parties.
   iv. For each corrupted $P_i \in \mathcal{P}$, $\mathcal{S}$ (on behalf of the honest parties) waits to receive $\delta\boldsymbol{\alpha}$ and $\delta\boldsymbol{s}$; and (on behalf of the honest R) receives from corrupted parties $\Delta\boldsymbol{\alpha}$, $\Delta\boldsymbol{s}$, v_i', and τ_i'. For each honest P_i, $\mathcal{S}$ treats the verification shares as accepted upon receipt and checks whether the relation $v = \tau$ holds. If it holds, $\mathcal{S}$ deems that R has received correct shares.
   v. Once shares from at least $2t{+}1$ parties satisfy $v = \tau$, $\mathcal{S}$ allows $\mathcal{F}_{\mathsf{AMC}}$ to deliver the output to R.
2) **Case R corrupted.**
   i. For every honest P_i, $\mathcal{S}$ (on behalf of P_i) receives $[\boldsymbol{s}]_{t\to i}$ from I and sends $\delta\boldsymbol{\alpha}$ and $\delta\boldsymbol{s}$ to all corrupted parties.
   ii. $\mathcal{S}$ (on behalf of the honest parties) waits to receive $\delta\boldsymbol{\alpha}$ and $\delta\boldsymbol{s}$ from all corrupted parties.
   iii. For each honest $P_i \in \mathcal{P}$, using the shares sent to corrupted parties together with the secrets τ and v known to the simulator, $\mathcal{S}$ computes the corresponding $\Delta\boldsymbol{\alpha}$, $\Delta\boldsymbol{s}$, v_i', and τ_i', and sends these to the corrupted R on behalf of P_i.

Fig. 19: Simulator for the $\mathcal{F}_{\mathsf{AMC}}$ when Sig is honest and I is corrupted.

Hybrid arguments:

Hyb0. In this hybrid, the simulator $\mathcal{S}$ receives the honest parties' inputs and executes the protocol faithfully. This coincides with the real-world execution.

Hyb1. In this hybrid, the simulator $\mathcal{S}$ changes only how it obtains the inputs used to compute the corrupted parties' verification shares: instead of using the honest dealer's inputs, $\mathcal{S}$ waits to receive $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$ from $\mathcal{F}_{\mathsf{AMC}}$. The sole difference from **Hyb0** is this source of $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$, which does not affect the joint output distribution. Consequently, **Hyb1** and **Hyb0** have identical output distributions.

Hyb2. In this hybrid, for each honest party P_i the simulator $\mathcal{S}$ does not presample all randomness at setup to produce the verification-related shares. Instead, when an honest P_i needs the values v_i' or τ_i', $\mathcal{S}$ derives them *on demand* from the messages already sent to corrupted parties together with the secrets τ and v (known to the simulator), using linearity and interpolation of degree-t Shamir sharings. This change only reorders when the underlying randomness is fixed and therefore does not affect the joint output distribution. Consequently, **Hyb2** and **Hyb1** have identical output distributions.

Hyb3. In this hybrid, when R is honest, the simulator $\mathcal{S}$ adds an *additional* check: upon receiving $\boldsymbol{s}'$ from I, it verifies whether $\boldsymbol{s} = \boldsymbol{s}'$. If so, $\mathcal{S}$ sends Proceed and $\boldsymbol{s}$ to $\mathcal{F}_{\mathsf{AMC}}$; otherwise, it sends Ignore. The only way this changes the distri-

bution relative to **Hyb2** is when $\boldsymbol{s} \neq \boldsymbol{s}'$ yet $\mathcal{S}$ would still collect $2t{+}1$ correct shares. Write $\boldsymbol{s} = \sum_k c_k\, \boldsymbol{s}_k$ and note that acceptance in **Hyb2** would entail $\langle \boldsymbol{\alpha}, \boldsymbol{s}' \rangle = \sum_k c_k \langle \boldsymbol{\alpha}, \boldsymbol{s}_k \rangle = \sum_k c_k\, \tau_k$ with $\tau_k := \langle \boldsymbol{\alpha}, \boldsymbol{s}_k \rangle$. Equivalently, with $\boldsymbol{v} := \boldsymbol{s}' - \sum_k c_k\, \boldsymbol{s}_k \neq \boldsymbol{0}$, the event is $\langle \boldsymbol{\alpha}, \boldsymbol{v} \rangle = 0$. For uniformly random $\boldsymbol{\alpha} \xleftarrow{\$} \mathbb{F}^n$ and fixed nonzero $\boldsymbol{v}$, the inner product $\langle \boldsymbol{\alpha}, \boldsymbol{v} \rangle$ is uniform over $\mathbb{F}$, so

$$\varepsilon_5 \;=\; \Pr\left[\langle \boldsymbol{\alpha}, \boldsymbol{s}' \rangle = \textstyle\sum_k c_k\, \tau_k\right] \;=\; \frac{1}{|\mathbb{F}|} \;=\; \frac{1}{2^{\kappa}},$$

which is negligible in κ. Therefore, the output distributions of **Hyb3** and **Hyb2** are statistically close.

Hyb4. In this hybrid, for each revelation in which R is honest and for every honest party P_i, the simulator $\mathcal{S}$ does not re-verify (on behalf of R) the verification shares sent by P_i. Instead, whenever R receives P_i's verification shares, $\mathcal{S}$ treats them as accepted. Since both Sig and P_i are honest and, by construction, $\boldsymbol{s} = \boldsymbol{s}'$, P_i always sends a correct verification set; hence R would accept it in **Hyb3** as well. Therefore, **Hyb4** and **Hyb3** have identical output distributions.

Hyb5. In this hybrid, we change only how R obtains $\boldsymbol{s}$. Once at least $2t{+}1$ correct shares have been received and $\boldsymbol{s} = \boldsymbol{s}'$ holds, the simulator $\mathcal{S}$ instructs $\mathcal{F}_{\mathsf{AMC}}$ to deliver $\boldsymbol{s}$ to R. Moreover, the vector $\boldsymbol{s}$ computed by $\mathcal{S}$ equals the one computed by $\mathcal{F}_{\mathsf{AMC}}$, so this modification affects only the delivery path and not the joint output distribution. Consequently, **Hyb5** and **Hyb4** have identical output distributions.

Note that **Hyb5** is the ideal-world scenario,Π_{AMC} statistically-securely computes $\mathcal{F}_{\mathsf{AMC}}$.

When Sig is corrupted and I is honest:

Simulator $\mathcal{S}$

1. For each honest $P_i \in \mathcal{H}$, $\mathcal{S}$ waits to receive the verification points and shares from Sig on behalf of P_i.
2. $\mathcal{S}$ receives $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$ from Sig and proceeds as follows.
   1) For every corrupted $P_i \in \mathcal{C}$, $\mathcal{S}$ samples two weights $(a^{(i)}, b^{(i)}) \xleftarrow{\$} \mathbb{F}^2$ and sends them to P_i.
   2) For each $P_i \in \mathcal{P}$:
      i. If P_i is corrupted, $\mathcal{S}$ waits to receive an aggregate verification set $\big(\Sigma_i, \{\alpha_u^i\}_{u\in[\kappa]}\big)$ from P_i. If all verification points in the set are consistent with $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$, then $\mathcal{S}$ regards I as having received a correct verification set.
      ii. If P_i is honest, $\mathcal{S}$ uses its own (simulated) $\big(\Sigma_i, \{\alpha_u^i\}_{u\in[\kappa]}\big)$. When this set is consistent with $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$, $\mathcal{S}$ likewise considers that I has received a correct verification set.
   3) When I has received $2t{+}1$ correct verification sets, $\mathcal{S}$ initializes a counter $\mathsf{count} := T$ and sends $(\mathsf{Init}, \mathsf{AMC}, T, (\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m))$ to $\mathcal{F}_{\mathsf{AMC}}$, then allows $\mathcal{F}_{\mathsf{AMC}}$ to deliver its output to I. Otherwise, $\mathcal{S}$ does not proceed.
3. For each revelation, $\mathcal{S}$ computes $\boldsymbol{s} = \sum_{\ell=1}^{m} c_\ell\, \boldsymbol{s}_\ell$. While $\mathsf{count} > 0$, $\mathcal{S}$ executes the following and then sets $\mathsf{count} \leftarrow \mathsf{count} - 1$:

1) **Case R honest.**
   i. Acting on behalf of I, $\mathcal{S}$ sends $[\boldsymbol{s}]_{t\to i}$ to each corrupted $P_i \in \mathcal{C}$.
   ii. For every corrupted $P_i \in \mathcal{C}$, $\mathcal{S}$ (on behalf of each honest party) computes and sends the corresponding $\delta\boldsymbol{\alpha}$ and $\delta\boldsymbol{s}$ to P_i.
   iii. For each $P_i \in \mathcal{P}$: if P_i is corrupted, $\mathcal{S}$ waits to receive the verification shares v_i', τ_i' from P_i; if P_i is honest, $\mathcal{S}$ computes the corresponding v_i', τ_i' using the messages from Sig together with those sent by corrupted parties. In both cases, $\mathcal{S}$ checks whether the shares from at least $2t+1$ parties are consistent with $\boldsymbol{s}$.
   iv. Once at least $2t+1$ parties' shares pass the check, $\mathcal{S}$ allows $\mathcal{F}_{\mathsf{AMC}}$ to deliver the output to R.
2) **Case R corrupted.**
   i. On behalf of I, $\mathcal{S}$ sends $\boldsymbol{s}$ to R.
   ii. Acting on behalf of I, $\mathcal{S}$ sends $[\boldsymbol{s}]_{t\to i}$ to each corrupted $P_i \in \mathcal{C}$.
   iii. For every corrupted $P_i \in \mathcal{C}$, $\mathcal{S}$ (on behalf of each honest party) computes and sends the corresponding $\delta\boldsymbol{\alpha}$ and $\delta\boldsymbol{s}$ to P_i.
   iv. Using the messages from Sig and from the corrupted parties, $\mathcal{S}$ computes, for each honest $P_i \in \mathcal{H}$, the values v_i' and τ_i', and sends them to the corrupted receiver R on behalf of P_i.

Fig. 20: Simulator for the $\mathcal{F}_{\mathsf{AMC}}$ when Sig is corrupted and I is hones.

Hybrid arguments:

Hyb0. In this hybrid, the simulator $\mathcal{S}$ receives the honest parties' inputs and executes the protocol faithfully. This coincides with the real-world execution.

Hyb1. In this hybrid, we change only how I obtains $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$. After receiving at least $2t+1$ correct verification sets, the simulator $\mathcal{S}$ sends $(\mathsf{Init}, \mathsf{AMC}, T, (\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m))$ to $\mathcal{F}_{\mathsf{AMC}}$. Thus, I receives $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$ from $\mathcal{F}_{\mathsf{AMC}}$ rather than reconstructing them locally. This change affects only the delivery path and does not alter the joint output distribution. Consequently, **Hyb1** and **Hyb0** have identical output distributions.

Hyb2. In this hybrid, the simulator $\mathcal{S}$ adds an extra consistency check for each honest party P_i's verification points: $\mathcal{S}$ also verifies that the reported points are consistent with $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$. If this holds, $\mathcal{S}$ deems the set *correct*. This can change the distribution only if an honest P_i happens to hold *inconsistent* verification points $\{(\alpha_u^i, y_u^i)\}_{u\in[\kappa]}$ but nevertheless provides an aggregate $\big(\Sigma_i, \{\alpha_u^i\}_{u\in[\kappa]}\big)$ that matches the honest value received by I. Writing a potentially inconsistent submission with primes as $\{\alpha_u'^i, y_u'^i\}_{u\in[\kappa]}$, and letting $w_u^{(i)} = a^{(i)}u + b^{(i)}$ be the (public) weights, the bad event is

$$\widehat{\Sigma}_i \;=\; \sum_{u=1}^{\kappa} w_u^{(i)} \Big(\sum_{\ell=1}^{m} L_\ell(\alpha_u'^i)\, \rho_i^{\ell-1} \Big) \;=\; \Sigma_i,$$

which, from the verifier's viewpoint, holds with probability at most

$$\varepsilon_6 \;=\; \frac{\kappa}{|\mathbb{F}|} \;=\; \frac{\kappa}{2^\kappa}.$$

Since $\mathcal{S}$ must accept at least $2t+1$ correct sets, a union bound over the (at most) t relevant opportunities gives a total failure probability at most $t\,\varepsilon_6$, which remains negligible in κ. Therefore, the output distributions of **Hyb2** and **Hyb1** are statistically close.
Note that $\mathbf{Game}_2$ corresponds to the ideal-world execution, where the protocol Π_{AMC} achieves a statistical security guarantee for securely computing the functionality $\mathcal{F}_{\mathsf{AMC}}$.

B.2 Analysis of the Communication Complexity

We compute the communication complexity of our protocol as follows. Π_{AMC} takes m vectors as inputs, and each vector is of size n. T is an input parameter. All parties execute the Initialization Phase only once, while the Revelation Phase can be executed for at most T times.

1. During the Initialization Phase:
   1) Sig sends $\{(\alpha_u^i, y_u^i)\}_{u\in[\kappa]}$, $[\boldsymbol{\alpha}]_{t\to i}$, $\{[\tau_\ell]_{t\to i}\}_{\ell\in[m]}$, $\{([\boldsymbol{a}_r]_{t\to i}, [\boldsymbol{b}_r]_{t\to i}, [c_r]_{t\to i})\}_{r\in[T]}$ to each party P_i, which requires $\mathcal{O}(mn\kappa + n^3\kappa + n\kappa^2)$ bits.
   2) Sig sends $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$ to I, which requires $\mathcal{O}(mn\kappa)$ bits.
   3) I broadcasts the pair $\{(a^{(i)}, b^{(i)})\}_{i\in[n]}$, which requires $\mathcal{O}(n^2\kappa + n^2\log n)$ bits.
   4) Each P_i, sends $\big(\Sigma_i, \{\alpha_u^i\}_{u\in[\kappa]}\big)$ to I, which requires $\mathcal{O}(n\kappa^2)$ bits.
   5) Therefore, the total communication cost is $\mathcal{O}(mn\kappa + n^3\kappa + n\kappa^2)$ bits during the *Initialization Phase*.
2. During the Revelation Phase:
   1) Each time, I sends $\boldsymbol{s}$ to R, which requires $\mathcal{O}(n\kappa)$ bits.
   2) Each time, I sends the corresponding share $[\boldsymbol{s}]_{t\to i}$ to party P_i for all $i \in [n]$, which requires $\mathcal{O}(n^2\kappa)$ bits.
   3) Each time, each party P_i sends $\delta\boldsymbol{\alpha}, \delta\boldsymbol{s}$ to all parties P_j with $j \in [n]$, which requires $\mathcal{O}(n^3\kappa)$ bits.
   4) Each time, each party P_i forwards $\Delta\boldsymbol{\alpha}, \Delta\boldsymbol{s}$ to R, which require $\mathcal{O}(n^2\kappa)$ bits.
   5) Each time, each party P_i sends v_i' to R, which requires $\mathcal{O}(n\kappa)$ bits.
   6) Each time, each party P_i sends τ_i' to R, which requires $\mathcal{O}(n\kappa)$ bits.
   7) Therefore, the total communication cost per revelation is $\mathcal{O}(n^3\kappa)$ bits. Since the *Revelation Phase* can be executed at most T times, the total communication cost is $\mathcal{O}(Tn^3\kappa)$ bits. As $T = 2n$, this simplifies to $\mathcal{O}(n^4\kappa)$ bits.

Therefore, Π_{AMC} requires communication of $\mathcal{O}(mn\kappa + n^4\kappa + n\kappa^2)$ bits in total.

C The Protocol of AcceptPoly

C.1 The Proof of $\Pi_{\mathsf{AcceptPoly}}$

Proof. To establish the security of the AcceptShare protocol, we introduce an ideal adversary $\mathcal{S}$ that interacts with both the environment $\mathcal{Z}$ and the ideal

functionalities. The role of $\mathcal{S}$ is to simulate honest parties in the real world and concurrently execute the real-world adversary $\mathcal{A}$. For clarity and simplicity in our proof, $\mathcal{S}$ acts as a proxy for communication between honest parties and the ideal functionalities of sub-protocols.

In this simulation framework, all communications from $\mathcal{Z}$ are relayed by $\mathcal{S}$ to $\mathcal{A}$, and vice versa, ensuring accurate message flow representation. When an honest party needs to communicate with another, $\mathcal{S}$ informs $\mathcal{A}$ of the message delivery, which allows $\mathcal{A}$ to provide feedback on the message's arrival time. This interaction assists $\mathcal{S}$ in instructing the ideal functionalities to appropriately delay outputs within the ideal world. Specifically, for each output that should be delayed before being sent to an honest party, $\mathcal{S}$ maintains this delay by default until it authorizes the functionality to proceed with the transmission. Through hybrid arguments, we will demonstrate that the distribution of outputs in the ideal world is indistinguishable from those in the real world.

The construction of the ideal adversary $\mathcal{S}$ proceeds as follows: upon claiming a message has been delivered, $\mathcal{S}$ merely indicates to $\mathcal{A}$ that such delivery has taken place. It is important to note that $\mathcal{S}$ does not necessarily have access to the content of these messages, thereby preserving the confidentiality of the communications.

Furthermore, to enhance the rigor of our analysis, we employ a series of hybrid steps, where each step incrementally refines the simulation closer to the ideal execution scenario. Each hybrid step minimizes the distinguishability between consecutive stages, culminating in a comprehensive proof that $\mathcal{S}$'s behavior accurately models the real-world adversary $\mathcal{A}$ under ideal conditions. This structured approach substantiates the robustness of the Accept Share protocol's security claims.

When *Sender* **is honest:**

Simulator $\mathcal{S}$

1. $\mathcal{S}$ receives from $\mathcal{F}_{\mathsf{AcceptPoly}}$ the verification polynomials $s'_i(r)$ for all corrupted parties $P_i \in \mathcal{C}$, and forwards $s'_i(r)$ to the corresponding P_i.
2. $\mathcal{S}$ initializes a counter $\mathsf{count} := T$ and a knowledge set $\mathcal{W} := \emptyset$.
3. For each revelation, while $\mathsf{count} > 0$, $\mathcal{S}$ executes the following and then sets $\mathsf{count} \leftarrow \mathsf{count} - 1$.
   1) **Case:** Acceptor P_j **honest.**
      (a) For every corrupted $P_i \in \mathcal{C}$ and the current challenge $c \in \mathbb{F}$, $\mathcal{S}$ evaluates $s'_i(c) := s'_i(r)\big|_{r=c}$. Let $\mathcal{M} := \{(\alpha^i,\, s'_i(c)) : P_i \in \mathcal{C}\}$ denote the multiset of abscissa–value pairs considered in this round. With the current knowledge $\mathcal{W} = \{(\boldsymbol{c}_1,\, s'_i(c_1)), \ldots, (\boldsymbol{c}_k,\, s'_i(c_k))\}$, $\mathcal{S}$ tests whether $\boldsymbol{c} \in \mathrm{span}_{\mathbb{F}}\{\boldsymbol{c}_1, \ldots, \boldsymbol{c}_k\}$. If so, say $\boldsymbol{c} = \sum_{j=1}^{k} a_j \boldsymbol{c}_j$, it sets $s'_i(c) := \sum_{j=1}^{k} a_j\, s'_i(c_j)$; otherwise, it augments $\mathcal{W} \leftarrow \mathcal{W} \cup \{(\boldsymbol{c},\, s'_i(c))\}$.
      (b) For each $P_i \in \mathcal{P}$:
         A. If P_i is corrupted, $\mathcal{S}$ waits to receive a purported value $\Delta_{\mathsf{Sender}\to i}(c)$ from P_i. If $\boldsymbol{c} \notin \mathrm{span}_{\mathbb{F}}\{\boldsymbol{c}_1, \ldots, \boldsymbol{c}_k\}$, it checks

whether $\Delta_{\mathsf{Sender}\to i}(c) \in \mathcal{M}$; if $\boldsymbol{c} \in \mathrm{span}_{\mathbb{F}}\{\boldsymbol{c}_1, \ldots, \boldsymbol{c}_k\}$, it checks whether $\Delta_{\mathsf{Sender}\to i}(c) = s'_i(c)$. If consistent with the honest sender's prior messages, $\mathcal{S}$ concludes that Acceptor has received a correct verification value.

B. If P_i is honest, $\mathcal{S}$ regards $\Delta_{\mathsf{Sender}\to i}(c)$ as correct upon arrival.

(c) If P_i is corrupted, $\mathcal{S}$ faithfully emulates the interactions with $\mathcal{F}_{\mathsf{AMC}}$ on behalf of Sender as required by the protocol.

(d) Upon receiving at least $2t+1$ correct verification values, $\mathcal{S}$ permits $\mathcal{F}_{\mathsf{AcceptPoly}}$ to deliver its output to Acceptor.

2) **Case: Acceptor corrupted.**

i. For every corrupted $P_i \in \mathcal{C}$, $\mathcal{S}$ computes the evaluation $s'_i(c) := s'_i(r)\big|_{r=c}$ and forms the current multiset $\mathcal{M} := \{(\alpha^i, s'_i(c)) : P_i \in \mathcal{C}\}$. With $\mathcal{W} = \{(\boldsymbol{c}_1, s'_i(c_1)), \ldots, (\boldsymbol{c}_k, s'_i(c_k))\}$ as above, it checks whether $\boldsymbol{c} \in \mathrm{span}_{\mathbb{F}}\{\boldsymbol{c}_1, \ldots, \boldsymbol{c}_k\}$.

ii. If $\boldsymbol{c} \in \mathrm{span}_{\mathbb{F}}\{\boldsymbol{c}_1, \ldots, \boldsymbol{c}_k\}$, say $\boldsymbol{c} = \sum_{j=1}^{k} a_j \boldsymbol{c}_j$, then $\mathcal{S}$ sets $s'_i(c) := \sum_{j=1}^{k} a_j\, s'_i(c_j)$.

iii. If $\boldsymbol{c} \notin \mathrm{span}_{\mathbb{F}}\{\boldsymbol{c}_1, \ldots, \boldsymbol{c}_k\}$, then for each honest P_j the simulator samples a fresh value $\Delta_{\mathsf{Sender}\to j}(c) \xleftarrow{\$} \mathbb{F}$ and, using the honest parties' values $\{\Delta_{\mathsf{Sender}\to j}(c)\}$ together with the corrupted polynomials $\{s'_i(r)\}$, constructs (on behalf of $\mathcal{F}_{\mathsf{AMC}}$) the authenticated evaluation vector $\boldsymbol{g}_{\mathsf{Sender}} = (g_{\mathsf{Sender}}(\alpha_1), \ldots, g_{\mathsf{Sender}}(\alpha_n))$, which it sends to the corrupted Acceptor.

iv. Finally, $\mathcal{S}$ sends $\Delta_{\mathsf{Sender}\to j}(c)$ to the corrupted Acceptor on behalf of each honest party P_j.

Fig. 21: Simulator for the $\mathcal{F}_{\mathsf{AcceptPoly}}$ when *Sender* is honest.

Hybrid Arguments:

Hyb0. In this hybird, $\mathcal{S}$ behaves as in the real-world execution by using honest inputs and following the protocol correctly.

Hyb1. In this hybrid, the simulator $\mathcal{S}$ changes only how it obtains a corrupted party's verification polynomial. Instead of using the honest Sender's inputs to construct it, $\mathcal{S}$ waits to receive $s'_c(r)$ for the corrupted party P_c from $\mathcal{F}_{\mathsf{AcceptPoly}}$ (and forwards it as prescribed). This modification merely alters the source of the polynomial and does not affect its distribution; hence the joint output distribution is unchanged. Therefore, **Hyb1** and **Hyb0** are identical in terms of output distribution.

Hyb2. In this hybrid, for each honest party P_i, the simulator $\mathcal{S}$ does not precompute that party's verification polynomial at setup; instead, it derives it *on demand* when first needed. This change merely reorders the timing of computations and does not affect the joint output distribution. Consequently, **Hyb2** and **Hyb1** have identical output distributions.

Hyb3. In this hybrid, the simulator $\mathcal{S}$ maintains a knowledge set $\mathcal{W}$ that tracks what the adversary has learned from prior revelations. Each element of $\mathcal{W}$ is a pair $(\boldsymbol{c}, s'_i(c))$ consisting of a query vector $\boldsymbol{c}$ and the corresponding evaluation

$s'_i(c)$ that was sent from Sender to the Acceptor P_i in earlier rounds. Initially, $\mathcal{W} = \emptyset$.

During a revelation in which the Acceptor P_j is corrupted, $\mathcal{S}$ updates $\mathcal{W}$ as follows. Let $\mathcal{W} = \{(\boldsymbol{c}_1, s'_i(c_1)), \ldots, (\boldsymbol{c}_k, s'_i(c_k))\}$ denote the current contents. If $\boldsymbol{c} \notin \mathrm{span}_{\mathbb{F}}\{\boldsymbol{c}_1, \ldots, \boldsymbol{c}_k\}$, then $\mathcal{S}$ appends $(\boldsymbol{c}, s'_i(c))$ to $\mathcal{W}$ since $s'_i(c)$ cannot be deduced from the previously revealed evaluations. Conversely, if $\boldsymbol{c} \in \mathrm{span}_{\mathbb{F}}\{\boldsymbol{c}_1, \ldots, \boldsymbol{c}_k\}$, say $\boldsymbol{c} = \sum_{j=1}^{k} a_j \boldsymbol{c}_j$, then by linearity the adversary can predict $s'_i(c) = \sum_{j=1}^{k} a_j\, s'_i(c_j)$, and no update to $\mathcal{W}$ is needed.

Constructing and maintaining $\mathcal{W}$ is purely bookkeeping and does not change any messages delivered to the environment or the adversary. Therefore, **Hyb3** and **Hyb2** have identical output distributions.

Hyb4. In this hybrid, when the Acceptor P_j is honest, the simulator $\mathcal{S}$ computes $s'_i(c)$ from the (pre-generated) data it previously sent to the corrupted party P_i. Then, on behalf of P_j, $\mathcal{S}$ receives $\Delta_{\mathsf{Sender}\to i}(c)$ from P_i and checks whether it matches, i.e., whether $\Delta_{\mathsf{Sender}\to i}(c) = s'_i(c)$. The only way the output distribution differs from **Hyb3** is if $\Delta_{\mathsf{Sender}\to i}(c) \neq s'_i(c)$ yet $\mathcal{S}$ still accepts—equivalently, the corrupted party submits a pair $(\alpha'_i, \Delta_{\mathsf{Sender}\to i}(c))$ that is inconsistent with the previously fixed value but nevertheless passes. For any single such submission, the acceptance event occurs with probability at most $1/|\mathbb{F}|$; by a union bound over at most t corrupted parties,

$$\varepsilon_1 \;\leq\; \frac{t}{|\mathbb{F}|} \;=\; \frac{t}{2^{\kappa}},$$

which is negligible in the security parameter κ. Hence the output distributions of **Hyb4** and **Hyb3** are statistically indistinguishable.

Hyb5. In this hybrid, during each revelation in which the Acceptor P_j is honest, the simulator $\mathcal{S}$ changes how it handles an honest party P_i's verification point. Rather than re-verifying the point on behalf of P_j as prescribed by the protocol, $\mathcal{S}$ simply treats P_i's submission as accepted upon receipt by P_j. This is sound because both Sender and P_i are honest, so P_i's verification shares are always valid and would be accepted anyway. Since this modification affects only the local checking procedure and not the values delivered, the joint output distribution is unchanged. Therefore, **Hyb5** and **Hyb4** have identical output distributions.

Hyb6. In this hybrid, the simulator $\mathcal{S}$ does not sample the entire polynomial $\Delta_{\mathsf{Sender}}(r)$ at setup. Instead, in each revelation with query vector $\boldsymbol{c}$ it produces only the evaluation $\Delta_{\mathsf{Sender}\to i}(c)$ for each corrupted party P_i. Let $\mathcal{W} = \{(\boldsymbol{c}_1, s'_i(c_1)), \ldots, (\boldsymbol{c}_k, s'_i(c_k))\}$ be the knowledge set accumulated so far. If $\boldsymbol{c} \in \mathrm{span}_{\mathbb{F}}\{\boldsymbol{c}_1, \ldots, \boldsymbol{c}_k\}$, say $\boldsymbol{c} = \sum_{j=1}^{k} a_j \boldsymbol{c}_j$, then $\mathcal{S}$ deterministically sets $\Delta_{\mathsf{Sender}\to i}(c) := \sum_{j=1}^{k} a_j\, s'_i(c_j)$. Otherwise (i.e., when $\boldsymbol{c}$ is linearly independent of $\{\boldsymbol{c}_j\}$), $\mathcal{S}$ generates $\Delta_{\mathsf{Sender}\to i}(c)$ exactly as in **Hyb5** (fresh and independent given the history). This modification only changes the timing/order of generation and does not affect the joint output distribution. Consequently, **Hyb6** and **Hyb5** have identical output distributions.

Hyb7. In this hybrid, for each revelation in which the Acceptor is corrupted and for every honest party P_j, the simulator $\mathcal{S}$ no longer computes the value $s'_j(c)$

at the challenge point c. Instead, $\mathcal{S}$ samples a value $\Delta_{\mathsf{Sender}\to j}(c) \xleftarrow{\$} \mathbb{F}$ and sends it to the corrupted Acceptor on behalf of P_j. In **Hyb6**, the value $\Delta_{\mathsf{Sender}\to j}(c)$ sent for an honest party is (from the acceptor's viewpoint) uniform over $\mathbb{F}$; hence sampling it directly preserves its distribution. Therefore, this modification only changes how the value is produced, not its law. Consequently, **Hyb7** and **Hyb6** have identical output distributions.

Hyb8. In this hybrid, for each revelation in which the Acceptor is corrupted, the simulator $\mathcal{S}$ does *not* compute the ideal functionality's output. Instead, using the honest parties' values $\{\Delta_{\mathsf{Sender}\to j}(c)\}$ together with the corrupted parties' polynomials $\{s'_i(r)\}$, $\mathcal{S}$ forms (on behalf of $\mathcal{F}_{\mathsf{AMC}}$) the authenticated evaluation vector $\boldsymbol{g}_{\mathsf{Sender}} = \big(g_{\mathsf{Sender}}(\alpha_1), \ldots, g_{\mathsf{Sender}}(\alpha_n)\big)$ and sends it to the corrupted Acceptor. Across at most T revelations (e.g., $T = n$), the adversary learns at most T evaluations of the Sender's polynomial $\Delta_{\mathsf{Sender}}(r)$, whose degree is m. Hence the corrupted view contains at most m evaluation constraints; since a degree-m polynomial is fixed by $m{+}1$ points, the value of $\Delta_{\mathsf{Sender}}(r)$ at any fresh point remains uniform over $\mathbb{F}$ from the adversary's perspective. Therefore, replacing the ideal output by the above constructed $\boldsymbol{g}_{\mathsf{Sender}}$ does not change the joint output distribution. Consequently, **Hyb8** and **Hyb7** have identical output distributions.

Hyb9. In this hybrid, for each revelation in which the Acceptor is honest and for every honest party P_i, the simulator $\mathcal{S}$ does not re-verify (on behalf of Acceptor) the verification value/point submitted by P_i (e.g., $\Delta_{\mathsf{Sender}\to i}(c)$). Instead, whenever Acceptor receives P_i's submission, $\mathcal{S}$ treats it as accepted. Since both Sender and P_i are honest, the submission is correct and would be accepted in **Hyb8** as well. Therefore, **Hyb9** and **Hyb8** have identical output distributions.

Note that **Hyb9** corresponds to the ideal-world execution, where the protocol $\Pi_{\mathsf{AcceptPoly}}$ achieves a statistical security guarantee for securely computing the functionality $\mathcal{F}_{\mathsf{AcceptPoly}}$.

When *Sender* is corrupted:

Simulator $\mathcal{S}$

1. For each honest $P_i \in \mathcal{P}$, the simulator $\mathcal{S}$, acting on behalf of P_i, receives from Sender the verification polynomial $\Delta_{\mathsf{Sender}\to i}(r)$.
2. $\mathcal{S}$ sets $\boldsymbol{s}_1 = \cdots = \boldsymbol{s}_m = \mathbf{0}$ and, on behalf of Sender, sends (Init, AcceptPoly, T, $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$) to $\mathcal{F}_{\mathsf{AcceptPoly}}$, where $\mathbf{0} \in \mathbb{F}^n$ denotes the all-zero vector.
3. $\mathcal{S}$ initializes a counter $\mathsf{count} := T$.
4. For each revelation, if $\mathsf{count} > 0$, $\mathcal{S}$ performs the following steps and then sets $\mathsf{count} \leftarrow \mathsf{count} - 1$.
   1) **Case:** Acceptor P_j **honest.**
      (a) $\mathcal{S}$ computes $\Delta_{\mathsf{Sender}\to j}(c) = \Delta_{\mathsf{Sender}\to j}(r)\big|_{r=c}$ from the polynomial received from Sender.
      (b) For each corrupted $P_i \in \mathcal{P}$, $\mathcal{S}$ waits to receive a purported verification value $\Delta_{\mathsf{Sender}\to i}(c)$ from P_i and faithfully emulates the interactions of $\mathcal{F}_{\mathsf{AMC}}$. In all cases, $\mathcal{S}$ checks whether the received

$\Delta_{\mathsf{Sender}\to i}(c)$ is consistent with the output that $\mathcal{F}_{\mathsf{AMC}}$ would produce. If so, $\mathcal{S}$ regards Acceptor P_j as having received a correct verification value.

(c) Upon collecting at least $2t{+}1$ correct verification values, $\mathcal{S}$ sends Proceed and $\Delta_{\mathsf{Sender}\to j}(r)$ to $\mathcal{F}_{\mathsf{AcceptPoly}}$ and allows $\mathcal{F}_{\mathsf{AcceptPoly}}$ to deliver its output to Acceptor P_j. If instead $\mathcal{S}$ receives $2t{+}1$ incorrect verification values, it sends Ignore to $\mathcal{F}_{\mathsf{AcceptPoly}}$.

2) **Case:** Acceptor P_j **corrupted.**

i. For each honest $P_i \in \mathcal{P}$, $\mathcal{S}$ evaluates $\Delta_{\mathsf{Sender}\to i}(c) = \Delta_{\mathsf{Sender}\to i}(r)\big|_{r=c}$ and sends this value to the corrupted Acceptor P_j on behalf of P_i.

ii. $\mathcal{S}$ emulates $\mathcal{F}_{\mathsf{AMC}}$ and sends $\big(\Delta_{\mathsf{Sender}\to 1}(c), \ldots, \Delta_{\mathsf{Sender}\to n}(c)\big)$ to the corrupted Acceptor P_j.

Fig. 22: Simulator for the $\mathcal{F}_{\mathsf{AcceptPoly}}$ when *Sender* is corrupted.

Hybrid arguments:

Hyb0. In this hybird, $\mathcal{S}$ behaves as in the real-world execution by using honest inputs and following the protocol correctly.

Hyb1. In this hybird, the process by which *Acceptor* P_j obtains $\Delta_{\mathsf{Sender}\to j}(r)$ is modified. Specifically, after receiving at least $2t+1$ correct verification points, $\mathcal{S}$ sends the message Proceed along with $\Delta_{\mathsf{Sender}\to j}(r)$ to $\mathcal{F}_{\mathsf{AcceptPoly}}$, allowing *Acceptor* P_j to receive the output directly from $\mathcal{F}_{\mathsf{AcceptPoly}}$. The primary difference between **Hyb0** and **Hyb1** lies in how *Acceptor* P_j acquires $\Delta_{\mathsf{Sender}\to j}(r)$: either from $\mathcal{S}$ or through $\mathcal{F}_{\mathsf{AcceptPoly}}$. However, this change does not impact the output distribution. Therefore, the distributions of **Hyb0** and **Hyb1** remain identical.

Note that **Hyb1** corresponds to the ideal-world execution, where the protocol $\Pi_{\mathsf{AcceptPoly}}$ achieves a statistical security guarantee for securely computing the functionality $\mathcal{F}_{\mathsf{AcceptPoly}}$.

C.2 Analysis of Communication Complexity

We compute the communication complexity of our protocol as follows. $\Pi_{\mathsf{AcceptPoly}}$ takes m vectors as inputs, and each vector is of size n. T' is an input parameter. All parties execute the Initialization Phase only once, while the Revelation Phase can be executed for at most T' times.

1. During the Initialization Phase:
   1) For each $i \in [n]$, *Sender* sends $\Delta_{\mathsf{Sender}\to i}(r)$ to P_i, resulting in a communication of $\mathcal{O}(mn\kappa)$ bits.
   2) Therefore, the total communication cost is $\mathcal{O}(mn\kappa)$ bits during the Initialization Phase.
2. During the Acceptance Phase:
   1) Each time, Each party P_i (for $i \in [n]$) sends $\Delta_{\mathsf{Sender}\to i}(c)$ to the *Acceptor*, which requires communication of $\mathcal{O}(n\kappa)$ bits.

2) Therefore, the total communication cost per revelation is $\mathcal{O}(n\kappa)$ bits. Since the Acceptance Phase can be executed for at most $T' = n$ times, the total communication cost is $\mathcal{O}(n^2\kappa)$ bits.

Therefore, $\Pi_{\mathsf{AcceptPoly}}$ requires communication of $\mathcal{O}(mn\kappa + n^2\kappa)$ bits in total.

D The Protocol of Privacy Reconstrution

D.1 Proposed Construction of Π_{PrivRec}

Protocol Π_{PrivRec}

Private Reconstruction Phase

Input: $n = 3t+1$ parties. N secrets $s_1, s_2, \ldots, s_N$, each shared using a distinct degree-t polynomial $p_1(x), p_2(x), \ldots, p_N(x)$.

Output: Reconstruction of all secrets $s_1, s_2, \ldots, s_N$ and their respective polynomials $p_1(x), p_2(x), \ldots, p_N(x)$.

Step 1: Random Sampling and Share Submission

1. The reconstructor R randomly selects $2t + 1$ parties from the total n parties.
2. Each selected party P_j submits its shares $[s_1]_j, [s_2]_j, \ldots, [s_N]_j$ to $\mathcal{R}$.

Step 2: Consistency Check via Interpolation

1. For each secret s_i, the reconstructor R performs the following:
   1) Using the submitted shares $[s_i]_j$ from the $2t + 1$ selected parties, attempt to interpolate a candidate polynomial $\hat{p}_i(x)$ of degree t using Lagrange interpolation: $\hat{p}_i(x) = \sum_{j=1}^{2t+1} [s_i]_j \cdot \ell_j(x)$, where $\ell_j(x)$ is the Lagrange basis polynomial defined as: $\ell_j(x) = \prod_{\substack{1 \le k \le 2t+1 \\ k \ne j}} \frac{x - x_k}{x_j - x_k}$.
2. Verify the consistency of $\hat{p}_i(x)$ by checking if it satisfies the received shares: $\hat{p}_i(x_j) \stackrel{?}{=} [s_i]_j, \quad$ for each $j \in [2t+1]$. If the check fails for any share, mark it as invalid and exclude it from the reconstruction process.

Step 3: Redundant Verification and Majority Decision

1. Randomly select another $2t + 1$ shares and repeat the interpolation and consistency check process to construct a new candidate polynomial $\tilde{p}_i(x)$.
2. Compare $\hat{p}_i(x)$ and $\tilde{p}_i(x)$. If they are consistent (i.e., $\hat{p}_i(x) = \tilde{p}_i(x)$), proceed. Otherwise, perform a majority decision:

$$p_i(x) = \arg \max_{p \in \{\hat{p}_i(x), \tilde{p}_i(x)\}} \text{Count of valid shares that fit the polynomial.}$$

Step 4: Global Interpolation and Secret Reconstruction

1. Using the selected polynomial $p_i(x)$, reconstruct the secret for each $i \in \{1, 2, \ldots, N\}$: $s_i = p_i(0)$.

Step 5: Error Detection and Correction

1. If inconsistencies persist or insufficient valid shares are available, request additional shares from the remaining $n - (2t + 1)$ parties and repeat Steps 2 to 4.

2. The protocol terminates when all secrets are successfully reconstructed or if the maximum number of retries is reached.

Fig. 23: The protocol for private reconstruction.

Lemma 5. *The protocol Π_{PrivRec} achieves t-security and correctly realizes the functionality $\mathcal{F}_{\mathsf{PrivRec}}$ in the given computational model.*

Proof. To establish the security of the PrivRec protocol, we introduce an ideal adversary $\mathcal{S}$ that interacts with both the environment $\mathcal{Z}$ and the ideal functionalities. The role of $\mathcal{S}$ is to simulate honest parties in the real world and concurrently execute the real-world adversary $\mathcal{A}$. For clarity and simplicity in our proof, $\mathcal{S}$ acts as a proxy for communication between honest parties and the ideal functionalities of sub-protocols.
In this simulation framework, all communications from $\mathcal{Z}$ are relayed by $\mathcal{S}$ to $\mathcal{A}$, and vice versa, ensuring accurate message flow representation. When an honest party needs to communicate with another, $\mathcal{S}$ informs $\mathcal{A}$ of the message delivery, which allows $\mathcal{A}$ to provide feedback on the message's arrival time. This interaction assists $\mathcal{S}$ in instructing the ideal functionalities to appropriately delay outputs within the ideal world. Specifically, for each output that should be delayed before being sent to an honest party, $\mathcal{S}$ maintains this delay by default until it authorizes the functionality to proceed with the transmission. Through hybrid arguments, we will demonstrate that the distribution of outputs in the ideal world is indistinguishable from those in the real world.
The construction of the ideal adversary $\mathcal{S}$ proceeds as follows: upon claiming a message has been delivered, $\mathcal{S}$ merely indicates to $\mathcal{A}$ that such delivery has taken place. It is important to note that $\mathcal{S}$ does not necessarily have access to the content of these messages, thereby preserving the confidentiality of the communications.
Furthermore, to enhance the rigor of our analysis, we employ a series of hybrid steps, where each step incrementally refines the simulation closer to the ideal execution scenario. Each hybrid step minimizes the distinguishability between consecutive stages, culminating in a comprehensive proof that $\mathcal{S}$'s behavior accurately models the real-world adversary $\mathcal{A}$ under ideal conditions. This structured approach substantiates the robustness of the PrivRec protocol's security claims.
Construction of the ideal adversary $\mathcal{S}$.
If R is corrupted:

Simulator $\mathcal{S}$

$\mathcal{S}$ receives the whole sharing $[s_1], \ldots, [s_N]$ from $\mathcal{F}_{\mathsf{PrivRec}}$. For each honest P_j, $\mathcal{S}$ sends the P_j's shares of $[s_1], \ldots, [s_N]$ to R on behalf of P_j.

Fig. 24: Simulator for the $\mathcal{F}_{\mathsf{PrivRec}}$.

Hybrid arguments:
Hyb0. In this hybrid, the simulator $\mathcal{S}$ receives the honest parties' inputs and executes the protocol faithfully. This coincides with the real-world execution.sing honest inputs and following the protocol correctly.

Hyb1. In this hybrid, $\mathcal{S}$ modifies the way it learns the shares of $s_1, \ldots, s_N$ for each honest party. Specifically:
- Instead of obtaining the shares directly from the honest parties, $\mathcal{S}$ retrieves them from the output of $\mathcal{F}_{\mathsf{PrivRec}}$.
Since $[s_1], \ldots, [s_N]$ form complete t-sharings, the shares of the honest parties are inherently included within the overall sharing. Consequently, this modification does not affect the output distribution.
Therefore, the output distributions of **Hyb1** and **Hyb0** remain identical.
Note that **Hyb1** corresponds to the ideal-world execution, where the protocol Π_{PrivRec} achieves a statistical security guarantee for securely computing the functionality $\mathcal{F}_{\mathsf{PrivRec}}$.
If R is honest:

Simulator $\mathcal{S}$

1. For each corrupted P_i, $\mathcal{S}$ receives his shares of $[s_1], \ldots, [s_N]$ from $\mathcal{F}_{\mathsf{PrivRec}}$.
2. For each corrupted P_i, $\mathcal{S}$ receives their messages $[s_1], \ldots, [s_N]$ sent to R. When the message arrives, $\mathcal{S}$ accepts P_i's shares if his share of s_j equals to $[s_j]$ for all $j \in [N]$. For each honest P_i, $\mathcal{S}$ regards that he accepts P_i's shares when the shares arrive.
3. After accepting $2t+1$ parties' shares, $\mathcal{S}$ allows $\mathcal{F}_{\mathsf{PrivRec}}$ to send the output to R.

Fig. 25: Simulator for the $\mathcal{F}_{\mathsf{PrivRec}}$.

Hybrid arguments:
Hyb0. In this hybrid, the simulator $\mathcal{S}$ receives the honest parties' inputs and executes the protocol faithfully. This coincides with the real-world execution.sing honest inputs and following the protocol correctly.
Hyb1. In this hybird, R obtains its output directly from $\mathcal{F}_{\mathsf{PrivRec}}$ instead of computing the output independently.
Due to the error-correction properties of Reed-Solomon codes, R will still correctly reconstruct the secrets $s_1, \ldots, s_N$. Since R receives at least $t+1$ shares from honest parties, these shares uniquely determine the entire sharing. Thus, R has all the necessary information to compute the entire sharing on its own.
This demonstrates that R effectively obtains the correct output after receiving $2t+1$ valid shares. Consequently, this change does not affect the output distribution.
Therefore, the output distributions of **Hyb1** and **Hyb0** remain identical.
Note that **Hyb1** corresponds to the ideal-world execution, where the protocol Π_{PrivRec} achieves a statistical security guarantee for securely computing the functionality $\mathcal{F}_{\mathsf{PrivRec}}$.
The protocol Π_{PrivRec} requires $\mathcal{O}(Nn\kappa)$ bits communication to send n parties' shares of the N complete t-sharing to R.

E Asynchronous Complete Secret Sharing(ACSS)

E.1 Proposed Construction of Π_{ACSS}

The proposed construction of Π_{ACSS} is described as follows.

Simulator $\mathcal{S}$

The execution flow for all parties follows the sequence: Π_{Sh}, Π_{Ver}, $\Pi_{\mathsf{Accept}_{\mathsf{Poly}}}$, and Π_{Comp}.

Fig. 26: Part-(1/4) of the simulator for the $\mathcal{F}_{\mathsf{ACSS}}$ when D is honest.

E.2 The Proof of Π_{ACSS}

Proof. To establish the security of the ACSS protocol, we introduce an ideal adversary $\mathcal{S}$ that interacts with both the environment $\mathcal{Z}$ and the ideal functionalities. The role of $\mathcal{S}$ is to simulate honest parties in the real world and concurrently execute the real-world adversary $\mathcal{A}$. For clarity and simplicity in our proof, $\mathcal{S}$ acts as a proxy for communication between honest parties and the ideal functionalities of sub-protocols.
In this simulation framework, all communications from $\mathcal{Z}$ are relayed by $\mathcal{S}$ to $\mathcal{A}$, and vice versa, ensuring accurate message flow representation. When an honest party needs to communicate with another, $\mathcal{S}$ informs $\mathcal{A}$ of the message delivery, which allows $\mathcal{A}$ to provide feedback on the message's arrival time. This interaction assists $\mathcal{S}$ in instructing the ideal functionalities to appropriately delay outputs within the ideal world. Specifically, for each output that should be delayed before being sent to an honest party, $\mathcal{S}$ maintains this delay by default until it authorizes the functionality to proceed with the transmission. Through hybrid arguments, we will demonstrate that the distribution of outputs in the ideal world is indistinguishable from those in the real world.
The construction of the ideal adversary $\mathcal{S}$ proceeds as follows: upon claiming a message has been delivered, $\mathcal{S}$ merely indicates to $\mathcal{A}$ that such delivery has taken place. It is important to note that $\mathcal{S}$ does not necessarily have access to the content of these messages, thereby preserving the confidentiality of the communications.
Furthermore, to enhance the rigor of our analysis, we employ a series of hybrid steps, where each step incrementally refines the simulation closer to the ideal execution scenario. Each hybrid step minimizes the distinguishability between consecutive stages, culminating in a comprehensive proof that $\mathcal{S}$'s behavior accurately models the real-world adversary $\mathcal{A}$ under ideal conditions. This structured approach substantiates the robustness of the ACSS protocol's security claims.
Construction of the ideal adversary $\mathcal{S}$.
When D is honest:

Simulator $\mathcal{S}$

Sharing Phase

1. For each corrupted P_j, $\mathcal{S}$ receives $\{q_1(\alpha_j), \ldots, q_N(\alpha_j)\}$ from $\mathcal{F}_{\mathsf{ACSS}}$.
2. Define $\mathsf{idx} = (\ell - 1)(t+1) + 1$.
   1) If $(t+1)$ is divisible by N, for $\ell \in [m']$ and corrupted P_j, $\mathcal{S}$ randomly selects a degree-$2t$ (column) polynomial $g_{\ell,j}(y)$ such that $g_{\ell,j}(\alpha_{-i}) = q_{\mathsf{idx}+i}(\alpha_j)$ for $i \in [0,t]$.
   2) Otherwise, for each $\ell \in [m'-1]$ and corrupted P_j, $\mathcal{S}$ randomly selects a degree-$2t$ (column) polynomial $g_{\ell,j}(y)$ such that $g_{\ell,j}(\alpha_{-i}) = q_{\mathsf{idx}+i}(\alpha_j)$ for $i \in [0,t]$; for $\ell = m'$ and corrupted P_j, $\mathcal{S}$ randomly samples $\lceil \frac{N}{t+1} \rceil (t+1) - N$ degree-t random polynomials $(q_{N+1}(x), \ldots, q_{m(t+1)}(x))$, s.t. $g_{\ell,j}(\alpha_{-i}) = q_{\mathsf{idx}+i}(\alpha_j)$ for $i \in [0,t]$.
   3) For each $\ell \in [m'+1, m]$ and corrupted P_j, $\mathcal{S}$ randomly selects a degree-$2t$ (column) polynomial $g_{\ell,j}(y)$.
3. For each $\ell \in [m]$ and corrupted P_i, $\mathcal{S}$ randomly selects a degree-t (row) polynomial $f_{\ell,i}(y)$ such that $f_{\ell,i}(\alpha_j) = g_{\ell,j}(\alpha_i)$ for each corrupted P_j on behalf of D.
4. $\mathcal{S}$ sends the polynomials $\{g_{\ell,j}(y)\}_{\ell \in [m]}$ and $\{f_{\ell,j}(x)\}_{\ell \in [m]}$ to each corrupted P_j.
5. $\mathcal{S}$ initializes a set $\mathcal{M}$ to $\varnothing$.
   For each $P_i \in \mathcal{P}$:
   1) If P_i is honest, when P_i receives his column polynomials, $\mathcal{S}$ broadcasts Yes_i on behalf of P_i. Then $\mathcal{S}$ delivers an initialization request to $\mathcal{F}_{\mathsf{AMC}}(P_i, D)$ on behalf of P_i and emulates $\mathcal{F}_{\mathsf{AMC}}(P_i, D)$ to deliver the output to D. When D receives the output and Yes_i, $\mathcal{S}$ includes P_i into $\mathcal{M}$. $\mathcal{S}$ delivers an initialization request to $\mathcal{F}_{\mathsf{AcceptPoly}}(P_i)$ on behalf of P_i.
   2) If P_i is corrupted, $\mathcal{S}$ emulates $\mathcal{F}_{\mathsf{AMC}}(P_i, D)$ to receive $(\mathsf{Init}, \mathsf{AMC}, T, (\boldsymbol{g_{1,i}}, \ldots, \boldsymbol{g_{m,i}}))$ from P_i. If Yes_i is received from P_i, $\mathcal{S}$ checks if, for each $\ell \in [m]$, $\boldsymbol{g_{\ell,i}} = (g_{\ell,i}(\alpha_1), \ldots, g_{\ell,i}(\alpha_n))$. If so, when D receives the output and Yes_i, $\mathcal{S}$ emulates $\mathcal{F}_{\mathsf{AMC}}(P_i, D)$ to send an output $(P_i, \mathsf{AMC}, (\boldsymbol{g_{1,i}}, \ldots, \boldsymbol{g_{m,i}}))$ to D and includes P_i into $\mathcal{M}$.
6. When $|\mathcal{M}| = 2t+1$, $\mathcal{S}$ broadcasts $\mathcal{M}$ on behalf of D.
7. For each honest P_j, $\mathcal{S}$ waits until P_j receives $\mathcal{M}$ and $\{\mathsf{OK}_i\}_{P_i \in \mathcal{M}}$ and then begins the simulation of P_j in the next phase.

Fig. 27: Part-(1/4) of the simulator for the $\mathcal{F}_{\mathsf{ACSS}}$ when D is honest.

Simulator $\mathcal{S}$

Verification Phase

1. Initialize a registry $\mathcal{R} := \{(0, \mathbf{0})\}$, where $\mathbf{0} \in \mathbb{F}[X,Y]$ denotes the zero bivariate polynomial ($\mathbf{0}(x,y) = 0$ for all $(x,y) \in \mathbb{F}^2$). Every future entry in $\mathcal{R}$ has the form $(r, F) \in \mathbb{F} \times \mathbb{F}[X,Y]$.
2. For each honest $P_i \in \mathcal{P}$:

1) When P_i receives the column polynomials from D, the simulator $\mathcal{S}$ broadcasts, on behalf of P_i, a random challenge $r_i \xleftarrow{\$} \mathbb{F}$.
2) For every honest party P_j and every $P_h \in \mathcal{M}$, once P_j receives r_i, the simulator sends on behalf of P_j the request $(\mathsf{Request}, \mathsf{AMC}, P_i, (r_i, r_i^2, \ldots, r_i^m))$ to $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$.
3) For each honest $P_h \in \mathcal{M}$, $\mathcal{S}$ emulates $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$ to deliver its output to P_i. For each corrupted $P_h \in \mathcal{M}$, $\mathcal{S}$ faithfully emulates $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$.
4) After P_i has received the outputs from $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$ for all $P_h \in \mathcal{M}$, the simulator regards P_i's family of column polynomials $\{g_{\ell,i}(y)\}_{\ell \in [m]}$ as accepted and proceeds to the next phase for P_i.

3. For each *corrupted* $P_i \in \mathcal{P}$:
1) For every honest P_j and every $P_h \in \mathcal{M}$, when $\mathcal{S}$ (on behalf of P_j) receives r_i, it sends $(\mathsf{Request}, \mathsf{AMC}, P_i, (r_i, r_i^2, \ldots, r_i^m))$ to $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$ on behalf of P_j.
2) For each corrupted $P_h \in \mathcal{M}$, $\mathcal{S}$ faithfully emulates $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$.
3) For each honest $P_h \in \mathcal{M}$, proceed as follows.
(i) If $(r_i, F) \in \mathcal{R}$ for some $F \in \mathbb{F}[X, Y]$, then $\mathcal{S}$ emulates $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$ to send to P_i the vector $\boldsymbol{g}_h = \big(g_h(\alpha_1), \ldots, g_h(\alpha_n)\big)$ with $g_h(y) = F(\alpha_h, y)$.
(ii) Otherwise, $\mathcal{S}$ samples a random bivariate polynomial $F \in \mathbb{F}[X, Y]$ of degree at most $(t, 2t)$, subject to the boundary conditions, for each corrupted P_j, $F(\alpha_j, y) = \sum_{\ell=1}^{m} r_i^\ell \, g_{\ell,j}(y)$, $F(x, \alpha_j) = \sum_{\ell=1}^{m} r_i^\ell \, f_{\ell,j}(x)$. It then emulates $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$ to send to P_i the corresponding $\boldsymbol{g}_h = (g_h(\alpha_1), \ldots, g_h(\alpha_n))$ with $g_h(y) = F(\alpha_h, y)$, and finally records the association by inserting (r_i, F) into $\mathcal{R}$.

Fig. 28: Part-(2/4) of the simulator for the $\mathcal{F}_{\mathsf{ACSS}}$ when D is honest.

Simulator $\mathcal{S}$

AcceptPoly Phase

1. For each honest $P_i \in \mathcal{P}/\mathcal{M}$:
1) $\mathcal{S}$ sends $(\mathsf{Request}, \mathsf{coin}, \mathcal{P})$ to $\mathcal{F}_{\mathsf{coin}}$ on behalf of each honest party P_j, emulates $\mathcal{F}_{\mathsf{coin}}$, and delivers its output to all parties.
2) For each honest party P_j and $P_h \in \mathcal{M}$, when P_j receives c_i, $\mathcal{S}$ sends $(\mathsf{Request}, \mathsf{AcceptPoly}, P_i, (c_i^1, c_i^2, \ldots, c_i^m))$ to $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$ on behalf of P_j.
3) For each honest $P_h \in \mathcal{M}$, $\mathcal{S}$ emulates $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$ to deliver an output to P_i. For each corrupted $P_h \in \mathcal{M}$, $\mathcal{S}$ faithfully emulates $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$.
4) When P_i receives the output from $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$ for all parties $P_h \in \mathcal{M}$, the simulator $\mathcal{S}$ considers that P_i accepts the polynomial $\Delta_{h \to i}(r)$ and begins the simulation of P_i in the next phase.

2. For each corrupted $P_i \in \mathcal{P}/\mathcal{M}$:

1) For each honest P_j and $P_h \in \mathcal{M}$, if $\mathcal{S}$ receives c_i on behalf of P_j, $\mathcal{S}$ sends $(\mathsf{Request}, \mathsf{AcceptShare}, P_i, (c_i^1, c_i^2, \ldots, c_i^m))$ to $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$ on behalf of P_j.
2) For each corrupted $P_h \in \mathcal{M}$, $\mathcal{S}$ faithfully emulates $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$.
3) For each honest $P_h \in \mathcal{M}$:
   i. $\mathcal{S}$ computes $\{g_{\ell,h}(\alpha_k)\}_{k\in[n],\ell\in[m]}$ based on the $\{F_\ell(x,y)\}_{\ell\in[m]}$ generated in the Verification phase.
   ii. $\mathcal{S}$ computes $\Delta_{\mathsf{h}\to i}(r) = \sum_{\ell=1}^{m} r^\ell g_{\ell,h}(\alpha_i)$ according to the protocol.
   iii. $\mathcal{S}$ sends $\Delta_{\mathsf{h}\to i}(r)$ to P_i on behalf of $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$.

Fig. 29: Part-(3/4) of the simulator for the $\mathcal{F}_{\mathsf{ACSS}}$ when D is honest.

Simulator $\mathcal{S}$

Completion Phase

Preparing:

1. For each honest party $P_i \in \mathcal{H}$, upon receiving $\{\Delta_{h\to i}(r)\}_{P_h\in\mathcal{M}}$ from $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$, party P_i samples m pairwise distinct points $r_1, \ldots, r_m \xleftarrow{\$} \mathbb{F}$. For each corrupted $P_h \in \mathcal{M}$, it forms the evaluation vector $\boldsymbol{y}_{h,i} = \big(\Delta_{h\to i}(r_1), \ldots, \Delta_{h\to i}(r_m)\big)^\top$ and the Vandermonde matrix $V \in \mathbb{F}^{m\times m}$ with entries $V_{\ell,j} := r_\ell^j$ for $\ell, j \in [m]$. Since the r_ℓ's are pairwise distinct, V is invertible; solving $V \cdot \boldsymbol{g}_{h\to i} = \boldsymbol{y}_{h,i}$ yields the coefficient vector $\boldsymbol{g}_{h\to i} = \big(g_{1,h}(\alpha_i), \ldots, g_{m,h}(\alpha_i)\big)^\top$, i.e., the entire set $\{g_{\ell,h}(\alpha_i)\}_{\ell\in[m],\, P_h\in\mathcal{M}}$.

Reconstructing row polynomials:

2. For each $P_v \in \mathcal{P}$, $\mathcal{S}$ does the following things:
   1) If P_v is honest:
      i. For each $P_h \in \mathcal{M}$ sends $\{g_{\ell,h}(\alpha_v)\}_{\ell\in[m]}$ to P_v:
         ⋆ If $P_h \in \mathcal{M}$ is honest, when P_v receives a share, $\mathcal{S}$ accepts the shares on behalf of P_v.
         ⋆ If $P_h \in \mathcal{M}$ is corrupted, when P_v receives a share, $\mathcal{S}$ checks to see if it is the same as the share received in the previous phase.
      ii. Upon P_v receiving $\{g_{\ell,h}(\alpha_i)\}_{\ell\in[m]}$ from $t+1$ different $P_h \in \mathcal{M}$, $\mathcal{S}$ computes $\{F_\ell(x,\alpha_v) = f_{\ell,v}(x)\}_{\ell\in[m]}$ on behalf of P_v.
   2) If P_v is corrupted:
      i. For each honest $P_h \in \mathcal{M}$, $\mathcal{S}$ sends $\{f_{\ell,v}(\alpha_h)\}_{\ell\in[m]}$ to P_v.

Reconstructing column polynomials:

3. For each $P_w \in \mathcal{P}$, $\mathcal{S}$ does the following things:
   1) **Case P_w honest.**
      i. For every corrupted $P_j \in \mathcal{C}$, $\mathcal{S}$, on behalf of P_w, sends the evaluation set $\{\, g_{\ell,j}(\alpha_w)\,\}_{\ell\in[m]}$ to P_j.
      ii. Let $P_{o'}$ denote any party that has accepted *row* polynomials and P_o any party that has accepted both *column* and *row* polynomials. Then $P_{o'}$ sends $\{\, \tilde{f}_{\ell,o'}(\alpha_w)\,\}_{\ell\in[m]}$ and P_o sends $\{\, \tilde{f}_{\ell,o}(\alpha_w)\,\}_{\ell\in[m]}$ to P_w.

- If $P_{o'}$ (resp. P_o) is honest, $\mathcal{S}$ accepts the received shares on behalf of P_w.
- If $P_{o'}$ (resp. P_o) is corrupted, $\mathcal{S}$ checks, for each $\ell \in [m]$, whether $\tilde{f}_{\ell,o'}(\alpha_w) = f_{\ell,o'}(\alpha_w)$ (resp. $\tilde{f}_{\ell,o}(\alpha_w) = f_{\ell,o}(\alpha_w)$) against the reference values computed from prior messages.

Collect all correct shares in the per-index sets $\mathcal{W}_w = \{\mathcal{W}_{1,w}, \ldots, \mathcal{W}_{m,w}\}$. As soon as $|\mathcal{W}_{\ell,w}| \geq 2t{+}1$ holds for every $\ell \in [m]$, $\mathcal{S}$ broadcasts $\mathsf{Yes'}_w$ on behalf of P_w.

iii. Upon receiving $\mathsf{Yes'}_j$ from at least $2t{+}1$ distinct parties P_j, $\mathcal{S}$ sends $(\mathsf{Request}, \mathsf{coin}, \mathcal{P})$ to $\mathcal{F}_{\mathsf{coin}}$ on behalf of P_w, emulates $\mathcal{F}_{\mathsf{coin}}$, and delivers its output to all parties.

iv. Upon receiving $d \in \mathbb{F}$ from $\mathcal{F}_{\mathsf{coin}}$, $\mathcal{S}$ computes

$$f_w(x) \;=\; \sum_{\ell=1}^{m} d^{\ell} f_{\ell,w}(x) \quad \text{with } \deg f_w = t,$$

and broadcasts $f_w(x)$ on behalf of P_w. If P_w has also accepted the column polynomials, compute

$$g_w(y) \;=\; \sum_{\ell=1}^{m} d^{\ell} g_{\ell,w}(y) \quad \text{with } \deg g_w = 2t,$$

and broadcast $g_w(y)$ as well.

v. Upon receiving, from each party P_o that accepted both row *and* column polynomials, the pair $(\tilde{g}_o(y), \tilde{f}_o(x))$ together with the evaluations $\{\, \tilde{f}_{\ell,o}(\alpha_w) \,\}_{\ell \in [m]}$, and from each party $P_{o'}$ that accepted only row polynomials the polynomial $\tilde{f}_{o'}(x)$ together with $\{\, \tilde{f}_{\ell,o'}(\alpha_w) \,\}_{\ell \in [m]}$, $\mathcal{S}$ performs the following checks:
   - Verify $\deg \tilde{g}_o = 2t$, $\deg \tilde{f}_o = t$, and $\deg \tilde{f}_{o'} = t$.
   - If $P_{o'}$ (resp. P_o) is honest, accept the received polynomials on behalf of P_w.
   - If $P_{o'}$ is corrupted, recompute $f_{o'}(x)$ from prior honest messages and check $\tilde{f}_{o'}(x) = f_{o'}(x)$. If P_o is corrupted, similarly recompute $(g_o(y), f_o(x))$ and check equality.

   If all checks pass for a corrupted P_o, set (viewing $f_o(\alpha_o)$ as a constant in y)

$$V(\alpha_o, y) \;:=\; g_o(y) \;+\; f_o(\alpha_o),$$

   and broadcast OK_{V_o} on behalf of P_w.

vi. Upon receiving OK_{V_o} from at least $2t{+}1$ distinct parties, $\mathcal{S}$ accepts $V(\alpha_o, y)$ on behalf of P_w.

vii. If all of the above conditions are satisfied, $\mathcal{S}$ broadcasts Done_w on behalf of P_w.

viii. Upon receiving Done_j from at least $2t{+}1$ distinct parties P_j, the simulator $\mathcal{S}$ authorizes $\mathcal{F}_{\mathsf{ACSS}}$ to deliver the output to P_w.

2) **Case P_w corrupted.**
   i. For each *honest* party $P_{o'}$ that has accepted row polynomials and each *honest* party P_o that has accepted both column and row polynomials, $\mathcal{S}$ sends, on their behalf, the evaluation sets $\{ f_{\ell,o'}(\alpha_w) \}_{\ell\in[m]}$ and $\{ f_{\ell,o}(\alpha_w) \}_{\ell\in[m]}$ respectively to P_w.
   ii. For every honest P_j, $\mathcal{S}$ broadcasts $\mathsf{Yes'}_w$ on behalf of P_j and sends $(\mathsf{Request}, \mathsf{coin}, \mathcal{P})$ to $\mathcal{F}_{\mathsf{Coin}}$ on that party's behalf.
   iii. $\mathcal{S}$ emulates $\mathcal{F}_{\mathsf{coin}}$ and delivers its output to all parties.
   iv. For each honest P_o, $\mathcal{S}$ computes $f_o(x)$ and $g_o(y)$ and broadcasts them on behalf of P_o. For each honest $P_{o'}$, $\mathcal{S}$ computes $f_{o'}(x)$ and broadcasts it on behalf of $P_{o'}$.
   v. For every honest P_j, $\mathcal{S}$ broadcasts OK_{V_o} and Done_j on that party's behalf.

Terminating:
For each $P_k \in \mathcal{P}$, $\mathcal{S}$ does the following things: If P_k is honest, $\mathcal{S}$ terminates on behalf of P_k

Fig. 30: Part-(4/4) of the simulator for the $\mathcal{F}_{\mathsf{ACSS}}$ when D is honest.

Hybrid arguments:

Hyb0. In this hybird, $\mathcal{S}$ behaves as in the real-world execution by using honest inputs and following the protocol correctly.

Hyb1. In this hybrid, during the *Sharing Phase* the simulator $\mathcal{S}$ first samples the corrupted parties' column and row polynomials and only subsequently samples the bivariate polynomials based on the dealer's input and those corrupted polynomials. This modification merely reorders the generation of the corrupted parties' polynomials and the bivariate polynomials, and hence does not affect the joint output distribution. Consequently, **Hyb1** and **Hyb0** have identical output distributions.

Hyb2. In this hybrid, the simulator $\mathcal{S}$ initializes a cache $\mathcal{R} := \{(0, \mathbf{0})\}$ that records pairs (r_i, F_i), where $F_i(x, y) := \sum_{k=1}^{m} r_i^k F_k(x, y)$ is the bivariate polynomial determined by the *Sharing Phase* polynomials $\{F_k(x, y)\}_{k=1}^{m}$ and the challenge $r_i \in \mathbb{F}$, and $\mathbf{0}$ is the zero polynomial. Whenever $\mathcal{S}$ needs a linear combination of honest parties' *column* polynomials, it instead forms the corresponding linear combination of the pre-generated bivariate polynomials $\{F_k(x, y)\}$ and then takes the required column (evaluation in y) as the result. Likewise, whenever a linear combination in the family $\{F_i\}$ is required and some r_i has already been recorded, $\mathcal{S}$ reuses the cached $(r_i, F_i) \in \mathcal{R}$ rather than recomputing it. This is merely memoization of deterministically defined values and does not affect the joint output distribution. Consequently, **Hyb2** and **Hyb1** have identical output distributions.

Hyb3. In this hybrid, during the *Verification Phase* the simulator $\mathcal{S}$ does not freshly (re)compute the ideal output of $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$ for a corrupted P_i when P_h is honest. Instead, it proceeds as follows. If the challenge r_i already appears in the cache $\mathcal{R}$ with an entry (r_i, F_i), then $\mathcal{S}$ uses the cached bivariate polynomial $F_i(x, y)$ to produce the corresponding output of $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$. Otherwise, $\mathcal{S}$ samples a fresh random bivariate polynomial $F_i \in \mathbb{F}[X, Y]$ of degree at most

$(t, 2t)$ that is consistent with the corrupted parties' row/column polynomials (i.e., it matches their prescribed univariate marginals), inserts (r_i, F_i) into $\mathcal{R}$, and then computes the functionality's output using F_i. Over the course of the phase there are at most $n < T+T'$ revelation requests per functionality instance $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$. Whenever a previously unseen r_i occurs, the distribution of the sampled F_i—uniform over the affine space of degree-$(t, 2t)$ bivariate polynomials consistent with the corrupted constraints—matches exactly the distribution used in **Hyb2**. Consequently, reusing cached (r_i, F_i) pairs or sampling them on first use only changes the timing of generation and not the law of the outputs, and therefore **Hyb3** and **Hyb2** have identical output distributions.

Hyb4. In this hybrid, during the *AcceptPoly* phase, whenever P_i is honest the simulator $\mathcal{S}$ skips re-verification and treats P_i as having received the correct polynomial as soon as it has obtained the outputs of $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$ for all $P_h \in \mathcal{M}$. This modification changes only the local checking procedure and not any values delivered to the environment or the adversary; hence the joint output distribution is unaffected. Therefore, **Hyb4** and **Hyb3** have identical output distributions.

Hyb5. In this hybrid, during the *AcceptPoly* phase and for every corrupted party P_i, the simulator $\mathcal{S}$ does not invoke $\mathcal{F}_{\mathsf{AcceptPoly}}$ to compute fresh outputs. Instead, whenever P_h is honest, $\mathcal{S}$ uses the bivariate polynomials $\{F_\ell(x, y)\}_{\ell\in[m]}$ prepared earlier (in the *Verification Phase*) to derive the honest column evaluations $\{\, g_{\ell,h}(\alpha_k)\,\}_{k\in[n],\,\ell\in[m]}$ with $g_{\ell,h}(y) = F_\ell(\alpha_h, y)$. It then forms the prescribed verification polynomial for P_i, $\Delta_{h\to i}(r) := \sum_{\ell=1}^{m} r^\ell\, g_{\ell,h}(\alpha_i)$, and sends $\Delta_{h\to i}(r)$ to P_i on behalf of $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$. This modification only changes the mechanism by which the output is produced: the $\{F_\ell\}$ are already fixed from the sharing/verification steps and are consistent with the corrupted row/column polynomials, so the induced distribution of $\{g_{\ell,h}(\alpha_k)\}$ and hence of $\Delta_{h\to i}(r)$ is identical to that of the ideal functionality. Therefore, the joint output distribution is unchanged, and **Hyb5** and **Hyb4** have identical output distributions.

Hyb6. In this hybrid, during the *Preparing* phase and for every honest party P_i, the simulator $\mathcal{S}$ does not follow the protocol to recompute polynomial coefficients from scratch. Instead, using the bivariate polynomials $\{F_\ell(x, y)\}_{\ell\in[m]}$ (which were fixed to be consistent with the corrupted parties' row/column polynomials), it directly obtains the relevant univariate polynomials via restriction, $f_{\ell,i}(x) := F_\ell(x, \alpha_i)$ and $g_{\ell,i}(y) := F_\ell(\alpha_i, y)$, and hence reads off their coefficients. This change only reorders the computation and does not affect the induced distribution of any values seen by the environment or the adversary. Consequently, **Hyb6** and **Hyb5** have identical output distributions.

Hyb7. In this hybrid, during the *Reconstructing Row Polynomials* phase, for every honest party P_v and any honest helper P_h, the simulator $\mathcal{S}$ does not reverify the correctness of the shares on behalf of P_v. Instead, as soon as P_v receives the shares from P_h, $\mathcal{S}$ accepts them on P_v's behalf. Since both the dealer D and P_h are honest, the shares sent by P_h are always correct, so P_v would accept them under the original protocol as well. Consequently, **Hyb7** and **Hyb6** have identical output distributions.

Hyb8. In this hybrid, during the *Reconstructing Row Polynomials* phase, for each honest P_v and each corrupted $P_h \in \mathcal{M}$, the simulator $\mathcal{S}$ does not run the protocol's verification of P_h's shares $(\alpha'_h, \{\tilde{g}_{\ell,h}(\alpha_v)\}_{\ell\in[m]})$. Instead, $\mathcal{S}$ checks whether these shares are consistent with the values it fixed in the *Sharing Phase*. The output distribution can differ only if a corrupted P_h submits $(\alpha'_h, \{\tilde{g}_{\ell,h}(\alpha_v)\}_{\ell\in[m]}) \neq (\alpha_h, \{g_{\ell,h}(\alpha_v)\}_{\ell\in[m]})$ that nevertheless passes the ensuing consistency requirements $\{F_\ell(x, \alpha_v) = f_{\ell,v}(x)\}_{\ell\in[m]}$. For a fixed pair (v, h) this happens with probability at most $\frac{1}{|\mathbb{F}|} = 2^{-\kappa}$. By a union bound over at most t corrupted helpers and at most $2t+1$ honest recipients, the overall failure probability is at most

$$\frac{t(2t+1)}{|\mathbb{F}|} \leq \frac{n^2}{|\mathbb{F}|},$$

which is negligible in the security parameter κ. Therefore, the output distributions of **Hyb8** and **Hyb7** are statistically close.

Hyb9. In this hybrid, during the *Reconstructing Column Polynomials* phase, for each honest party P_w the simulator $\mathcal{S}$ does not re-verify the shares sent by honest providers $P_{o'}$ (row-only) and P_o (row+column). Instead, as soon as P_w receives the shares $\{ f_{\ell,o'}(\alpha_w) \}_{\ell\in[m]}$ and $\{ f_{\ell,o}(\alpha_w) \}_{\ell\in[m]}$, $\mathcal{S}$ treats them as accepted on P_w's behalf. Since the dealer D, P_w, $P_{o'}$, and P_o are honest, these shares are always correct and would be accepted under the original protocol as well. Consequently, **Hyb9** and **Hyb8** have identical output distributions.

Hyb10. In this hybrid, during the *Reconstructing Column Polynomials* phase and for each honest party P_w, the simulator $\mathcal{S}$ does not run the protocol's full verification against shares sent by *corrupted* providers $P_{o'}$ (row-only) and P_o (row+column). Instead, for every $\ell \in [m]$, $\mathcal{S}$ checks the received evaluations against the reference values determined from prior honest messages:

$$\tilde{f}_{\ell,o'}(\alpha_w) \stackrel{?}{=} f_{\ell,o'}(\alpha_w) \qquad \text{and} \qquad \tilde{f}_{\ell,o}(\alpha_w) \stackrel{?}{=} f_{\ell,o}(\alpha_w).$$

Thus, the output distribution can change only if a corrupted provider supplies shares that *differ* from the previously fixed values but nevertheless, after the public coin $d \xleftarrow{\$} \mathbb{F}$ is sampled from $\mathcal{F}_{\mathsf{coin}}$, yield recombined polynomials $(\tilde{g}_o(y), \tilde{f}_o(x))$ and $\tilde{f}_{o'}(x)$ satisfying simultaneously:

(i) degree constraints $\deg \tilde{g}_o = 2t$, $\deg \tilde{f}_o = t$, and $\deg \tilde{f}_{o'} = t$; and
(ii) equality checks $\tilde{f}_{o'}(x) = f_{o'}(x)$, $\tilde{f}_o(x) = f_o(x)$, $\tilde{g}_o(y) = g_o(y)$.

Answer (via Schwartz–Zippel). In this hybrid, a corrupted party may try to submit (per–index) shares $\{\tilde{f}_{\ell,o'}(\alpha_w)\}_{\ell\in[m]}$ or $\{\tilde{f}_{\ell,o}(\alpha_w)\}_{\ell\in[m]}$ that (i) are *inconsistent* with the honest reference rows $\{f_{\ell,o'}(\cdot)\}, \{f_{\ell,o}(\cdot)\}$, yet (ii) still pass the later recombination checks after the public coin $d \xleftarrow{\$} \mathbb{F}$ is sampled. Writing the recombined rows/columns as

$$f_{o'}(x) = \sum_{\ell=1}^{m} d^\ell f_{\ell,o'}(x), \qquad f_o(x) = \sum_{\ell=1}^{m} d^\ell f_{\ell,o}(x), \qquad g_o(y) = \sum_{\ell=1}^{m} d^\ell g_{\ell,o}(y),$$

a forging attempt fixes polynomials $\{\tilde{f}_{\ell,\cdot}\}$ in advance (before d is known). Consider the *difference polynomials in* d

$$H^x_{o'}(d;x) := \sum_{\ell=1}^{m} d^\ell \big(\tilde{f}_{\ell,o'}(x) - f_{\ell,o'}(x)\big),$$

$$H^x_o(d;x) := \sum_{\ell=1}^{m} d^\ell \big(\tilde{f}_{\ell,o}(x) - f_{\ell,o}(x)\big), \quad H^y_o(d;y) := \sum_{\ell=1}^{m} d^\ell \big(\tilde{g}_{\ell,o}(y) - g_{\ell,o}(y)\big).$$

If any share family is tampered, at least one of the above is a nonzero polynomial in d of degree at most $m-1$. Since d is uniform in $\mathbb{F}$ and fixed *after* the adversary's choice, Schwartz–Zippel yields, for a single equality test,

$$\Pr\big[\tilde{f}_{o'}(x) = f_{o'}(x)\big] \;=\; \Pr\big[H^x_{o'}(d;x) \equiv 0\big] \;\le\; \frac{m-1}{|\mathbb{F}|}.$$

Analogously,

$$\Pr\big[\tilde{f}_o(x) = f_o(x)\big] \le \frac{m-1}{|\mathbb{F}|}, \qquad \Pr\big[\tilde{g}_o(y) = g_o(y)\big] \le \frac{m-1}{|\mathbb{F}|}.$$

Hence, by a union bound, the probability that a corrupted party forges shares that survive *all* post-coin checks (degrees $2t, t, t$ are trivial to satisfy syntactically) is bounded by

$$\Pr[\textsf{forge passes}] \;\le\; \frac{3(m-1)}{|\mathbb{F}|} \;\le\; \frac{3m}{|\mathbb{F}|} \;=\; \frac{3m}{2^\kappa},$$

which is negligible in the security parameter κ. If at most t parties collude, an additional union bound over the t corrupted senders gives

$$\Pr[\textsf{any forge passes}] \;\le\; \frac{3t(m-1)}{|\mathbb{F}|} \;=\; \frac{3t(m-1)}{2^\kappa}.$$

Consequently, **Hyb10** and **Hyb9** have identical output distributions.

Hyb11. In this hybrid, during the *Reconstructing Column Polynomials* phase and for each honest party P_w, the simulator $\mathcal{S}$ does not re-verify the polynomials sent by honest providers $P_{o'}$ (row-only) and P_o (row+column). Instead, as soon as P_w receives $(\tilde{g}_o(y), \tilde{f}_o(x))$ and $\tilde{f}_{o'}(x)$, $\mathcal{S}$ treats them as accepted on P_w's behalf. Since the dealer D, P_w, $P_{o'}$, and P_o are honest, these polynomials are always correct and would be accepted under the original protocol as well. Therefore, the joint output distribution is unchanged, and **Hyb11** and **Hyb10** have identical output distributions.

Hyb12. In this hybrid, during the *Reconstructing Column Polynomials* phase, for each *corrupted* P_w and *honest* providers $P_{o'}$ (row-only) and P_o (row+column), the simulator $\mathcal{S}$ does not recompute the evaluation sets $\{f_{\ell,o'}(\alpha_w)\}_{\ell\in[m]}$ and $\{f_{\ell,o}(\alpha_w)\}_{\ell\in[m]}$. Instead, using the bivariate polynomials $\{F_\ell(x,y)\}_{\ell\in[m]}$ fixed earlier, it derives $\{g_{\ell,w}(\alpha_{o'})\}_{\ell\in[m]} = \{F_\ell(\alpha_w,\alpha_{o'})\}_{\ell\in[m]}$ and

$\{ f_{\ell,w}(\alpha_o) \}_{\ell\in[m]} = \{ F_\ell(\alpha_o, \alpha_w) \}_{\ell\in[m]}$, and then emulates $\mathcal{F}_{\mathsf{coin}}$ to deliver the corresponding messages to P_w. Because when $P_{o'}$ and P_o are honest, their polynomials are not used to determine anything during the *Sharing Phase*, deferring the generation of these honest evaluations—and producing them by restricting the already-fixed $\{F_\ell\}$—only changes the order of computation and not the induced distribution. Therefore, **Hyb12** and **Hyb11** have identical output distributions.

Hyb13. In this hybrid, during the *Completion Phase* and for each honest party P_w, the simulator $\mathcal{S}$ no longer computes P_w's output locally. Instead, once P_w has accepted Done_j from at least $2t{+}1$ distinct parties P_j, $\mathcal{S}$ lets P_w obtain the output directly from $\mathcal{F}_{\mathsf{ACSS}}$. This change does not affect correctness: the sets $\{f_{\ell,j}(\alpha_w)\}_{\ell\in[m]}$ contributed by each such P_j are accepted only if P_j is honest or if the values are consistent with the dealer-consistent bivariate polynomials fixed during the *Sharing Phase*. Hence, for every accepted j and every $\ell \in [m]$, $f_{\ell,j}(\alpha_w) = F_\ell(\alpha_j, \alpha_w) = g_{\ell,w}(\alpha_j)$, so P_w receives $2t{+}1$ evaluations at distinct points of each degree-$2t$ column polynomial $g_{\ell,w}(\cdot)$, which uniquely determines $g_{\ell,w}$ and therefore the final output. Since only the delivery mechanism is changed (not the distributed values themselves), the joint output distribution is unchanged. Consequently, **Hyb13** and **Hyb12** have identical output distributions.

Note that **Hyb13** corresponds to the ideal world: when the dealer D is honest, the protocol Π_{ACSS} is a statistically secure realization of the functionality $\mathcal{F}_{\mathsf{ACSS}}$.

When D is corrupted:

Simulator $\mathcal{S}$

Sharing Phase

1. For each $P_i \in \mathcal{P}$:
   1) If P_i is honest:
      i. When P_i receives $\{g_{\ell,i}(y)\}_{\ell\in[m]}$ and $\{f_{\ell,i}(x)\}_{\ell\in[m]}$ from D, if the degrees of these polynomials are $2t$ and t respectively, $\mathcal{S}$ broadcasts Yes_i on behalf of P_i.
      ii. $\mathcal{S}$ follows the protocol to compute $\boldsymbol{g_{\ell,i}} = (g_{\ell,i}(\alpha_1), \ldots, g_{\ell,i}(\alpha_n))$ for each $\ell \in [m]$.
      iii. $\mathcal{S}$ sends $(\mathsf{Init}, \mathsf{AMC}, T, (\boldsymbol{g_{1,i}}, \ldots, \boldsymbol{g_{m,i}}))$ to $\mathcal{F}_{\mathsf{AMC}}(P_i, D)$ on behalf of P_i.
      iv. $\mathcal{S}$ sends $(\mathsf{Init}, \mathsf{AMC}, T, (\boldsymbol{g_{1,i}}, \ldots, \boldsymbol{g_{m,i}}))$ to $\mathcal{F}_{\mathsf{AcceptPoly}}(P_i)$ on behalf of P_i.
      v. $\mathcal{S}$ faithfully emulates $\mathcal{F}_{\mathsf{AMC}}(P_i, D)$ and $\mathcal{F}_{\mathsf{AcceptPoly}}(P_i)$.
   2) If P_i is corrupted, $\mathcal{S}$ faithfully emulates $\mathcal{F}_{\mathsf{AMC}}(P_i, D)$ and $\mathcal{F}_{\mathsf{AcceptPoly}}(P_i)$.
2. For each honest party, upon this party receiving $\mathcal{M}$ from D, $\mathcal{S}$ follows the protocol to verify the $\mathcal{M}$ set. If $\mathcal{S}$ succeeds, he begins the next simulation phase of this honest party.

3. Let $\mathcal{H}$ be the first $t+1$ honest parties in $\mathcal{M}$. For each $\ell \in [m]$, $\mathcal{S}$ reconstructs a degree-$(t, 2t)$ bivariate polynomial $F_\ell(x, y)$ such that $F_\ell(\alpha_j, y) = g_{\ell,j}(y)$ for each $P_j \in \mathcal{H}$.
4. For each $\ell \in [m]$, $\mathcal{S}$ computes each corrupted P_j's $g_{\ell,j}(y) = F_\ell(\alpha_j, y)$. Then $\mathcal{S}$ sets $\tilde{\boldsymbol{g}}_{\boldsymbol{\ell},\boldsymbol{j}} = (g_{\ell,j}(\alpha_1), \ldots, g_{\ell,j}(\alpha_n))$ for each $\ell \in [m]$.

Fig. 31: Part-(1/4) of the simulator for the $\mathcal{F}_{\mathsf{ACSS}}$ when D is corrupted.

Simulator $\mathcal{S}$

Verification Phase

1. For each $P_i \in \mathcal{P}$:
   1) If P_i is honest:
      i. Upon receiving $\{g_{\ell,i}(y)\}_{\ell\in[m]}$ and $\{f_{\ell,i}(x)\}_{\ell\in[m]}$ from D with $\deg g_{\ell,i} = 2t$ and $\deg g_{\ell,i} = t$ for all $\ell \in [m]$, the simulator $\mathcal{S}$ broadcasts a random $r_i \xleftarrow{\$} \mathbb{F}$ and, on behalf of P_i, follows the protocol to derive the aggregate polynomial $g_i(y)$ and $f_i(x)$.
      ii. For each $P_h \in \mathcal{M}$, once every honest party has received r_i, $\mathcal{S}$ sends on that party's behalf the request $(\mathsf{Request}, \mathsf{AMC}, P_i, (r_i, r_i^2, \ldots, r_i^m))$ to $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$.
      iii. For each $P_h \in \mathcal{M}$, $\mathcal{S}$ faithfully emulates $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$.
      iv. $\mathcal{S}$ then performs the following checks:
         – Verify, for every $\ell \in [m]$, that $F_\ell(\alpha_i, y) = g_{\ell,i}(y)$ and $F_\ell(x, \alpha_i) = f_{\ell,i}(x)$ (as prescribed by the protocol).
         – Upon receipt of $\{\boldsymbol{g}_h\}_{P_h\in\mathcal{M}}$, carry out the protocol's consistency checks for $g_i(y)$.

      If both checks pass, $\mathcal{S}$ accepts $\{g_{\ell,i}(y)\}_{\ell\in[m]}$ and $\{f_{\ell,i}(x)\}_{\ell\in[m]}$; otherwise, it rejects.
   2) If P_i is corrupted:
      i. For each $P_h \in \mathcal{M}$, once every honest party has received r_i, $\mathcal{S}$ sends on that party's behalf $(\mathsf{Request}, \mathsf{AMC}, P_i, (r_i, r_i^2, \ldots, r_i^m))$ to $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$.
      ii. For each $P_h \in \mathcal{M}$, $\mathcal{S}$ faithfully emulates $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$.

Fig. 32: Part-(2/4) of the simulator for the $\mathcal{F}_{\mathsf{ACSS}}$ when D is corrupted.

Simulator $\mathcal{S}$

AcceptPoly Phase

1. For each honest $P_i \in \mathcal{P}/\mathcal{M}$:
   1) $\mathcal{S}$ sends $(\mathsf{Request}, \mathsf{coin}, \mathcal{P})$ to $\mathcal{F}_{\mathsf{coin}}$ on behalf of each honest party P_j, emulates $\mathcal{F}_{\mathsf{coin}}$, and delivers its output to all parties.
   2) For each honest party P_j and $P_h \in \mathcal{M}$, when P_j receives c_i, $\mathcal{S}$ sends $(\mathsf{Request}, \mathsf{AcceptPoly}, P_i, (c_i^1, \ldots, c_i^m))$ to $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$ on behalf of P_j.

3) For each honest $P_h \in \mathcal{M}$, $\mathcal{S}$ emulates $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$ to deliver an output to P_i. For each corrupted $P_h \in \mathcal{M}$, $\mathcal{S}$ faithfully emulates $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$.
4) When P_i has received the outputs of $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$ for all $P_h \in \mathcal{M}$, the simulator $\mathcal{S}$ considers P_i to have accepted $\Delta_{h\to i}(r)$ and begins simulating P_i in the next phase.

2. For each corrupted $P_i \in \mathcal{P}/\mathcal{M}$:
1) For each honest P_j and $P_h \in \mathcal{M}$, if $\mathcal{S}$ receives c_i on behalf of P_j, $\mathcal{S}$ sends $(\mathsf{Request}, \mathsf{AcceptPoly}, P_i, (c_i^1, \ldots, c_i^m))$ to $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$ on behalf of P_j.
2) For each $P_h \in \mathcal{M}$, $\mathcal{S}$ faithfully emulates $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$.

Fig. 33: Part-(3/4) of the simulator for the $\mathcal{F}_{\mathsf{ACSS}}$ when D is corrupted.

Simulator $\mathcal{S}$

Completion Phase

Reconstructing row polynomials:

2. For each $P_v \in \mathcal{P}$, $\mathcal{S}$ does the following things:
1) If P_v is honest:
i. For each $P_h \in \mathcal{M}$ sends $\{g_{\ell,h}(\alpha_i)\}_{\ell\in[m]}$ to P_v:
⋆ If $P_h \in \mathcal{M}$ is honest, when P_v receives a share, $\mathcal{S}$ accepts the shares on behalf of P_v.
⋆ If $P_h \in \mathcal{M}$ is corrupted, when P_v receives a share, $\mathcal{S}$ checks to see if it is the same as the share received in the previous phase.
ii. Upon P_v receiving $\{g_{\ell,h}(\alpha_i)\}_{\ell\in[m]}$ from $t+1$ different $P_h \in \mathcal{M}$, $\mathcal{S}$ computes $\{F_\ell(x, \alpha_v) = f_{\ell,v}(x)\}_{\ell\in[m]}$ on behalf of P_v.
2) If P_v is corrupted:
i. For each honest $P_h \in \mathcal{M}$, $\mathcal{S}$ sends $\{F_\ell(\alpha_h, \alpha_v)\}_{\ell\in[m]}$ to P_v.

Reconstructing column polynomials:

3. For each $P_w \in \mathcal{P}$, $\mathcal{S}$ does the following things:
1) If P_w is honest:
i. For each corrupted $P_j \in \mathcal{C}$, the simulator $\mathcal{S}$ computes the evaluations $\{ F_\ell(\alpha_j, \alpha_w) \}_{\ell\in[m]}$ and, on behalf of P_w, sends them to P_j.
ii. Let $P_{o'}$ denote any party that has accepted the *row* polynomials only, and P_o any party that has accepted both *column* and *row* polynomials. Then $P_{o'}$ sends $\{ \tilde{f}_{\ell,o'}(\alpha_w) \}_{\ell\in[m]}$ and P_o sends $\{ \tilde{f}_{\ell,o}(\alpha_w) \}_{\ell\in[m]}$ to P_w. The simulator handles these as follows:
– If $P_{o'}$ (resp. P_o) is honest, $\mathcal{S}$ accepts the received shares on behalf of P_w.
– If $P_{o'}$ (resp. P_o) is corrupted, $\mathcal{S}$ checks, for each $\ell \in [m]$, whether $\tilde{f}_{\ell,o'}(\alpha_w) = F_\ell(\alpha_w, \alpha_{o'})$ (resp. $\tilde{f}_{\ell,o}(\alpha_w) = F_\ell(\alpha_w, \alpha_o)$).

Collect all correct shares in per-index sets $\mathcal{W}_w = \{\mathcal{W}_{1,w}, \ldots, \mathcal{W}_{m,w}\}$. As soon as $|\mathcal{W}_{\ell,w}| \geq 2t{+}1$ holds for every $\ell \in [m]$, $\mathcal{S}$ broadcasts $\mathsf{Yes'}_w$ on behalf of P_w.

iii. $\mathcal{S}$ broadcasts $\mathsf{Yes'}_w$ on behalf of P_w.

iv. Upon receiving $\mathsf{Yes'}_j$ from at least $2t{+}1$ distinct parties P_j, $\mathcal{S}$ sends $(\mathsf{Request}, \mathsf{coin}, \mathcal{P})$ to $\mathcal{F}_{\mathsf{coin}}$ on behalf of P_w, emulates $\mathcal{F}_{\mathsf{coin}}$, and delivers its output to all parties.

v. Upon receiving $d \in \mathbb{F}$ from $\mathcal{F}_{\mathsf{coin}}$, the simulator computes

$$f_w(x) \;=\; \sum_{\ell=1}^{m} d^{\ell}\, F_{\ell}(x, \alpha_w) \qquad (\deg f_w = t),$$

and broadcasts $f_w(x)$ on behalf of P_w. If P_w has also accepted the column polynomials, it computes

$$g_w(y) \;=\; \sum_{\ell=1}^{m} d^{\ell}\, F_{\ell}(\alpha_w, y) \qquad (\deg g_w = 2t),$$

and broadcasts $g_w(y)$ as well.

vi. Upon receiving, from each party P_o that accepted both row and column polynomials, the pair $(\tilde{g}_o(y), \tilde{f}_o(x))$ together with the evaluations $\{\, \tilde{f}_{\ell,o}(\alpha_w) \,\}_{\ell \in [m]}$, and from each party $P_{o'}$ that accepted only row polynomials the polynomial $\tilde{f}_{o'}(x)$ together with $\{\, \tilde{f}_{\ell,o'}(\alpha_w) \,\}_{\ell \in [m]}$, $\mathcal{S}$ performs:
   - Degree checks: $\deg \tilde{g}_o = 2t$, $\deg \tilde{f}_o = t$, and $\deg \tilde{f}_{o'} = t$.
   - If $P_{o'}$ (resp. P_o) is honest, accept the received polynomial(s) on behalf of P_w.
   - If $P_{o'}$ is corrupted, recompute $f_{o'}(x)$ via the previously determined $F_{\ell}(x, y)$ and check $\tilde{f}_{o'}(x) = f_{o'}(x)$. If P_o is corrupted, similarly recompute $(g_o(y), f_o(x))$ and check equality.

   If all checks pass for a corrupted P_o, define

$$V(\alpha_o, y) \;:=\; g_o(y) \;+\; f_o(\alpha_o),$$

   and broadcast OK_{V_o} on behalf of P_w.

vii. Upon receiving OK_{V_o} from at least $2t{+}1$ distinct parties, $\mathcal{S}$ accepts $V(\alpha_o, y)$ on behalf of P_w.

2) If P_w is corrupted:

(i) For each *honest* party $P_{o'}$ that has accepted row polynomials and each *honest* party P_o that has accepted both column and row polynomials, $\mathcal{S}$ sends, on their behalf, the evaluation sets $\{\, f_{\ell,o'}(\alpha_w) \,\}_{\ell \in [m]}$ and $\{\, f_{\ell,o}(\alpha_w) \,\}_{\ell \in [m]}$ respectively to P_w.

(ii) For every honest P_j, $\mathcal{S}$ broadcasts $\mathsf{Yes'}_w$ on that party's behalf and sends $(\mathsf{Request}, \mathsf{coin}, \mathcal{P})$ to $\mathcal{F}_{\mathsf{coin}}$ on that party's behalf.

(iii) $\mathcal{S}$ emulates $\mathcal{F}_{\mathsf{coin}}$ and delivers its output to all parties.

(iv) For each honest P_o, $\mathcal{S}$ computes $f_o(x)$ and $g_o(y)$ and broadcasts them on behalf of P_o. For each honest $P_{o'}$, $\mathcal{S}$ computes $f_{o'}(x)$ and broadcasts it on behalf of $P_{o'}$.

(v) For every honest P_j, $\mathcal{S}$ broadcasts OK_{V_o} on that party's behalf.

4. $\mathcal{S}$ sets $\mathsf{idx} = (\ell-1)(t+1)+1$. For each $i \in [0,t]$, $\mathcal{S}$ computes $F_\ell(x, \alpha_{-i}) = q_{\mathrm{idx}+i}(x)$. In the end, $\mathcal{S}$ sends $(\mathsf{dealer}, \mathsf{ACSS}, \{q_1(\cdot), \ldots, q_N(\cdot)\})$ to $\mathcal{F}_{\mathsf{ACSS}}$ on behalf of D.
5. If P_w is honest, upon receiving Done_j from at least $2t{+}1$ distinct parties P_j, $\mathcal{S}$ allows $\mathcal{F}_{\mathsf{ACSS}}$ to send an output to P_w. $\mathcal{S}$ broadcasts Done_w on behalf of P_w.

Terminating: For each $P_k \in \mathcal{P}$, $\mathcal{S}$ does the following things: If P_k is honest, $\mathcal{S}$ terminates on behalf of P_k.

Fig. 34: Part-(4/4) of the simulator for the $\mathcal{F}_{\mathsf{ACSS}}$ when D is corrupted.

Hybrid arguments:

Hyb0. In this hybird, $\mathcal{S}$ behaves as in the real-world execution by using honest inputs and following the protocol correctly.

Hyb1. In this hybrid, during the *Sharing Phase* the simulator $\mathcal{S}$ performs one additional (bookkeeping) step. Let $\mathcal{H} \subseteq \mathcal{M}$ denote any set of $t{+}1$ honest parties. For each $\ell \in [m]$, $\mathcal{S}$ reconstructs a bivariate polynomial $F_\ell(x,y)$ (of the appropriate degree, e.g., at most $(t,2t)$) such that $F_\ell(\alpha_i, y) = g_{\ell,i}(y)$ for all $P_i \in \mathcal{H}$. Subsequently, for each corrupted party P_j, $\mathcal{S}$ defines the corresponding column polynomial by $\tilde{g}_{\ell,j}(y) := F_\ell(\alpha_j, y)$, and records its evaluation vector $\tilde{\boldsymbol{g}}_{\ell,j} = \big(\tilde{g}_{\ell,j}(\alpha_1), \ldots, \tilde{g}_{\ell,j}(\alpha_n)\big)$. Since $|\mathcal{H}| = t{+}1$, the x–marginal is uniquely determined and each F_ℓ is well-defined. However, these reconstructed polynomials are not used to influence any subsequent computation nor any message delivered to the environment or adversary. Hence this augmentation does not affect the joint output distribution. Therefore, **Hyb1** and **Hyb0** have identical output distributions.

Hyb2. During the *Verification Phase*, for each honest P_i and each round $\lambda \in [L']$, the simulator $\mathcal{S}$ forms the bivariate polynomial $F_i(x,y) := \sum_{\ell=1}^{m} r_i^\ell F_\ell(x,y)$. If P_i accepts his column polynomials $\{g_{\ell,i}(y)\}_{\ell\in[m]}$, then from these evaluations he can reconstruct a degree-$(t,2t)$ bivariate polynomial $\widetilde{F}_i(x,y)$ satisfying, for all $P_h \in \mathcal{M}$, $\widetilde{F}_i(\alpha_h, y) = \sum_{\ell=1}^{m} r_i^\ell g_{\ell,h}(y)$. $\mathcal{S}$ additionally checks whether $F_i(x,y) = \widetilde{F}_i(x,y)$. Whenever an honest P_i accepts his column polynomials, we have $F_i(\alpha_h, y) = \widetilde{F}_i(\alpha_h, y)$ for all $P_h \in \mathcal{M}$. Since $\mathcal{H} \subseteq \mathcal{M}$ with $|\mathcal{H}| = t{+}1$, these $t{+}1$ distinct x-lines uniquely determine the x-marginal of a degree-t bivariate polynomial, hence $F_i(x,y) = \widetilde{F}_i(x,y)$ identically. Therefore, this additional check never alters the behavior, and **Hyb2** and **Hyb1** have identical output distributions.

Hyb3. During the *Verification Phase*, for each honest P_i and every $\ell \in [m]$, the simulator $\mathcal{S}$ additionally checks whether $F_\ell(\alpha_i, y) = g_{\ell,i}(y)$. When P_i accepts his column polynomials, the combined relation $F_i(\alpha_i, y) = g_i(y)$ holds, where $F_i(x,y) = \sum_{\ell=1}^{m} r_i^\ell F_\ell(x,y)$ and $g_i(y) = \sum_{\ell=1}^{m} r_i^\ell g_{\ell,i}(y)$. The output distribution can change only if there exists some ℓ with $F_\ell(\alpha_i, y) \neq g_{\ell,i}(y)$ but nevertheless

$F_i(\alpha_i, y) = g_i(y)$ still holds due to the random choice of r_i. Let the discrepancy polynomials be $D_\ell(y) := F_\ell(\alpha_i, y) - g_{\ell,i}(y) = \sum_{j=0}^{2t} h_{\ell,j}\, y^j$, and define, for each $j \in \{0, \ldots, 2t\}$, the univariate polynomial in x $H_{i,j}(x) := \sum_{\ell=1}^{m} h_{\ell,j}\, x^\ell$. If some $D_\ell \not\equiv 0$, then there exists j with $H_{i,j} \not\equiv 0$ and $\deg H_{i,j} \le m$. The bad event $F_i(\alpha_i, y) = g_i(y)$ despite some $D_\ell \not\equiv 0$ is equivalent to $H_{i,j}(r_i) = 0$ for *all* $j = 0, \ldots, 2t$. Since r_i is sampled uniformly from $\mathbb{F}$ by the honest P_i, the union bound gives, for a fixed honest P_i,

$$\begin{aligned}\Pr\left[\exists\, \ell :\ F_\ell(\alpha_i, y) \neq g_{\ell,i}(y)\ \wedge\ F_i(\alpha_i, y) = g_i(y)\right] &\le \sum_{j=0}^{2t} \Pr[H_{i,j}(r_i) = 0] \\ &\le \frac{(2t+1)\, m}{|\mathbb{F}|}.\end{aligned}$$

Applying a union bound over at most $2t{+}1$ honest parties yields

$$\varepsilon_1 \ \le\ \frac{m(2t+1)^2}{|\mathbb{F}|} \ \le\ \frac{m\, n^2}{|\mathbb{F}|} \ =\ \frac{m\, n^2}{2^\kappa},$$

which is negligible in the security parameter κ. Therefore, the output distributions of **Hyb3** and **Hyb2** are statistically close.

Hyb4. In this hybrid, during the *AcceptShare* phase, whenever P_i is honest the simulator $\mathcal{S}$ skips re-verification and treats P_i as having received the correct shares once it has obtained the outputs of $\mathcal{F}_{\mathsf{AcceptShare}}(P_h)$ for all $P_h \in \mathcal{M}$. This change affects only the local checking procedure and does not modify any values delivered to the environment or the adversary; hence the joint output distribution is unchanged. Therefore, **Hyd5** and **Hyd4** have identical output distributions.

Hyb5. In this hybrid, during the *Reconstructing Row Polynomials* phase, for each honest party P_v the simulator $\mathcal{S}$ changes how it handles verification shares sent by honest providers P_h. Rather than re-verifying P_h's shares on behalf of P_v as prescribed by the protocol, $\mathcal{S}$ simply treats them as accepted upon receipt. Since P_h is honest, these shares are always valid and would be accepted under the original procedure as well. This modification affects only the local checking mechanism and not any values delivered to the environment or the adversary; hence the joint output distribution is unchanged. Therefore, **Hyd6** and **Hyd5** have identical output distributions.

Hyb6. In this hybrid, during the *Reconstructing Column Polynomials* phase, for each honest party P_w, the simulator $\mathcal{S}$ does not re-verify the shares sent by honest providers $P_{o'}$ (row-only) and P_o (row+column). Instead, as soon as P_w receives the shares $\{\, f_{\ell,o'}(\alpha_w)\,\}_{\ell\in[m]}$ and $\{\, f_{\ell,o}(\alpha_w)\,\}_{\ell\in[m]}$, $\mathcal{S}$ treats them as accepted on P_w's behalf. Since the dealer D, P_w, $P_{o'}$, and P_o are honest, these shares are always correct and would be accepted under the original protocol as well. Consequently, **Hyb6** and **Hyb5** have identical output distributions.

Hyd7 In this hybird, a corrupted $P_{o'}, P_o$ can only change the outcome if its altered shares $\{\tilde{f}_{\ell,\cdot}(\alpha_w)\}_{\ell\in[m]}$ make the recombined polynomials equal to the honest ones *after* the public coin d is sampled. The resulting difference in d is a nonzero polynomial of degree at most $m-1$, so by Schwartz–Zippel the success

probability per provider is at most $(m-1)/|\mathbb{F}|$. A union bound over the three equalities ($\tilde{f}_{o'}(x) = f_{o'}(x)$, $\tilde{f}_o(x) = f_o(x)$, $\tilde{g}_o(y) = g_o(y)$) gives $\Pr[\mathsf{forge\ passes}] \leq 3(m-1)/|\mathbb{F}| \leq 3m/|\mathbb{F}| = 3m/2^\kappa$, and with up to t corrupted providers it is $\leq 3t(m-1)/|\mathbb{F}|$. This is negligible in κ, so **Hyb7** and **Hyb6** have (statistically) indistinguishable output distributions.

Hyb8. In this hybrid, during the *Reconstructing Column Polynomials* phase, for each honest party P_w the simulator $\mathcal{S}$ does not re-verify polynomials sent by honest providers $P_{o'}$ (row-only) and P_o (row+column). Instead, as soon as P_w receives $(\tilde{g}_o(y), \tilde{f}_o(x))$ and $\tilde{f}_{o'}(x)$, $\mathcal{S}$ treats them as accepted on P_w's behalf. Since the dealer D and P_w are honest, $P_{o'}$ and P_o always send correct polynomials, so these values would be accepted under the original protocol as well. Consequently, **Hyb8** and **Hyb7** have identical output distributions.

Hyb9. In this hybrid, for each *corrupted* P_w and *honest* providers $P_{o'}$ (row-only) and P_o (row+column), the simulator $\mathcal{S}$ does not recompute $\{ f_{\ell,o'}(\alpha_w) \}_{\ell\in[m]}$ and $\{ f_{\ell,o}(\alpha_w) \}_{\ell\in[m]}$. Instead, using the bivariate polynomials $\{F_\ell(x,y)\}_{\ell\in[m]}$ fixed earlier, it returns $\{ g_{\ell,w}(\alpha_{o'}) \}_{\ell\in[m]} = \{ F_\ell(\alpha_w, \alpha_{o'}) \}_{\ell\in[m]}$, $\{ f_{\ell,w}(\alpha_o) \}_{\ell\in[m]} = \{ F_\ell(\alpha_o, \alpha_w) \}_{\ell\in[m]}$, and emulates $\mathcal{F}_{\mathsf{coin}}$ to deliver these values to P_w. Since when $P_{o'}$ and P_o are honest their polynomials do not influence any randomness in the *Sharing Phase*, producing these evaluations by restricting the already fixed $\{F_\ell\}$ merely changes the order of computation and not the distribution. Therefore, **Hyb9** and **Hyb8** have identical output distributions.

Hyb10. In this hybrid, during the *Completion Phase*, once an honest party has received and verified correct shares from all $P_h \in \mathcal{M}$, the simulator $\mathcal{S}$ reconstructs the polynomials $q_1(x), \ldots, q_N(x)$ and sends $\big(\mathsf{Dealer},\ \mathsf{ACSS},\ \{q_1(x), \ldots, q_N(x)\}\big)$ to $\mathcal{F}_{\mathsf{ACSS}}$. This modification merely delegates reconstruction and does not alter any values visible to the environment or the adversary; hence the joint output distribution is unchanged. Therefore, **Hyb10** and **Hyb9** have identical output distributions.

Hyb11. In this hybrid, during the *Completion Phase* and for each honest party P_w, the simulator $\mathcal{S}$ does not compute P_w's output locally. Instead, once P_w has accepted Done_j from at least $2t{+}1$ distinct parties P_j, $\mathcal{S}$ lets P_w obtain the output directly from $\mathcal{F}_{\mathsf{ACSS}}$. This change preserves correctness: each accepted set $\{f_{\ell,j}(\alpha_w)\}_{\ell\in[m]}$ is either supplied by an honest P_j or is consistent with the dealer-consistent polynomials fixed during the *Sharing Phase*. Thus, for every $\ell \in [m]$, $f_{\ell,j}(\alpha_w) = F_\ell(\alpha_j, \alpha_w) = g_{\ell,w}(\alpha_j)$, and P_w receives $2t{+}1$ evaluations at distinct points of each degree-$2t$ column polynomial $g_{\ell,w}(\cdot)$, which uniquely determines $\{g_{\ell,w}\}_{\ell\in[m]}$ and hence the final output. Since only the delivery mechanism is modified, the joint output distribution is unchanged. Therefore, **Hyb11** and **Hyb10** have identical output distributions.

Note that **Hyb11** corresponds to the ideal world: when the dealer D is corrupted, the protocol Π_{ACSS} is a statistically secure realization of the functionality $\mathcal{F}_{\mathsf{ACSS}}$.

E.3 Analysis of the Communication Complexity

We make a recap of the parameters in our Π_{ACSS}. D divides his N input polynomials into $m' = N/(t+1)$ groups. For each group, every $t+1$ polynomial will be

batched to be shared by a bivariate polynomial. Therefore, there is a bivariate polynomial for each group. Taking security into account, we add $T \cdot T' = 2n^2$ random groups, where $T = 2n$ and $T' = n$. As a result, there are $m = m' + T \cdot T'$ groups in total. Our $\mathcal{F}_{\mathsf{AMC}}$ will take m vectors as inputs and each vector is of length $L = n$.

1. During the Sharing Phase:
   1) For each group, D sends a degree-$2t$ column polynomial to each $P_i \in \mathcal{P}$, since there are m groups, resulting in a total communication of $\mathcal{O}(mn^2\kappa)$ bits for all parties.
   2) Then each $P_i \in \mathcal{P}$ broadcasts Yes_i (each Yes_i can be encoded with $\mathcal{O}(\log n)$ bits), the total communication is $\mathcal{O}(n^3 \log n)$ bits.
   3) Finally, D broadcasts set $\mathcal{M}$, which requires $\mathcal{O}(n^2 \log n)$ bits. Here we omit the communication of each $\mathcal{F}_{\mathsf{AMC}}$ and $\mathcal{F}_{\mathsf{AcceptShare}}$, and we will compute it later.

   Therefore, we need communication of $\mathcal{O}(mn^2\kappa + n^3 \log n)$ bits during the Sharing Phase.
2. During the Verification Phase:
   1) Each $P_i \in \mathcal{P}$ broadcasts a random element, resulting in a total communication of $\mathcal{O}(n^2\kappa + n^3 \log n)$ bits. The communication of each $\mathcal{F}_{\mathsf{AMC}}$ is still omitted.

   Therefore, we need $\mathcal{O}(n^2\kappa + n^3 \log n)$-bit communication during the Verification phase.
3. During the AcceptPoly Phase:
   1) Each online party generates a random element for every $P_i \in \mathcal{P}$, resulting in a total communication of $\mathcal{O}(n^4\kappa)$ bits. The communication of each $\mathcal{F}_{\mathsf{AcceptPoly}}$ is still omitted.

   Therefore, we need $\mathcal{O}(n^4\kappa)$-bit communication during the AcceptPoly phase.
4. During the Completion Phase:
   1) reconstructing row polynomials: for each $P_v \in \mathcal{P}$ and $P_h \in \mathcal{M}$,
      i. P_h sends $\{g_{\ell,h}(\alpha_v)\}_{\ell \in [m]}$ to P_v, which requires communication of $\mathcal{O}(m\kappa)$ bits.

      Therefore, reconstructing row polynomials requires $\mathcal{O}(mn^2\kappa)$ bits.
   2) reconstructing column polynomials: for each $P_w \in \mathcal{P}$
      i. Each P_w computes $\{f_{\ell,w}(\alpha_j)\}_{\ell \in [m]}$ for each $w \in [n]$, and sends them to party P_j, resulting in a total communication of $\mathcal{O}(mn^2\kappa)$ bits.
      ii. P_w broadcasts Yes'_w(each Yes'_w can be encoded with $\mathcal{O}(\log n)$ bits), which requires in a total communication of $\mathcal{O}(n^3 \log n)$ bits.
      iii. P_w sends $(\mathsf{Request}, \mathsf{RandCoin}, \mathcal{P})$ to $\mathcal{F}_{\mathsf{coin}}$, which requires in a total communication of $\mathcal{O}(n^3\kappa)$ bits.
      iv. P_w broadcasts $f_w(x)$. If P_w has also accepted the column polynomials, broadcasts $g_w(y)$ as well, which requires in a total communication of $\mathcal{O}(n^3\kappa + n^3 \log n)$ bits.
      v. P_w broadcasts OK_{V_o} (each OK_{V_o} can be encoded with $\mathcal{O}(\log n)$ bits), which requires in a total communication of $\mathcal{O}(n^3 \log n)$ bits.
      vi. P_w broadcasts Done_w (each Done_w can be encoded with $\mathcal{O}(\log n)$ bits), which requires in a total communication of $\mathcal{O}(n^3 \log n)$ bits.

Therefore, reconstructing column polynomials requires $\mathcal{O}(mn^2\kappa + n^3\kappa + n^3 \log n)$ bits.

Then, the total communication cost is $\mathcal{O}(mn^2\kappa + n^3\kappa + n^3 \log n)$ bits during the Completion Phase.

In the end, we consider the communication cost of Π_{AMC}. For each P_h in $\mathcal{M}$, according to Theorem 2, the communication of realizing each $\mathcal{F}_{\mathsf{AMC}}(P_h, D)$ is $\mathcal{O}(mn\kappa + Tn^3\kappa + n\kappa^2)$ bits. Since we have defined $m = m' + T \cdot T'$ and $T = 2n, T' = n$, the total communication cost is $\mathcal{O}(mn^2\kappa + n^5\kappa + n^2\kappa^2)$ bits.
Meanwhile, we consider the communication cost of $\Pi_{\mathsf{AcceptPoly}}$. For each P_h in $\mathcal{M}$, according to Theorem 3, the communication of realizing each $\mathcal{F}_{\mathsf{AcceptPoly}}(P_h)$ is $\mathcal{O}(mn\kappa + T'n\kappa)$ bits. Since we have defined $m = m' + T \cdot T'$ and $T = 2n, T' = n$, the total communication cost is $\mathcal{O}(mn^2\kappa + n^3\kappa)$ bits.
Therefore, the protocol Π_{ACSS} requires communication of $\mathcal{O}(mn^2\kappa + n^5\kappa + n^2\kappa^2)$ bits. Since $mn = (m' + T \cdot T') \cdot n = \mathcal{O}(N)$, when we take $m = n^3$, the protocol Π_{ACSS} requires communication of $\mathcal{O}(Nn\kappa + n^5\kappa + n^2\kappa^2)$ bits.